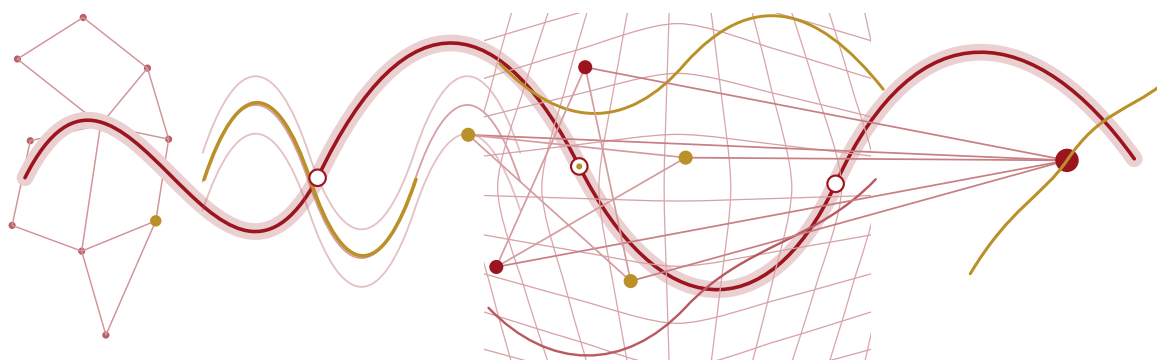


THE STUDENT MATHEMATICAL JOURNAL

CONSTRUCTOR UNIVERSITY



IN THIS ISSUE

Ahmed Maghri — CS'27

The Qubit and Shor's Algorithm

Noah Horne — PhDS'26

*Lagrangian Putt-Putt: Geodesic
Motion on C^2 -Surfaces*

Aly Assaf — ECE'27

Shannon's Theorem and Polar Codes

Pablo Santos Guerrero — MMDA'27

*A Classification of Principal
Divisors on $\mathbb{CP}^1$*

Taha Hbirri — CS'27

Reaching the Impossible Agreement

A Student Publication of
The Constructor University Mathematical Society

`math.constructor.university`
`constructor-math.github.io`

The Journal

The *Student Mathematical Journal* (**SMJ**) is the undergraduate mathematics journal of Constructor University Bremen, published annually by the Mathematics Society at Constructor University (**MSCU**).

The **SMJ** publishes expository and research articles written by undergraduate students. All articles originate in the **MSCU**'s own Mathematics Undergraduate Seminar (**UgS**) and are developed into self-contained expositions, accessible to a broad mathematical audience.

Cover Art

On the left, discrete structures and relations. In the middle, waves of analysis, probability, information, passing through a curved surface of geometry, physics, topology. On the right, paths converge into a shared network, different approaches meet in collective understanding. Gold marks the ultimate mathematical triumph.

Editorial Board

Editor-in-Chief

Omar Elshinawy — MATH'25

Managing Editor

Ahmed Maghri — CS'27

Faculty Advisors

Prof. Dr. Sören Petrat
Prof. Dr. Keivan Mallahi-Karai
Dr. Ivan Ovsyannikov
Prof. Dr. Nikolai Leopold

Staff

LaTeX Leads

Sarang Sule — MMDA'28
Ahmed Maghri — CS'27
Omar Elshinawy — MATH'25

Webmaster

Leyla Hajizada — CS'28

Submission Guidelines

We welcome submissions from students at Constructor University & visiting students who have participated in the Mathematical Undergraduate Seminar (**UgS**).

Each paper is reviewed by two **UgS** seminarists, or, by a faculty advisor when appropriate. Authors iterate with reviewers until the manuscript is suitable for final typesetting.

To participate in the **UgS**, there is an application process. One simply attends to a member of the **MSCU** board, possible through the respective contact channels, or closely follows the **MSCU**'s latest announcements.

Recorded Talks

Each article in the **SMJ** is accompanied by a recorded talk delivered at the **UgS**. The recordings are freely available and provide an accessible entry point to each paper's ideas.

elshenawyom.github.io/teaching/ugs
constructor-math.github.io/Subpages/ugs

Contact

Address.

Constructor University
Campus Ring 1
28759 Bremen
Germany

Email.

constructor.maths@gmail.com

Main Site.

<https://math.constructor.university>

Undergraduate Site.

<https://constructor-math.github.io>

Contents

On the SMJ's First Volume	i
<i>Omar Elshinawy — MATH'25</i>	
Editor's Address	ii
<i>Omar Elshinawy — MATH'25</i>	
 Student Articles	
The Qubit and Shor's Algorithm: A Step-by-Step Exploration	1
<i>Ahmed Maghri — CS'27 — UgS I</i>	
Shannon's Theorem and Polar Codes	15
<i>Aly Assaf — ECE'27 — UgS II</i>	
Lagrangian Putt-Putt: A Holistic Investigation into Holonomically Constrained Forced-Geodesic Motion on Arbitrary C^2-Surfaces	41
<i>Noah Horne — PhDS'26 — UgS III</i>	
A Classification of Principal Divisors on $\mathbb{CP}^1$	59
<i>Pablo Santos Guerrero — MMDA'27 — UgS V</i>	
Reaching the Impossible Agreement	81
<i>Taha Hbirri — CS'27 — UgS V</i>	

On the SMJ's First Volume

A Short Story

The Mathematical Undergraduate Seminar (**UgS**) is a tradition of student research and exposition, first introduced as part of the MATH programme. This tradition has persisted for years on, until the Spring of 2024. The programme was restructured into an MMDA — Mathematics, Modeling & Data Analytics — variant, and the seminar was discontinued.

As a seminarist myself, this tradition is encoded in Constructor (well, Jacobs...) University's mathematical DNA. A discontinuation in itself would be a step backwards for its mathematics society. So, we relaxed the curricular condition and took the seminar outside the classroom. The **UgS** was therefore reborn as a student-led initiative, attracting a superset of mathematics and non-mathematics students.

elshenawyom.github.io/teaching/ugs
constructor-math.github.io/Subpages/ugs

In its incarnation, the seminar is now a thriving hub of mathematical exploration, bringing this community closer than ever before. This journal is the fruit of this rebirth.

Acknowledgements

The **SMJ** expresses its sincere gratitude to the Mathematics faculty for advising the journal. In particular, we are indebted to Prof. Dr. Sören Petrat, Prof. Dr. Keivan Mallahi Karai, Dr. Ivan Ovsyannikov, and Prof. Dr. Nikolai Leopold for both their academic and moral contributions that shaped this volume.

To those who dared to believe,

- Aly Assaf ECE'27
- Loredana-Maria Iacob CBT'27
- Klesta Qehaja MMDA'27
- Pablo Santos Guerrero MMDA'27
- Vano Kakiashvili MMDA'27
- Mohamed Farouk PhDS'27

without your contributions to the **UgS**, the inauguration of this journal would not have been deemed possible.

Finally, we thank you (yes, you!) for reading, caring, and being part of this community. We look forward to seeing you in the next volumes of the **SMJ**.

Editor's Address

Dear reader,

It is with great pleasure that we present the inaugural edition of the *Student Mathematical Journal*, on behalf of the Constructor University Mathematics Society.

This journal was born from a desire to give undergraduate researchers a venue to communicate their work and to foster a culture of mathematical inquiry at Constructor University Bremen. Each article collected here originated in our *Mathematical Undergraduate Seminar* (UgS) and has been developed into an expository piece intended to be accessible to a wide audience.

In its very-first volume, the **SMJ** proudly presents to you five excellent articles spanning quantum computation, information theory, differential geometry, and algebraic geometry. Most remarkably, the authors hail from mathematics, physics, computer science, and electrical engineering. As I sign-off as Editor-in-Chief, I am excited to see how this journal evolves in the coming years.

To the students, I encourage you to get involved in this exciting venture that is the **UgS**. There you will always find a place to be curious, to ask questions, to make mistakes and learn from them, to grow as a mathematician, and most importantly, as a person.

To the seminarists, I invite you to pass on the torch to the next generation of students, to explore fascinating mathematics through the **UgS**, to share that same fascination through the **SMJ**.

To Ahmed Maghri CS'27, you entered the **SMJ** as a seminarist, and now you succeed me as the Editor-in-Chief. In between, I have watched you grow into the brilliant mathematician and capable leader that you are today. I trust you to uphold what came before and forge what comes next.

The **SMJ** is a testament to the mathematical excellence of those who contributed to it, and to the potential of those courageous enough to try. May it transcend the brevity of our time and echo softly in the spaces we leave behind. $\tilde{\zeta}$

Yours Sincerely,

Omar Elshinawy — MATH'25

Editor-in-Chief of the **SMJ**

The Qubit and Shor's Algorithm: A Step-by-Step Exploration

Ahmed Maghri

CS'27

Abstract. Shor's algorithm presents a polynomial-time quantum solution to the integer factorization problem, threatening classical encryption protocols like RSA. This article introduces the mathematical and computational foundations required to understand the algorithm, including modular arithmetic, continued fractions, and quantum principles. We provide a step-by-step breakdown of Shor's algorithm and its quantum subroutine, emphasizing accessibility and conceptual clarity, while also addressing practical constraints in current quantum hardware. This article aims to make the concepts of Shor's algorithm accessible to readers with limited background in mathematics and quantum computing, while maintaining mathematical rigor and conceptual depth throughout.

Acknowledgements. I would like to sincerely thank Omar Elshinawy, President of the Mathematics Society at Constructor University during the academic year 2024/2025, for his valuable guidance and support throughout the research and writing of this article. I also wish to thank Professor Keivan Mallahi-Karai for his thorough feedback and careful review of this manuscript, which significantly improved both its mathematical rigor and clarity. A heartfelt thanks goes to the Mathematics Society at Constructor University as a whole, whose enthusiasm and collaborative spirit made this research journey both enriching and enjoyable. Finally, and most importantly, I express my deepest gratitude to my parents, without whom none of this would have been possible.

0 A Note on Prerequisites

This article assumes no prior knowledge of quantum computing or advanced mathematics. Basic familiarity with the following concepts will be helpful: binary-to-decimal conversion (e.g., $101_2 = 4 + 0 + 1 = 5$), elementary complex numbers ($i^2 = -1$), and matrix operations such as transpose and complex conjugate. Any other concepts are introduced as they appear. Readers who wish to go deeper on any topic are encouraged to consult the references at the end of the article.

1 Introduction

This article takes the reader on a journey through Shor's algorithm. This journey will be structured as follows: We will start with introducing what is cryptography itself, explaining

its goal, RSA, and a brief introduction to the algorithm we will spend most time on, Shor's algorithm. We will then establish some essential background on quantum computing, so that the reader is able to follow the details of the explained algorithm. And finally, we will move to going over all of Shor's Algorithm, and how it is supposed to break RSA.

2 Introduction to Cryptography

Cryptography is the study of mathematical techniques for securing communication and data in the presence of adversaries. Its primary goal is to ensure the confidentiality, integrity, authenticity, and non-repudiation of information. We are mainly dealing with two subjects in it: Encryption & Decryption.

Definition 1

In cryptography, **encryption** (more specifically, **encoding**) is the process of transforming information in a way that, ideally, only authorized parties can decode.

2.1 RSA

The RSA cryptosystem was introduced in 1977 by Rivest, Shamir, and Adleman [3], from whom it takes its name. It is a public-key encryption scheme designed to securely encode messages over insecure communication channels by relying on the computational difficulty of factoring large composite integers. Let's discuss how it works and what is the algorithm behind it.

2.2 Algorithm of RSA

Take Alice and Bob. Alice and Bob both have two big prime numbers (in general around 313 digits). Let's name the prime numbers of Alice P_1 and P_2 , and Bob's P_3 and P_4 .

Both Alice and Bob are going to multiply their two big prime numbers to get a new big semiprime number. Let's call them respectively SP_1 and SP_2 .

Definition 2

A **symmetric** (private-key) cryptosystem uses a single secret key shared between sender and receiver — the same key encrypts and decrypts the message. The problem is that both parties must first agree on the key, which requires a secure channel to begin with. An **asymmetric** (public-key) cryptosystem solves this by using a pair of mathematically linked keys: a **public key**, shared openly and used to encrypt, and a **private key**, kept secret and used to decrypt. The two keys are linked such that encrypting with one can only be undone by the other. Crucially, deriving the private key from the public key is computationally infeasible.

P_1 and P_2 are then called the private keys of Alice, SP_1 is her public key. And similarly for Bob. If Alice wants to send something to Bob, she is going to hash her message in a way such that she uses his public key SP_2 , and the only way to decrypt her message, must be that her recipient knows the two big prime numbers P_3 and P_4 , that created the public key SP_2 . In our case, Bob knows them, but no one else does. Both the public keys of Alice and Bob can be viewed by anyone, as well as her hashed message. However, due to the fact that only Bob

knows what P_3 and P_4 are, he is the only one able to decode the hashed message.

Note 1. That we are not covering how the message is hashed, and we won't. This part is not relevant for the rest of the article and thus is out of scope.

Note 2. RSA is an asymmetric key system because two different keys are used to send and to receive the message.

2.3 How to Make RSA Irrelevant?

The whole RSA is standing on one assumption: finding the prime factors of a semiprime number is a hard problem. More precisely, no polynomial-time classical algorithm for integer factorization has ever been found, and the best known classical algorithms still run in sub-exponential but super-polynomial time. For a 313-digit semiprime number, even the most powerful supercomputers would take thousands of years to factor it. Thus, to break RSA, it is enough to find an efficient factoring algorithm — one that runs in polynomial time. So far, no one has been able to do so on a classical computer.

And this is where Shor's algorithm comes in.

Note 3. Many have probably heard that factoring a semiprime number is an NP problem. While it is true, it does not really say anything about RSA being strong or not. Without entering into too much detail, an NP problem is just a problem where one can verify if a solution is correct in polynomial time [1], which is indeed possible in prime factorization: one can check if the product of 2 prime numbers is equal to the semiprime number. But it does not say anything about whether a problem is in P or not — any problem in P (problems which we proved are solvable in polynomial time) is also in NP; summing $3 + 5$ is also an NP problem.

2.4 Introduction to Shor's Algorithm

Shor's algorithm, developed by Peter Shor in 1994 [4], is a quantum algorithm for efficiently factoring large integers. It runs in polynomial time on a quantum computer and poses a fundamental threat to classical cryptographic systems such as RSA, whose security relies on the hardness of integer factorization.

But before explaining how it works, you may have noticed that there is a quite scary word in its definition, "quantum". Thus, before explaining the algorithm and how it is implemented, for the sake of the reader's comfort, I will briefly introduce the world of Quantum Computing.

3 Introduction to the World of Quantum Computing

Note 4. Our goal in this article is not to focus on quantum computing, there are multiple lectures about it, and some really advanced topics will be needed during section 4.3 to explain the implementation of Shor's algorithm, and these concepts need their own article. If you feel a bit lost during the quantum computing parts, this is totally normal, but I am going to do my best to keep you on board.

Note 5. While reading this article, you may encounter some intermediary algorithms, while the article will still give a short explanation of them, again our goal won't be to go deeply into them. If you would like to know more about them, a quick Google search will serve you.

3.1 The Mathematics of Quantum States

Before diving into quantum computing, let us start from familiar ground. A classical computer processes information using **bits** — the smallest unit of information, which can take one of two values: 0 or 1. These bits are physically realized through the presence or absence of electrical signals. A classical register is simply a collection of bits, and all classical computation reduces to manipulating these 0s and 1s through logical operations.

Quantum computing builds on this foundation but introduces a fundamentally different model of information. Instead of bits, it uses **quantum bits**, or **qubits**.

We have all heard this, without really getting what is happening behind it. We also heard the famous sentence: While bits must be either in a 0 or 1 state, a qubit can be in a superposition of states $|0\rangle$ and $|1\rangle$. Some people also heard that when you measure a quantum state, you can't measure the superposition, you just randomly measure one of the possible states. At measurement, you can say that the quantum states collapse into exactly one. Let's try to discover what is really happening behind the scenes.

Formally, a quantum state (for the sake of the understanding we will first define a quantum state made by only 1 qubit; so a state "between" the $|0\rangle$ and the $|1\rangle$) is described by amplitudes α and β (where $\alpha, \beta \in \mathbb{C}$), with probability $|\alpha|^2$ of measuring 0, and probability $|\beta|^2$ of measuring 1. Thus, we have:

$$|\alpha|^2 + |\beta|^2 = 1$$

The randomness is indeed real. But it is not uniform as many people would think, it is explicitly governed by α and β . And we write for a state X :

$$|X\rangle = \alpha |0\rangle + \beta |1\rangle$$

For a state with 2 qubits, instead of 1, we get 2^2 possibilities:

$$|X\rangle = \alpha |00\rangle + \beta |01\rangle + \gamma |10\rangle + \delta |11\rangle$$

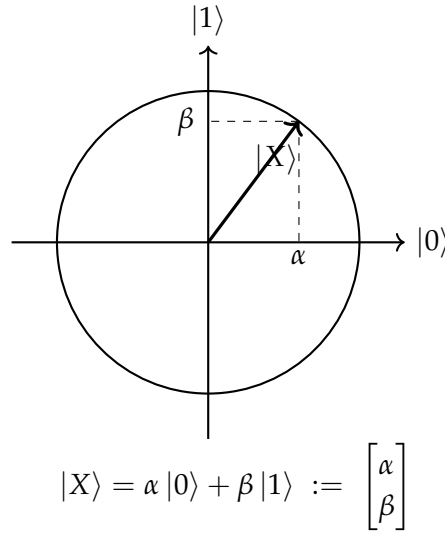
$$|\alpha|^2 + |\beta|^2 + |\gamma|^2 + |\delta|^2 = 1$$

As an example, consider the state Y where:

$$|Y\rangle = \frac{1}{2} |00\rangle + \frac{1}{2} |01\rangle + \frac{1}{2} |10\rangle + \frac{1}{2} |11\rangle$$

If we try to measure the state Y , we have equally $\frac{1}{4}$ probability of measuring either 00, 01, 10, or 11.

Now, a more sophisticated way to define quantum states is by looking at them as elements of the unit sphere (since $|\alpha|^2 + |\beta|^2 = 1$) of a vector space. Indeed, one can notice that for a state made by one qubit, it is basically a vector of $\mathbb{C}^2$, that has basis $\{|0\rangle, |1\rangle\}$. If we take the previous quantum state we mentioned, it would correspond to the following vector:



For a quantum state made by two qubits, the vector would live in $\mathbb{C}^2 \otimes \mathbb{C}^2$, with basis $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$.

$$|X\rangle = \alpha |00\rangle + \beta |01\rangle + \gamma |10\rangle + \delta |11\rangle := \begin{bmatrix} \alpha \\ \beta \\ \gamma \\ \delta \end{bmatrix}$$

In general, a system of n qubits lives in the space $(\mathbb{C}^2)^{\otimes n}$, which has dimension 2^n .

Note 6. The symbol $\otimes$ denotes the tensor product. On the surface, it looks similar to the Cartesian product: both produce ordered pairs. However, they differ in how scalar multiplication and addition behave.

In $X \times Y$:

- $c(x, y) = (cx, cy)$
- $(x, y) + (w, z) = (x + w, y + z)$

In $X \otimes Y$:

- $c(x \otimes y) = cx \otimes y = x \otimes cy$
- $(x + w) \otimes y = x \otimes y + w \otimes y$
- $x \otimes (y + z) = x \otimes y + x \otimes z$

As a consequence, elements like $x \otimes y + w \otimes z$ cannot be simplified further — and these are precisely the entangled states.

Now that we have established that quantum states are vectors living in $(\mathbb{C}^2)^{\otimes n}$, it is natural to ask: how do we transform them? The answer is through linear maps on these vector spaces, represented as matrices. In the quantum computing world, we call these **quantum gates**.

Definition 3

A **quantum gate** is a unitary operator acting on $(\mathbb{C}^2)^{\otimes n}$. Every quantum gate must be represented by a unitary matrix U , meaning its conjugate transpose satisfies $U^\dagger U = I$.

The unitarity condition is not arbitrary — it ensures that quantum gates are reversible, and that they preserve the normalization condition $|\alpha|^2 + |\beta|^2 = 1$: a valid quantum state must remain a valid quantum state after a transformation.

A simple first example is the **Pauli-X gate**, the quantum analogue of a classical NOT gate. It simply flips $|0\rangle$ to $|1\rangle$ and $|1\rangle$ to $|0\rangle$:

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

$$X|0\rangle = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix} = |1\rangle, \quad X|1\rangle = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} = |0\rangle$$

You can verify that $X^\dagger X = I$, confirming it is indeed unitary. With this foundation, we are ready to look at a less trivial — and far more powerful — gate: the Hadamard gate, which we will encounter in section 4.

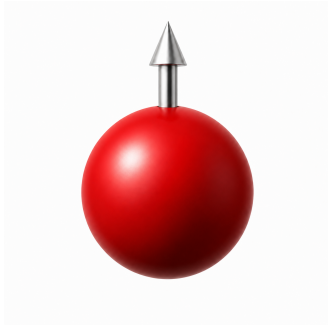
3.2 From Theory to Hardware: Building a Qubit

The physical implementation of qubits is a deep topic in its own right, and going into full detail would take us far from the scope of this article. That said, to give the reader a sense of grounding, we briefly sketch one of the most common approaches.

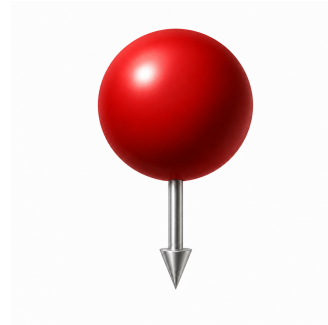
So how do we create such a computer? How do we create a qubit? Usually in a classical computer, a bit of 0 or 1 is created by using the “presence” or “absence” of electricity, but we cannot do it for quantum computers — there is no “electricity a bit present or a bit absent”.

We had to find a new way to describe the 0 and the 1 state.

A widely used method to build qubits is by using the **spin of an electron**, often the electron of a phosphorus atom embedded in silicon. The two possible spin orientations — **spin-up** and **spin-down** — are used to represent the states $|1\rangle$ and $|0\rangle$ respectively.



Spin-up: $|1\rangle$



Spin-down: $|0\rangle$

But in contrast to classical bits, the electron's spin can also be in a superposition of both up and down at once, meaning it's partly $|0\rangle$, partly $|1\rangle$.

This is what allows the qubit to behave fundamentally differently from a classical bit and makes quantum computing possible. Now that we have our foundations, we can move on to



Superposition: $\alpha |0\rangle + \beta |1\rangle$

the interesting part.

4 Shor's Algorithm: A Step-by-Step Walkthrough

Let's first state clearly the goal of the algorithm: **Given a semiprime number N , find the two prime numbers P_1 and P_2 such that: $P_1 \times P_2 = N$.**

The classical way to find these numbers is just by guessing randomly, also known as the brute force method. The essence or the main idea of Shor's algorithm is to translate the problem of integer factorization, into a problem of period finding, and to show that given a random guess g , this bad guess can be systematically transformed into a better one.

4.1 Initialization

We start by guessing randomly a number g such that $1 < g < N$. This leads to two cases:

Case 1: We got lucky. g is a factor of N , or shares common factors with N . In this case, we are done thanks to Euclid's algorithm.

Definition 4

Euclid's Algorithm is a classical method for efficiently computing the **greatest common divisor (GCD)** of two integers. It works by repeatedly replacing the larger number with its remainder when divided by the smaller, until the remainder is zero. The last non-zero remainder is the *GCD*.

Note 7. Using that, to find a factor of N , we don't need then to guess a factor, just to find a number that shares factors with N .

After finding one of the prime numbers, let's assume P_1 , compute N/P_1 and this will give you P_2 .

Case 2: Most probably, we won't get lucky. And this will make us start our process of making our bad guess a better one. To achieve that, we will borrow a quite helpful theorem.

Theorem 1

For any two whole numbers A and B that are coprime, there exists a power r and a multiple m such that:

$$A^r = m \cdot B + 1$$

Equivalently:

$$A^r \equiv 1 \pmod{B}$$

A good example to visualize it will be $A = 2$ and $P = 5$, we find that $r = 4$:

$$2^1 \equiv 2 \pmod{5} \quad 2^2 \equiv 4 \pmod{5} \quad 2^3 \equiv 3 \pmod{5} \quad 2^4 \equiv 1 \pmod{5}$$

In our case, our A is our bad guess g , and B is our semiprime number N that we want to factor, and ultimately, we know the existence of our r .

4.2 Making the Bad Guess a Better Guess

Let's assume that we found r , such that $g^r \equiv 1 \pmod{N}$. Bring 1 to the left and you get:

$$g^r - 1 \equiv 0 \pmod{N}$$

And using our famous identity: $(a + b)(a - b) = a^2 - b^2$:

Replace a by g , and b by 1, and going from the right side of the equality to the left, we get:

$$\underbrace{(g^{r/2} - 1)}_{P'_1} \cdot \underbrace{(g^{r/2} + 1)}_{P'_2} \equiv 0 \pmod{N}$$

Notice that r has to be even, otherwise, $r/2$ won't be a whole number. If we found an r odd, we repeat our algorithm from the beginning.

What we found now is actually really close to what we want, instead of finding the two numbers P_1 and P_2 such that $P_1 \times P_2 = N$, we found that N divides the product of two other numbers P'_1 and P'_2 . Which can be written as:

$$P'_1 \times P'_2 = mN, \quad m \in \mathbb{N}$$

Even more precisely, we found two numbers P'_1 and P'_2 that share factors with N . Thus:

$$\gcd(P'_1, N) = P_1 \text{ or } P_2. \text{ (We can replace } P'_1 \text{ by } P'_2, \text{ the outcome stays the same).}$$

And remember that thanks to our beautiful Euclid's algorithm, we are able to compute $\gcd(P'_1, N) = P_1$ easily. Compute then $\frac{N}{P_1} = P_2$, (and vice versa if we got that $\gcd(P'_1, N) = P_2$). And we have successfully found our P_1 and P_2 .

End of our algorithm.

Or maybe not?

You may feel that this is too easy, in the sense where, using this algorithm we could directly break the RSA, why are we using it? Remember also that I gave you an introduction about quantum computing, but I didn't talk at all about it at any point.

Let's recall the algorithm's main steps.

- 1 – Make a guess, $g < N$ that shares no factors with N .
- 2 – Find r such that $g^r = mN + 1$.
- 3 – If r is even, calculate $(g^{r/2} + 1)$ and $(g^{r/2} - 1)$. If r is odd, go back to step 1.
- 4 – Use Euclid's algorithm to find the greatest common divisor.

Where is the **quantum** here?

4.3 The Quantum Subroutine of Shor's Algorithm

Notice that we didn't discuss before how we are finding r , which is in fact the hardest (probably due to the quantum concepts) part of the whole algorithm.

Note 8. Through this part, the quantum part of the explanation (which is still the main thing) is going to be a bit simplified, essentially when we come to some deep notions such as Quantum Fourier Transform, etc. Such quantum concepts as said before need even their own whole article. And as again mentioned before, one of the primary goals of the article is to keep the reader on board. There will be some analogies to the classical architecture of a computer, for the sake of your understanding.

4.3.1 Initialization

To achieve this quantum subroutine, we will need two quantum registers, as normal registers that we have in normal computers, these ones are just going to contain qubits instead of normal bits.

First register: is going to be $2n$ qubits, initialized as:

$$|0\rangle^{\otimes 2n}$$

The number n just determines how accurate we want our calculations to be, however, the 2nd register size depends on the first one.

Second register is going to be n qubits, initialized to: $|1\rangle$

Note that $\otimes$ in context of quantum computing doesn't mean the XOR gate as a lot of people would think. It is a tensor product. Just consider it as a normal multiplication but between the quantum states.

Note also that for the first register, we mean that all the bits are set to 0, meaning that the state of that register is: $|000 \dots 000\rangle$.

But for the second register, we mean that the value of that register is one, meaning that the state of that register is: $|000 \dots 001\rangle$. It is just a story of notations.

4.3.2 Hadamard Transform

Let's first define how the Hadamard Transform works. The Hadamard gate is formally defined as this:

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

What does it do? Applying the Hadamard Transform to a $|1\rangle$ or $|0\rangle$ gives this:

$$\begin{aligned} H|0\rangle &= \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \\ H|1\rangle &= \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \end{aligned}$$

During this part, we will be interested essentially in its application on the 0 state, since our first register is just full of 0's.

Notice that on 0, it created exactly a state such that the probability of getting one of the states x such that $x \in \{0, \dots, 2^n - 1\}$ (converting the decimal to binary) is exactly equal.

Let's try to apply it to 0 states with 2 qubits.

$$H^{\otimes 2}|00\rangle = \frac{1}{2}(|00\rangle + |01\rangle + |10\rangle + |11\rangle)$$

And on 3 qubits?

$$H^{\otimes 3}|000\rangle = \frac{1}{\sqrt{8}}(|000\rangle + |001\rangle + |010\rangle + |011\rangle + |100\rangle + |101\rangle + |110\rangle + |111\rangle)$$

Did you notice the pattern? If not let me show it to you:

Whenever we apply the H gate to a register of size n , such that all its qubits are 0, we get a state such that the probability of measuring a state in $\{0, 1, 2, \dots, 2^n - 1\}$ is exactly equal for all of them.

You may wonder now why we are doing that, we will see why soon.

To end this part, we will formally say that the state of our first register now is:

$$\frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle$$

4.3.3 U

We will apply another transformation to the full two-register system. Let's call this transformation U . Formally, U acts on a pair of states $|x\rangle \otimes |y\rangle$ as follows:

$$U(|x\rangle \otimes |y\rangle) = |x\rangle \otimes |y \cdot g^x \bmod N\rangle$$

With g the random guess that we picked before. In other words, U reads the value x from the first register and uses it to update the second register: it multiplies the second register's value by g^x , modulo N .

How are we going to use U ? First let's look at our state now.

The quantum state of our full system (including the 1st and the 2nd register) is the tensor product $\otimes$ of both first and second register. As I said $\otimes$ is just a normal multiplication of the states. Our first register is now a sum of equal states, and our second register is still just in a 1 state. "Expanding" the expression:

$$\left(\frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} |x\rangle \right) \otimes |1\rangle$$

gives:

$$\frac{1}{\sqrt{2^n}} \sum_{x=0}^{2^n-1} (|x\rangle \otimes |1\rangle)$$

Don't overthink about it, it is the exact same way as you would expand $(a + b + c)d = ad + bd + cd$. For each x (the values in each state from 0 to $2^n - 1$), we apply the transformation x times on the qubit at state 1 associated to it. It is a bit hard to understand it like that, we can use a visualization to understand it more:

For each pair $|x\rangle \otimes |1\rangle$, we apply U getting:

$$|x\rangle \otimes |1\rangle \longrightarrow |x\rangle \otimes U^x |1\rangle = |x\rangle \otimes |g^x \bmod N\rangle$$

As an example, applying U to a state $|101\rangle \otimes |1\rangle$ (101 read as $4 + 0 + 1 = 5$) gives:

$$|101\rangle \otimes U^5 |1\rangle = |101\rangle \otimes |g^5 \bmod N\rangle$$

And again, g is our random guess that we picked before.

Note 9. $|x\rangle \otimes |y\rangle$ is the same as $|x\rangle |y\rangle$

Notice that when applying the transformation, we will get a certain periodicity in our quantum system due to modulo properties. To visualize it even better, let us consider that our x is going from 0 to 7 (using thus 3 qubits), that our N was 26, and our random guess was 3, after applying everything we had before, we will get:

$$\begin{aligned} & \frac{1}{\sqrt{8}} (|000\rangle |3^0 \bmod 26\rangle + |001\rangle |3^1 \bmod 26\rangle + |010\rangle |3^2 \bmod 26\rangle + \\ & |011\rangle |3^3 \bmod 26\rangle + |100\rangle |3^4 \bmod 26\rangle + |101\rangle |3^5 \bmod 26\rangle + \\ & |110\rangle |3^6 \bmod 26\rangle + |111\rangle |3^7 \bmod 26\rangle) \end{aligned}$$

Which exactly gives:

$$\begin{aligned} & \frac{1}{\sqrt{8}} (|000\rangle |1 \bmod 26\rangle + |001\rangle |3 \bmod 26\rangle + |010\rangle |9 \bmod 26\rangle + \\ & |011\rangle |1 \bmod 26\rangle + |100\rangle |3 \bmod 26\rangle + |101\rangle |9 \bmod 26\rangle + \\ & |110\rangle |1 \bmod 26\rangle + |111\rangle |3 \bmod 26\rangle) \end{aligned}$$

Notice also that the period is 3, which is exactly the number r that we are searching for, $3^3 \equiv 1 \pmod{26}$.

Thus, if we are able to extract the periodicity from the state, we would be done.

4.3.4 The Periodicity, a.k.a. r

This is actually a much harder task than expected. When you see the representation of the state in front of you it seems obvious, however, as we said before, if we just measure now our state it is just going to give us a random state since they all have equal probability. Which is not what we want. In fact, we are not able to extract the periodicity. However, we have something called the QFT (for **Q**uantum **F**ourier **T**ransform).

The QFT is the quantum analogue of the classical Discrete Fourier Transform (DFT), which is a well-known tool in signal processing for decomposing a signal into its frequency components. The DFT of a sequence $x_0, x_1, \dots, x_{N-1}$ is defined as:

$$X_k = \sum_{n=0}^{N-1} x_n \cdot e^{\frac{2\pi i k n}{N}}$$

The QFT is defined as:

$$\text{QFT} |x\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{\frac{2\pi i x k}{N}} |k\rangle$$

The analogy is clear: where the DFT maps a sequence of numbers to their frequency components, the QFT maps a quantum state $|x\rangle$ to a superposition of states $|k\rangle$, with amplitudes encoding the same exponential structure. Applied to our periodic quantum state, the QFT transforms the hidden period r into measurable frequency information. The QFT doesn't extract r directly — it extracts the frequency $1/r$, from which we recover r using the relation:

$$\text{Period} = \frac{1}{\text{frequency}}$$

A full treatment of the QFT would require its own article [2]. What matters for our purposes is the following: after applying the QFT, the amplitudes of our state are peaked near multiples of j/r , where j is an integer. This means that when we measure, we will most probably obtain a value close to j/r , from which we can extract r using the algorithm mentioned in the next section.

4.3.5 Continued Fraction Algorithm (CFA)

After obtaining j/r , extracting r from it is not straightforward. If during your measurement, you measured that $j/r = 0.312$, extracting the period from that is still a question that we need to answer.

For that, we use the **Continued Fraction Algorithm**. The best way to explain the CFA is to give an example.

Let's assume that we found that $j/r = 0.312$.

We start by writing it as a fraction:

$$0 + \frac{312}{1000}$$

We then reverse the fraction and get this:

$$= 0 + \frac{1}{\frac{1000}{312}} = 0 + \frac{1}{3 + \frac{8}{39}}$$

And we repeat the same process until we get a 1 in the numerator.

$$0.312 = 0 + \frac{1}{3 + \frac{1}{4 + \frac{1}{1 + \frac{1}{7}}}}$$

For each fraction, we get a better approximation for 0.312:

$$0.312 \approx \frac{1}{3}, \quad j = 1, \quad r = 3 \qquad 0.312 \approx \frac{1}{3 + \frac{1}{4}} = \frac{4}{13}, \quad j = 4, \quad r = 13$$

$$0.312 \approx \frac{1}{3 + \frac{1}{4 + \frac{1}{1}}} = \frac{5}{16}, \quad j = 5, \quad r = 16$$

For a more detailed treatment of the algorithm and its post-quantum implications, see [5].

We choose an approximation of r such that it follows two conditions:

- 1– r should be less than N
- 2– r should be an even number.

If none of our approximations gives a valid r , we repeat the same process from the beginning. And with that, our quantum subroutine comes to an end. Let us take a step back and appreciate what just happened. We started with a seemingly impossible problem: factoring a large semiprime N . Rather than attacking it directly, Shor's algorithm translated it into a period-finding problem — one that a quantum computer can solve efficiently. We made a random guess g , used the quantum subroutine to find its period r , and from that period extracted the prime factors of N using nothing more than Euclid's algorithm and some modular arithmetic.

Now that we have the full picture, we can revisit the main steps of the algorithm and fill in the missing piece:

- 1 – Make a guess, $g < N$ that shares no factors with N .
- 2 – Use the quantum subroutine to find r such that $g^r \equiv 1 \pmod{N}$:
 - Initialize two quantum registers: the first with $2n$ qubits in state $|0\rangle^{\otimes 2n}$, the second with n qubits in state $|1\rangle$.
 - Apply the Hadamard transform to the first register, creating an equal superposition of all states $|x\rangle$ for $x \in \{0, \dots, 2^n - 1\}$.
 - Apply U to the full system: $U(|x\rangle \otimes |y\rangle) = |x\rangle \otimes |y \cdot g^x \bmod N\rangle$.

- Apply the QFT to the first register to extract frequency information.
- Measure the first register to obtain j/r , then apply the CFA to extract r .

3 – If r is even, calculate $(g^{r/2} + 1)$ and $(g^{r/2} - 1)$. If r is odd, go back to step 1.

4 – Use Euclid's algorithm to find the greatest common divisor.

This is the full picture of Shor's algorithm. But a natural question remains: if this algorithm can break RSA, why is our data still safe today?

5 Final Thoughts: Does Shor's Algorithm Really Break RSA?

During this article, you went through all of Shor's algorithm and had a whole journey with it. I hope that I was able to make you enjoy the journey. You probably have a lot of questions in your mind now, and one of them would be that you saw with your own eyes that we have an algorithm that breaks our usual encryption. So how is our data safe?

Actually, we didn't truly break it. We are assuming that we have a quantum computer that is able to run Shor's algorithm, and we don't. Shor's algorithm would need 4100 logical qubits, or 20,000,000 physical qubits to run.

Currently, the main actor in this field is IBM. As of 2023, their most advanced processor — the Condor — reached 1,121 physical qubits, and by 2026 IBM plans to deploy a combined 4,158-qubit system through interconnected processors. Both figures remain far from the 20,000,000 physical qubits Shor's algorithm would require. But the day we are able to make a quantum computer with 20,000,000 physical qubits, we will definitely be able to break RSA, so the question is not if we are going to be able to, but **when**?

References

- [1] Sanjeev Arora and Boaz Barak. *Computational Complexity: A Modern Approach*. Cambridge University Press, 2009.
- [2] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 2010.
- [3] R. L. Rivest, A. Shamir, and L. Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978.
- [4] Peter W. Shor. Algorithms for quantum computation: Discrete logarithms and factoring. In *Proceedings 35th Annual Symposium on Foundations of Computer Science*. IEEE, 1994.
- [5] K. L. Watkins. An exposition on Peter Shor's polynomial-time factoring algorithm and its effects on post-quantum cryptography. Texas A&M University.

Shannon's Theorem and Polar Codes

Aly Assaf

ECE'27

Abstract. Reliable communication over a noisy channel requires redundancy, but adding redundancy lowers the communication rate. For the discrete memoryless channels considered in this paper, Shannon's noisy-channel coding theorem resolves this tension by identifying a capacity: at rates below capacity, families of increasingly long codes can make block-decoding errors arbitrarily unlikely, whereas rates above capacity cannot support reliable communication. The theorem establishes what is possible, but it does not itself provide an explicit and efficient code. This paper introduces polar codes as one structured response to that construction problem. Beginning with a repetition-code example, we develop the communication model, the binary symmetric channel, rate, Hamming distance, and linear codes. We then use typical noise patterns and binary entropy to motivate the capacity of the binary symmetric channel and introduce mutual information for more general channels. Finally, an information-theoretic preview and two- and four-bit binary erasure channel examples show how a recursive transform creates synthetic channels of unequal quality. Placing information in reliable positions and freezing the remaining positions to known values produces a polar code that can be encoded and decoded efficiently. Together, Shannon's theorem and polar codes illustrate the relationship between a theoretical communication limit and one explicit method for approaching it.

1 Introduction: How Can an Unreliable Channel Carry a Reliable Message?

1.1 A Message Damaged in Transit

Suppose Alice wants to send Bob one binary bit, either a 0 or a 1. The bit travels through some physical medium, such as a wire, an optical fiber, or a radio link. We will call this medium a *channel*. Real channels are imperfect: interference, a weak signal, or faults in the equipment may cause the bit to flip during transmission. Alice may send a 1 while Bob receives a 0.

The important problem is not simply that an error occurred. Bob sees only the received 0, so he cannot determine whether Alice sent a 0 correctly or whether she sent a 1 that was flipped. The received bit alone gives him no way to distinguish these two cases. How can Alice make her message easier to recover?

One simple idea is to repeat the bit three times:

$$0 \mapsto 000, \quad 1 \mapsto 111.$$

Suppose Alice wants to send 1, and therefore transmits 111. If noise changes the second bit, Bob receives 101. He can choose the value that occurs most often and recover the original 1. This procedure is called *majority decoding*, and the construction is called a *repetition code*.

The two additional bits do not add anything new to Alice's message. Instead, they provide *redundancy*: extra structure that helps Bob recognize and correct an error. The repetition code is not perfect. If two of the three transmitted bits flip, for example $111 \rightarrow 100$, majority decoding produces the wrong answer. Nevertheless, this small example demonstrates the basic purpose of coding theory: the channel may damage the transmitted object, yet a carefully chosen representation can allow the receiver to recover the original message.

1.2 The Price of Reliability

We corrected one possible error, but we paid for this protection. Alice used the channel three times to communicate only one bit of new information. A message of one thousand bits would become three thousand transmitted bits under the same repetition scheme. We could instead repeat each bit five, seven, or some larger odd number of times, making majority decoding more resistant to errors, but this would consume still more time, energy, and bandwidth.

This exposes the main trade-off in coding theory. We would like a code to be *reliable*, meaning that the probability that the final decoded message differs from the original message can be made very small. At the same time, we want to transmit information at a high rate rather than spending nearly every channel use on repeated data. Finally, the sender and receiver must be able to encode and decode without an unreasonable amount of computation.

The repetition code performs the necessary task, but it does so inefficiently. This naturally leads us to ask two questions. Can redundancy be arranged more intelligently, so that the probability of a decoding error becomes small without the transmission rate collapsing toward zero? If this is possible, what is the greatest rate that a particular noisy channel can support?

1.3 The Destination: Shannon's Answer and Arıkan's Construction

Claude Shannon gave a surprising answer in 1948. This paper focuses on discrete memoryless channels, a specific class that will be defined in Section 2, rather than attempting to cover every possible communication model. A channel in this class has a limiting value called its *capacity*. Informally, Shannon's noisy-channel coding theorem says the following [10]:

For the discrete memoryless channels considered in this paper, every communication rate below the channel capacity can be achieved by families of increasingly long codes whose probability of a decoding error approaches zero. If the communication rate is above capacity, reliable communication is impossible.

This statement does not say that noise disappears, nor does it promise that one fixed, finite code will never fail. Instead, it says that below capacity we can choose longer and more carefully designed codes so that decoding the wrong message becomes as unlikely as desired. The

theorem therefore gives a boundary between rates that can support reliable communication and rates that cannot.

Shannon's argument was mainly an existence proof. It showed that sufficiently good codes must exist, but it did not give engineers a direct and computationally efficient procedure for constructing and decoding them. Knowing the destination is not the same as knowing how to reach it. This left an important challenge: can we describe an explicit family of codes whose rate approaches capacity, whose probability of error approaches zero, and whose encoding and decoding remain manageable?

Polar codes, introduced by Erdal Arkan, provide a particularly clean answer for symmetric binary-input memoryless channels [1]. Their key idea is *channel polarization*: several identical uses of a noisy channel are combined and reorganized into synthetic channels that become increasingly reliable or increasingly unreliable. Information is placed in the reliable channels, while the unreliable channels are fixed to values known by both the sender and receiver.

We will build toward this idea gradually. We first formalize the communication process suggested by the repetition example and introduce rate, Hamming distance, and linear codes. We then study the binary symmetric channel and use entropy to understand Shannon's capacity limit. Finally, we return to the construction problem and use the binary erasure channel as a transparent worked example of polarization. This path takes us from the first question—whether reliable communication is possible—to an explicit method for approaching the limit Shannon discovered.

2 What a Simple Code Teaches Us

2.1 The Communication System

The repetition code from the introduction already contains every part of a basic communication system. Alice begins with a message, changes it into a form that can resist noise (encoding), and sends it through a channel. Bob observes the channel output and tries to reconstruct the message (decoding). We can summarize this process as

$$\mathbf{u} \xrightarrow{\text{encoder } E} \mathbf{x} \xrightarrow{\text{channel}} \mathbf{y} \xrightarrow{\text{decoder } D} \hat{\mathbf{u}}.$$

Suppose the message contains k bits. We write it as a vector $\mathbf{u} = (u_1, \dots, u_k) \in \{0, 1\}^k$. The encoder is a function

$$E : \{0, 1\}^k \longrightarrow \{0, 1\}^n$$

that maps $\mathbf{u}$ to an n -bit *codeword* $\mathbf{x} = E(\mathbf{u})$. The encoder must assign different codewords to different messages; otherwise, Bob could not distinguish those messages even if the channel introduced no errors. The set of all possible codewords is the *codebook*

$$\mathcal{C} = \{E(\mathbf{u}) : \mathbf{u} \in \{0, 1\}^k\}.$$

The notation $|S|$ means the number of elements in a finite set S . There are 2^k possible binary messages, so an encoder that assigns one distinct codeword to each message produces $|\mathcal{C}| = 2^k$ codewords. For the redundant codes studied here, $n > k$. The full space $\{0, 1\}^n$ then contains

2^n possible length- n words, which is much larger than the codebook. Nevertheless, only 2^k of those words can be valid codewords because Alice has only 2^k possible messages to encode:

$$\mathcal{C} \subsetneq \{0,1\}^n, \quad |\mathcal{C}| = 2^k < 2^n.$$

The remaining length- n words are not assigned to messages. This unused space is what allows valid codewords to be separated from one another and will become essential when we discuss minimum-distance decoding.

The channel may change some components of $\mathbf{x}$ and produce a received word $\mathbf{y} \in \{0,1\}^n$. The decoder is a function $D : \{0,1\}^n \rightarrow \{0,1\}^k$ that uses $\mathbf{y}$ to form an estimate $\hat{\mathbf{u}} = D(\mathbf{y})$. In triple repetition, for example, $k = 1$, $n = 3$, $E(0) = 000$, $E(1) = 111$, and D uses majority decoding.

It is useful to distinguish two kinds of error. A *bit error* occurs in the channel whenever some received bit y_i differs from the transmitted bit x_i . A *block-decoding error* occurs when Bob's final estimate $\hat{\mathbf{u}}$ differs from the entire original message $\mathbf{u}$. These events are not the same: the received word 101 contains one bit error relative to 111, but majority decoding still recovers the correct message.

2.2 A Precise Noise Model: The Binary Symmetric Channel

Real channels may suffer from many different forms of interference, but a simpler model lets us isolate the main coding problem. In the *binary symmetric channel* with crossover probability p , written $\text{BSC}(p)$, both the input X and the output Y are bits. The channel flips the input with probability p and leaves it unchanged with probability $1 - p$.

An economical way to write the model is

$$Y = X \oplus Z, \quad Z \sim \text{Bern}(p), \tag{1}$$

where $\oplus$ denotes XOR, or addition modulo 2. Thus $X \oplus 0 = X$ and $X \oplus 1$ is the opposite bit. The notation $Z \sim \text{Bern}(p)$ means

$$\Pr(Z = 1) = p, \quad \Pr(Z = 0) = 1 - p.$$

The noise bit Z therefore records whether a flip occurred.

For an n -bit codeword, we use the channel n times and write

$$\mathbf{y} = \mathbf{x} \oplus \mathbf{z}.$$

Here the XOR is applied component by component. The noise bits $Z_1, \dots, Z_n$ are assumed to be independent, each with the same probability p of equaling 1. In other words, knowing what happened to an earlier bit does not change the probabilities for the current bit. This is the *memoryless* assumption. The channel is called *symmetric* because a 0 is flipped to 1 with the same probability that a 1 is flipped to 0.

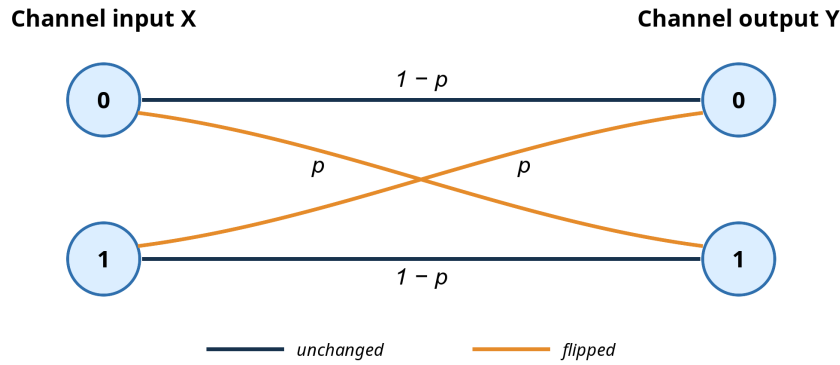


Figure 1: A $\text{BSC}(p)$. Straight arrows represent an unchanged bit with probability $1 - p$, while crossing arrows represent a flip with probability p .

Why do we normally restrict p to the range $0 \leq p \leq 1/2$? If $p = 0$, the channel is perfect. If $p = 1/2$, the two possible outputs are equally likely no matter which bit Alice sends, so Bob receives no clue about the input. Finally, if $p > 1/2$, Bob can flip every received bit himself. The resulting bit is wrong with probability $1 - p < 1/2$, making the original channel equivalent to an easier one in our chosen range. Notice that this explanation uses only the behavior of the channel; we do not need a definition of information yet.

The BSC is one example of a *discrete memoryless channel*: a channel with finite input and output alphabets whose output probabilities depend only on the current input. We will use the BSC for our discussion of Shannon's theorem. Later, the binary erasure channel will provide a more transparent worked example of polarization.

2.3 Rate and the Redundancy Trade-Off

The encoder begins with k message bits and produces n transmitted bits. The *rate* of the code is

$$R = \frac{k}{n}. \quad (2)$$

It measures the number of message bits carried per transmitted bit. Uncoded transmission has $n = k$ and rate $R = 1$, but it adds no protection. Triple repetition uses three channel inputs for every message bit, so $n = 3k$ and $R = 1/3$.

This is the price of redundancy: when more transmitted bits are used to represent the same message, the rate decreases. However, a lower rate does not automatically produce a better code. Compare the repetition code

$$0 \mapsto 000, \quad 1 \mapsto 111$$

with the equally long encoding

$$0 \mapsto 000, \quad 1 \mapsto 001.$$

Both codes have rate $1/3$, but a single flip in the final position can change 000 into 001. The second code therefore cannot reliably correct even one error. What matters is not only how

much redundancy is added, but also how that redundancy separates the possible codewords. Because $n > k$, the encoder has 2^n available locations for only 2^k codewords; a good encoder uses this extra room to keep the codewords apart. Hamming distance makes this idea precise.

2.4 Hamming Distance and Minimum-Distance Decoding

The codewords 000 and 111 differ in all three positions. This count is their *Hamming distance*. In general, the Hamming distance between two binary words of the same length is

$$d_H(\mathbf{x}, \mathbf{x}') = |\{i : x_i \neq x'_i\}|.$$

The *Hamming weight* $\text{wt}(\mathbf{x})$ is the number of 1s in $\mathbf{x}$. Equivalently,

$$d_H(\mathbf{x}, \mathbf{x}') = \text{wt}(\mathbf{x} \oplus \mathbf{x}').$$

The minimum distance of a code is the smallest distance between two distinct codewords:

$$d_{\min} = \min_{\substack{\mathbf{x}, \mathbf{x}' \in \mathcal{C} \\ \mathbf{x} \neq \mathbf{x}'}} d_H(\mathbf{x}, \mathbf{x}'). \quad (3)$$

For a linear code over a finite field $\mathbb{F}_q$, the notation $[n, k, d]_q$ records its codeword length n , message dimension k (original message length), and minimum distance d and base q . Such a code contains q^k codewords. For a binary linear code, $q = 2$ and the subscript is often omitted, giving $[n, k, d]$. The next subsection will illustrate finite-field arithmetic and linearity in the binary case. Triple repetition is therefore a $[3, 1, 3]_2$ code, or simply a $[3, 1, 3]$ binary linear code.

Distance also suggests a decoding rule. Suppose Alice transmits 111 and Bob receives 101. We calculate

$$d_H(101, 111) = 1, \quad d_H(101, 000) = 2.$$

In general, a *maximum-likelihood decoder* chooses the candidate codeword that makes the observed channel output most probable:

$$\hat{\mathbf{x}}_{\text{ML}} = \arg \max_{\mathbf{c} \in \mathcal{C}} \Pr(\mathbf{Y} = \mathbf{y} \mid \mathbf{X} = \mathbf{c}).$$

The calculation becomes especially simple on a BSC(p). If a candidate codeword $\mathbf{c}$ lies at Hamming distance a from $\mathbf{y}$, then exactly those a positions would have needed to flip, an event with probability

$$p^a (1 - p)^{n-a}.$$

Because $p < 1 - p$, a candidate requiring fewer flips is more likely than one requiring more flips. Therefore, on the BSC, maximum-likelihood decoding is exactly *minimum-distance decoding*:

$$\hat{\mathbf{x}} = \arg \min_{\mathbf{c} \in \mathcal{C}} d_H(\mathbf{y}, \mathbf{c}).$$

For the received word 101, this rule selects 111. The equivalence is a special property of the BSC and other symmetric error models; for a general channel, the closest word in Hamming distance need not be the most likely one.

Although Hamming space is discrete rather than Euclidean, schematic circles can help us visualize its decoding regions. The *Hamming ball* of radius t around a word $\mathbf{x}$ contains every binary word at distance at most t from $\mathbf{x}$. For the repetition code, the two radius-one balls are

$$\begin{aligned} B_1(000) &= \{000, 001, 010, 100\}, \\ B_1(111) &= \{111, 110, 101, 011\}. \end{aligned}$$

They do not overlap. Thus every received word containing at most one error lies in a unique ball and can be corrected to its center.

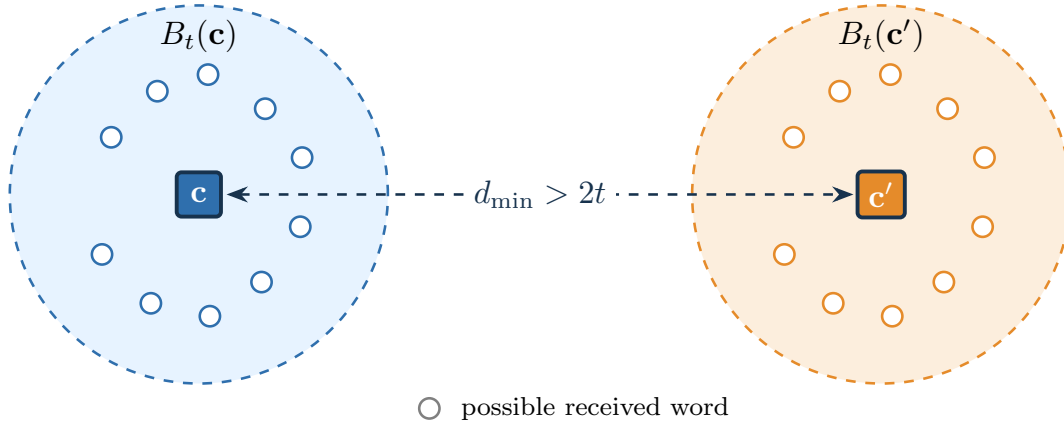


Figure 2: Two discrete decoding regions of radius t in Hamming space. The squares are codewords, and the small circles are possible received words. If the minimum distance satisfies $d_{\min} > 2t$, a received word caused by at most t errors belongs to at most one such region.

This picture gives two general guarantees. A code with minimum distance d is guaranteed to

$$\text{detect any pattern of up to } d - 1 \text{ errors,} \quad \text{correct any pattern of up to } \left\lfloor \frac{d - 1}{2} \right\rfloor \text{ errors.}$$

The floor notation $\lfloor r \rfloor$ means the greatest integer less than or equal to r . It appears here because the number of errors must be a whole number. The detection statement follows because fewer than d flips cannot turn one valid codeword into another. For correction, the radius- t balls around distinct codewords must be disjoint, which requires $2t < d$. Triple repetition, with $d = 3$, can therefore detect up to two errors or correct up to one error. By contrast, the alternative code $\{000, 001\}$ has minimum distance 1 and has no guaranteed error correction despite using the same number of transmitted bits. Redundancy provides room to separate codewords, but the encoder must use that room well.

2.5 Why Linear Structure Is Useful

Listing every codeword and comparing each one with the received word is one direct way to perform minimum-distance decoding. This is possible for triple repetition, but it becomes impractical when k is large because the codebook contains 2^k words. A useful code therefore needs a concise structure. Polar codes belong to the important family of *linear codes*, whose encoding can be described by matrix multiplication.

The entries of these matrices belong to the binary field $\mathbb{F}_2 = \{0, 1\}$. Addition in $\mathbb{F}_2$ is XOR, so $1 + 1 = 0$, while multiplication follows the usual binary rules. A binary code $\mathcal{C} \subseteq \mathbb{F}_2^n$ is linear if

it is a vector subspace. Concretely, the all-zero word belongs to $\mathcal{C}$, and the XOR of any two codewords is another codeword.

x_1	x_2	$x_1 \oplus x_2$
0	0	0
0	1	1
1	0	1
1	1	0

Figure 3: Truth table for XOR, the addition operation in $\mathbb{F}_2$. Its output is 1 exactly when the two input bits differ.

A linear encoder can be specified by a full-row-rank $k \times n$ *generator matrix* G . Treating the message as a row vector, encoding becomes

$$\mathbf{x} = \mathbf{u}G, \quad (4)$$

where every operation is performed in $\mathbb{F}_2$. For triple repetition,

$$G = \begin{bmatrix} 1 & 1 & 1 \end{bmatrix}.$$

The message $u = 0$ produces $0G = 000$, while $u = 1$ produces $1G = 111$. The same description that previously required two separate encoding rules has become one matrix operation.

Linear codes can also be described by a *parity-check matrix* H . A word $\mathbf{x}$ is a valid codeword exactly when $H\mathbf{x}^T = \mathbf{0}$. If G and H describe the same code, then $HG^T = 0$. Parity-check matrices are valuable in many coding methods, but the polar-code construction later in this paper will be easier to follow through its generator matrix.

The first step of that construction will combine two bits according to

$$(u_1, u_2) \mapsto (u_1 \oplus u_2, u_2).$$

This operation is itself linear, since

$$\begin{bmatrix} u_1 & u_2 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} u_1 \oplus u_2 & u_2 \end{bmatrix}.$$

We have therefore developed the language needed to describe codes and their reliability. What remains is the larger question posed in the introduction: for a channel such as the BSC, how high can the rate be while the decoding error still becomes increasingly unlikely?

3 Shannon's Answer: The Capacity of the Binary Symmetric Channel

Section 2 showed how a finite code can trade rate for distance. This raises the question: What is the best possible trade-off? In other words, what is the maximum rate we can achieve while

still communicating reliably?

Reliability in this context is a property of a family of codes: the probability of a block-decoding error approaches zero as the codeword length n grows. For a BSC(p) with $p < 1/2$, repeating a single bit more and more times is reliable in this sense, but its rate approaches zero. We want to know whether reliable families of codes can instead maintain a positive rate. Shannon's answer for the BSC is the destination of this section.

Suppose we use a BSC(p) n times, where n is large. An encoder chooses a codebook $\mathcal{C}$ containing $|\mathcal{C}| = 2^k$ codewords from the 2^n possible binary strings, giving rate

$$R = \frac{k}{n} = \frac{\log_2 |\mathcal{C}|}{n}. \quad (5)$$

Theorem 3.1: Simplified noisy-channel coding theorem for the BSC

For a BSC(p) with $0 \leq p < 1/2$, the capacity is $C_{\text{BSC}} = 1 - h_2(p)$, where the binary entropy function $h_2(p)$ will be defined and motivated in Subsection 3.2. Then:

1. If $R < C_{\text{BSC}}$, there exists a family of increasingly long codes whose rates are at least R and whose block-error probabilities approach zero.
2. If $R > C_{\text{BSC}}$, no family whose rates remain at least R can have its block-error probability approach zero.

This simplified statement makes no claim about the boundary case $R = C_{\text{BSC}}$.

Theorem 3.1 tells us what we are working toward. To see why this particular threshold appears, return to the length- n output space. For a fixed n , increasing R places more codewords into that space, while noise creates a cloud of plausible received words around each one. How many such clouds can the space hold before reliable decoding becomes impossible?

Sections 3.1–3.3 give a brief proof sketch rather than a complete proof of Shannon's theorem. Their purpose is to explain where the BSC capacity formula comes from. The argument has three steps:

1. Determine which noise patterns are typical when n is large. These patterns form the high-probability "cloud" around a transmitted codeword.
2. Count the typical patterns in one cloud. The binary entropy function $h_2(p)$, defined in Subsection 3.2, will show that there are roughly $2^{nh_2(p)}$ of them.
3. Compare the size of one cloud with the 2^n possible received words. This packing comparison suggests how many distinguishable codewords can fit and therefore which rates should be reliable.

This calculation will predict the threshold $1 - h_2(p)$ and give the main intuition behind both sides of Shannon's result. It is not by itself a rigorous proof: finite noise clouds may overlap, rare noise patterns remain possible, and an existence argument must show that suitable codebooks can actually be chosen. The theorem above states the result we are trying to understand; after

the sketch, Subsection 3.4 introduces the language needed to express channel capacity beyond the BSC.

3.1 What Typical Noise Looks Like

The first step is to describe the likely noise cloud around one transmitted codeword. For each channel use, let $Z_i = 1$ if the i th bit is flipped and $Z_i = 0$ otherwise. The BSC assumptions tell us that

$$Z_1, \dots, Z_n \text{ are independent,} \quad Z_i \sim \text{Bern}(p). \quad (6)$$

The total number of flips is

$$K_n = Z_1 + \dots + Z_n, \quad (7)$$

which has a binomial distribution and expected value $\mathbb{E}[K_n] = np$. This does not mean that exactly np bits will flip. Instead, the law of large numbers says that the fraction K_n/n becomes increasingly concentrated near p . For every fixed $\varepsilon > 0$,

$$\Pr\left(\left|\frac{K_n}{n} - p\right| > \varepsilon\right) \rightarrow 0 \quad \text{as } n \rightarrow \infty. \quad (8)$$

We will call a noise vector $\mathbf{z} \in \{0,1\}^n$ *typical* when its fraction of 1s is close to p . More precisely, for a chosen tolerance ε , define

$$\mathcal{T}_\varepsilon^{(n)} = \left\{ \mathbf{z} \in \{0,1\}^n : \left| \frac{\text{wt}(\mathbf{z})}{n} - p \right| \leq \varepsilon \right\}. \quad (9)$$

Equation (8) says that the actual BSC noise lies in this set with probability approaching 1. Thus a transmitted word $\mathbf{x}$ is very likely to produce an output in the translated set $\mathbf{x} \oplus \mathcal{T}_\varepsilon^{(n)}$. Rare patterns outside this region remain possible, but their total probability becomes small.

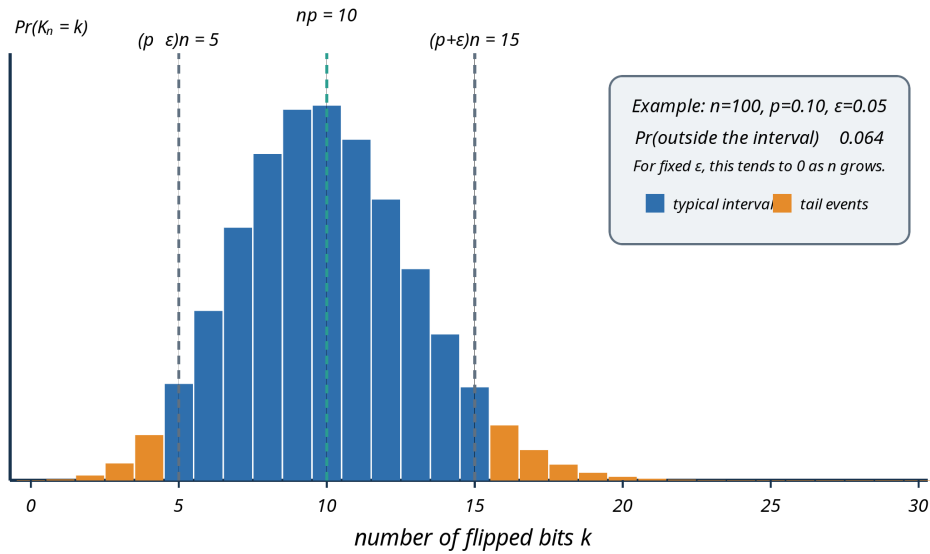


Figure 4: For $n = 100$ and $p = 0.10$, the binomial number of flipped bits K_n concentrates near its mean np . The highlighted interval is $[(p - \varepsilon)n, (p + \varepsilon)n]$ with $\varepsilon = 0.05$.

3.2 Counting Likely Noise Patterns: Binary Entropy

The first step told us which noise patterns make up the likely cloud around a codeword. The second step is to count them. Before doing so, it helps to attach a numerical meaning to uncertainty. If an outcome a has probability $P(a)$, its *self-information* is

$$i(a) = \log_2 \frac{1}{P(a)} = -\log_2 P(a). \quad (10)$$

An unlikely outcome carries more self-information because observing it is more surprising. An outcome with probability 0.99, for example, carries little self-information because it was already strongly expected.

The average self-information of a random variable is its *entropy*. For example, suppose you flip a fair coin once. Let X represent the coin outcome, with $X = 1$ for heads and $X = 0$ for tails. A fair coin therefore has the Bernoulli distribution $X \sim \text{Bern}(1/2)$, meaning

$$\Pr(X = 1) = 0.5, \quad \Pr(X = 0) = 0.5. \quad (11)$$

Its entropy is the probability-weighted average of the two outcomes' self-information:

$$H(X) = -[0.5 \log_2 0.5 + 0.5 \log_2 0.5] = 1 \text{ bit}. \quad (12)$$

Thus, before the toss is observed, the fair coin has 1 bit of uncertainty.

If the coin is biased, say

$$\Pr(X = 1) = 0.9, \quad \Pr(X = 0) = 0.1, \quad (13)$$

then its entropy is

$$H(X) = -[0.9 \log_2 0.9 + 0.1 \log_2 0.1] \approx 0.47 \text{ bits}. \quad (14)$$

The biased coin has less uncertainty because one outcome is much more predictable.

The BSC noise indicator is also a Bernoulli random variable: it equals 1 with probability p and 0 with probability $1 - p$. Applying the same average-surprise calculation gives the *binary entropy function*

$$h_2(p) = -p \log_2 p - (1 - p) \log_2 (1 - p), \quad (15)$$

where $0 \log_2 0$ is interpreted as 0. Figure 5 shows how the binary entropy changes with p .

We can now return to the counting problem. A length- n noise vector with exactly w flips can be formed by choosing which w positions contain a 1. There are

$$\binom{n}{w} \quad (16)$$

such vectors. When w is close to np , Stirling's approximation—an approximation for factorials when the numbers involved are large—gives, on the exponential scale,

$$\binom{n}{\lfloor np \rfloor} \approx 2^{nh_2(p)}. \quad (17)$$

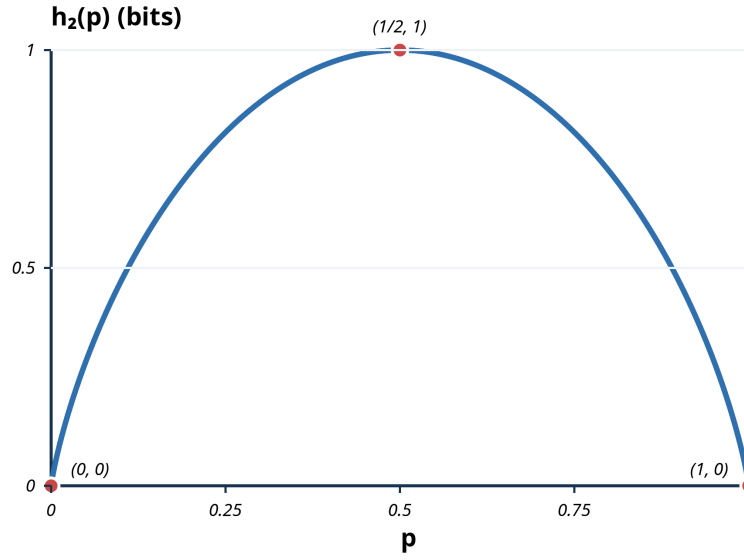


Figure 5: The binary entropy function $h_2(p)$, which reaches its maximum value 1 at $p = 1/2$.

Here $\lfloor np \rfloor$ rounds np down to a whole number so that it can represent a possible number of flipped bits. For a small tolerance ε , the exponential growth rate of the entire typical set is close to $h_2(p)$, and it approaches $h_2(p)$ as the tolerance is tightened. Factors that grow only polynomially with n are hidden by the approximation. The important conclusion is that one codeword has roughly $2^{nh_2(p)}$ likely noise patterns around it on the exponential scale.

3.3 From Counting to the BSC Capacity

The first two steps showed that one codeword has roughly $2^{nh_2(p)}$ likely outputs after passing through the BSC; that is, its likely-output cloud has roughly this many elements. To see why, fix a codeword $\mathbf{c} \in \mathcal{C}$. Equation (17), together with the typical-set discussion following it, shows that there are roughly $2^{nh_2(p)}$ relevant noise patterns $\mathbf{z}$ on the exponential scale. Each one produces a received word $\mathbf{y} = \mathbf{c} \oplus \mathbf{z}$. Thus $2^{nh_2(p)}$ is the approximate size of the likely-output cloud around one codeword.

The final step compares all these clouds with the entire output space. The factor $|\mathcal{C}|$ is the number of valid codewords, $2^{nh_2(p)}$ is the approximate number of likely outputs per codeword, and 2^n is the total number of length- n received words. If the likely-output clouds are to remain distinguishable, their combined size should not exceed the available output space. This gives the packing estimate

$$|\mathcal{C}| 2^{nh_2(p)} \lesssim 2^n. \quad (18)$$

Here $\lesssim$ reminds us that this is an exponential-scale estimate rather than an exact finite-length inequality. If the left-hand side were substantially larger than 2^n , many likely-output clouds would have to overlap, making different transmitted codewords difficult to distinguish. Solving Equation (18) for the codebook size gives

$$|\mathcal{C}| \lesssim \frac{2^n}{2^{nh_2(p)}} \quad (19)$$

$$= 2^{n(1-h_2(p))}. \quad (20)$$

Starting from Equation (20), take the base-2 logarithm of both sides. Because $\log_2$ is an increasing function, the direction of the estimate is preserved:

$$\log_2 |\mathcal{C}| \lesssim \log_2 \left(2^{n(1-h_2(p))} \right) \quad (21a)$$

$$= n(1 - h_2(p)). \quad (21b)$$

Dividing both sides by n gives

$$\frac{\log_2 |\mathcal{C}|}{n} \lesssim 1 - h_2(p). \quad (21c)$$

Equation (5) identifies the expression on the left as the code rate, so

$$R \lesssim 1 - h_2(p). \quad (21d)$$

The packing sketch therefore suggests $1 - h_2(p)$ as the limiting rate. Shannon's theorem makes this prediction rigorous and identifies it as the *capacity* of the BSC(p):

$$C_{\text{BSC}} = 1 - h_2(p) \quad \text{bits per channel use.} \quad (21e)$$

For example, if $p = 0.1$, then

$$C_{\text{BSC}} = 1 - h_2(0.1) \quad (22)$$

$$\approx 1 - 0.469 \quad (23)$$

$$= 0.531. \quad (24)$$

For a block length $n = 1000$, the output space contains 2^{1000} words and each codeword has on the order of 2^{469} typical noise patterns. Their ratio is approximately 2^{531} , suggesting about 531 message bits per 1000 channel uses at the limiting rate. This is an asymptotic counting estimate, not a guarantee that a particular length-1000 code can transmit 531 bits with negligible error.

This argument is counting intuition, not yet a proof. The Hamming balls in Section 2 described a worst-case guarantee: every pattern of up to t errors had to be corrected. A typical region describes only the errors that carry almost all of the probability, and rare errors outside it are allowed to cause decoding failures. Furthermore, a rigorous coding proof does not require every finite-length typical region to be perfectly disjoint. It shows instead that their total ambiguity can be made increasingly unlikely when the rate is below the threshold.

3.4 From the BSC to General Channels

The expression $1 - h_2(p)$ is specific to the BSC, but Shannon's theorem applies more broadly to discrete memoryless channels. We now introduce the language that expresses how much information one random variable provides about another.

For a discrete random variable X , its *alphabet* $\mathcal{X}$ is the set of values it can take, and $P_X(x) = \Pr(X = x)$ is the probability that it takes a particular value x . Its *entropy* is the average self-information

$$H(X) = - \sum_{x \in \mathcal{X}} P_X(x) \log_2 P_X(x). \quad (25)$$

Entropy measures uncertainty before an observation is made, in bits. After observing one

particular output $Y = y$, the remaining uncertainty about the input is

$$H(X | Y = y) = - \sum_{x \in \mathcal{X}} P_{X|Y}(x | y) \log_2 P_{X|Y}(x | y), \quad (26)$$

where $P_{X|Y}(x | y) = \Pr(X = x | Y = y)$ is the probability of input x after output y has been observed. Different outputs can leave different amounts of uncertainty. The *conditional entropy* averages Equation (26) over every possible output y in the output alphabet $\mathcal{Y}$:

$$H(X | Y) = \sum_{y \in \mathcal{Y}} P_Y(y) H(X | Y = y), \quad (27)$$

where $P_Y(y) = \Pr(Y = y)$. The difference between the uncertainty before and after observing the output is the *mutual information*

$$I(X; Y) = H(X) - H(X | Y). \quad (28)$$

It measures how much observing Y reduces our uncertainty about X .

Return to the $\text{BSC}(p)$ and suppose Alice uses 0 and 1 equally often. As Figure 5 shows, $H(X) = 1$ bit. Given the output Y , the input equals that output with probability $1 - p$ and differs from it with probability p . Therefore,

$$H(X | Y) = h_2(p), \quad (29)$$

and hence

$$I(X; Y) = 1 - h_2(p). \quad (30)$$

This reproduces the capacity suggested by our counting argument.

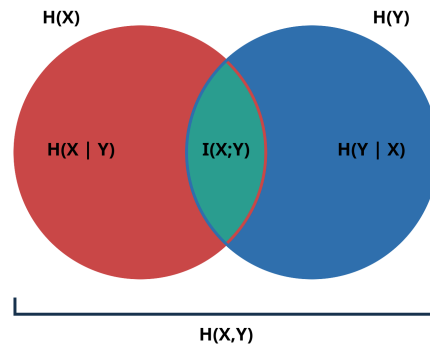


Figure 6: A heuristic information diagram. The overlap represents $I(X; Y)$, while the non-overlapping portions represent $H(X | Y)$ and $H(Y | X)$. Entropy is not literally a geometric area in general.

Interchanging the roles of the variables defines $H(Y | X)$ in the same way. Figure 6 also uses $H(X, Y)$ for the entropy of the pair (X, Y) considered jointly. The polar transform will give us a reason to define joint entropy and its chain rule formally in Subsection 4.1.

For a general discrete memoryless channel, different input distributions may carry different

amounts of information. Its capacity is therefore defined by

$$C = \max_{P_X} I(X; Y), \quad (31)$$

where the maximum is taken over all probability distributions P_X on the channel input alphabet. The symmetry of the BSC makes the uniform input distribution optimal, so Equation (31) reduces to $C = 1 - h_2(p)$ in that case.

Shannon's theorem answers a fundamental question: it tells us which communication rates are possible. However, a limit is not yet a design. Knowing that reliable codes exist below capacity does not tell Alice and Bob which code to use, how to generate its codewords, or how Bob should decode without an enormous search. We must therefore change our question. Instead of asking only, "What rates are possible?", we now ask, "How can we explicitly build and efficiently decode codes that approach those rates?"

4 Polar Codes: Creating Good and Bad Synthetic Channels

Shannon's theorem proves that reliable codes exist below capacity, but it does not hand us a specific codebook or an efficient decoder. Polar codes address this construction problem with a recursive transformation introduced by Erdal Arıkan. We will first study how the two-bit operator redistributes uncertainty, then place it around actual uses of a binary erasure channel. After applying the operator recursively to four bits, we will generalize the construction, select information and frozen positions, describe the decoder, and state what the polar-coding theorem guarantees.

4.1 The Two-Bit Transform: An Information-Theoretic Preview

A useful way to motivate the transformation is to follow in Arıkan's footsteps and attempt the compression of two bits, which we will call U and V . If we keep only their XOR, $(U, V) \rightarrow U \oplus V$, the operation is not reversible: several input pairs produce the same output. We therefore keep a second value that is linearly independent of $U \oplus V$, choosing a copy of V . The resulting transform is

$$(U, V) \longrightarrow (U \oplus V, V). \quad (32)$$

Given $U \oplus V$ and V , we recover $U = (U \oplus V) \oplus V$, so no information is lost. Before placing noisy channels around this operator, we will study how it redistributes uncertainty between two successive tasks: first determining $U \oplus V$, then determining V after the first value is known. This information-theoretic view observes the behavior produced by the polarization operator; the BEC example in the next subsection will connect that behavior to what a receiver can actually decode from noisy outputs.

Although this transformation does not compress the pair, it can redistribute the uncertainty between its two coordinates. To see this in a simple setting, let U and V be independent and identically distributed Bernoulli random variables with

$$\Pr(U = 1) = \Pr(V = 1) = q, \quad 0 \leq q \leq \frac{1}{2}. \quad (33)$$

The parameter q describes the distribution of these input variables; it is not the BSC crossover

probability p . This calculation concerns the uncertainty in the operator's inputs before any noisy outputs are introduced. It is not yet a calculation of a synthetic channel's capacity; choosing $0 < q < 1/2$ simply makes the operator's redistribution of entropy visible.

Statement 1. For $0 < q < 1/2$, the probability $q' = \Pr(U \oplus V = 1)$ is greater than q .

The event $U \oplus V = 1$ occurs when exactly one of the two input bits equals 1. The two possible cases are mutually exclusive, while the random variables U and V are independent. Therefore,

$$q' = \Pr(U \oplus V = 1) \quad (34a)$$

$$= \Pr(U = 1, V = 0) + \Pr(U = 0, V = 1) \quad (34b)$$

$$= q(1 - q) + (1 - q)q \quad (34c)$$

$$= 2q(1 - q). \quad (34d)$$

The difference is $q' - q = q(1 - 2q)$, which is positive for $0 < q < 1/2$. At the endpoints $q = 0$ and $q = 1/2$, the difference is zero.

Statement 2. For $0 < q < 1/2$, the transformation separates the two entropies:

a) The first coordinate becomes more uncertain: $H(U \oplus V) > H(U) = H(V)$.

b) Once $U \oplus V$ is known, the second coordinate becomes less uncertain: $H(V | U \oplus V) < H(U) = H(V)$.

Recall that $h_2(q)$ is the entropy of a Bernoulli random variable with parameter q . Because $q' > q$ and both values lie between 0 and $1/2$, $h_2(q') > h_2(q)$. This gives the first inequality.

For the second inequality, we need to compare the uncertainty in the complete pair before and after the transform. The *joint entropy* $H(U, V)$ is the entropy of the pair (U, V) considered as one random object. If $\mathcal{U}$ and $\mathcal{V}$ are the alphabets of U and V , and $P_{U,V}(u, v) = \Pr(U = u, V = v)$ is their joint probability, then

$$H(U, V) = - \sum_{u \in \mathcal{U}} \sum_{v \in \mathcal{V}} P_{U,V}(u, v) \log_2 P_{U,V}(u, v). \quad (35)$$

Because U and V are independent, their joint entropy is the sum of their individual entropies:

$$H(U, V) = H(U) + H(V) = 2h_2(q). \quad (36)$$

The transform is reversible, so the transformed pair contains exactly the same total uncertainty. We now need to separate that total into the uncertainty in the first coordinate and the uncertainty remaining in the second once the first is known. The *chain rule for entropy* states that, for any two discrete random variables A and B ,

$$H(A, B) = H(A) + H(B | A). \quad (37)$$

Applying this rule to the transformed pair gives

$$H(U \oplus V, V) = H(U \oplus V) + H(V | U \oplus V) = 2h_2(q). \quad (38)$$

Since $H(U \oplus V) = h_2(q')$, we obtain

$$H(V | U \oplus V) = 2h_2(q) - h_2(q') < h_2(q). \quad (39)$$

The first coordinate has therefore become more uncertain, while the second becomes less uncertain once the first is known. From this information-theoretic perspective, the polarization operator has redistributed uncertainty without changing its total amount. Figure 7 shows the transform.

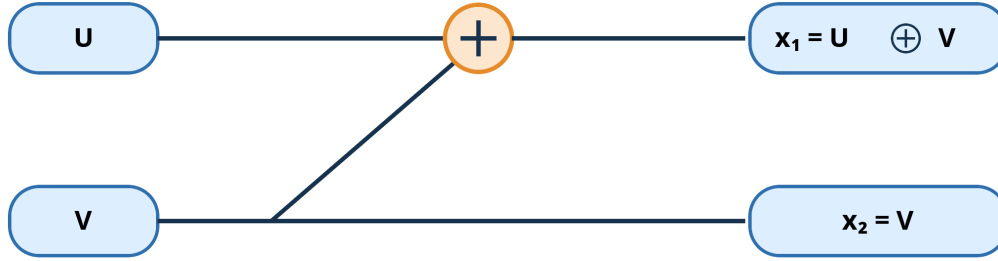


Figure 7: The reversible transform $(U, V) \rightarrow (U \oplus V, V)$. The circled plus denotes XOR.

The entropy calculation gives an information-theoretic preview of the operator: one coordinate becomes more uncertain, the other becomes less uncertain once the first is known, and their total uncertainty is preserved. It does not yet include noisy outputs observed by a receiver. The next subsection places the same transform around two uses of a binary erasure channel, derives the resulting decoding reliabilities, and then applies the calculation recursively to four bits.

4.2 A Worked Example: The Binary Erasure Channel

We will now place the transform around actual noisy channel uses. We begin with a two-bit system because one transform is enough to create one less reliable decoding task and one more reliable decoding task. After deriving their reliability formulas, we will apply the same formulas again in a four-bit system to see the recursive effect.

A *binary erasure channel* with erasure probability ε , written $\text{BEC}(\varepsilon)$, has input $X \in \{0, 1\}$ and output $Y \in \{0, 1, ?\}$. Its behavior is

$$Y = \begin{cases} X, & \text{with probability } 1 - \varepsilon, \\ ?, & \text{with probability } \varepsilon. \end{cases} \quad (40)$$

The channel never changes a 0 into a 1 or a 1 into a 0. Instead, the symbol “?” tells the receiver exactly which positions were erased. We assume that erasures occur independently from one channel use to the next, so the channel is memoryless. An unerased output reveals its input bit completely, while an erased output reveals nothing about that bit. The average information delivered per use, and therefore the capacity, is

$$C_{\text{BEC}} = 1 - \varepsilon \quad \text{bits per channel use.} \quad (41)$$

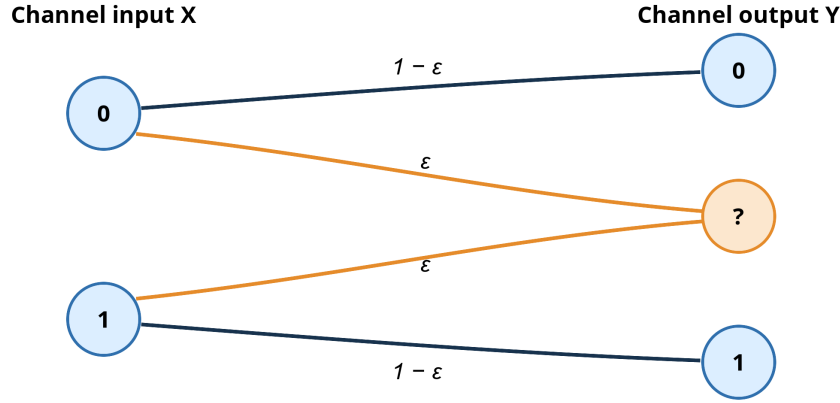


Figure 8: A $\text{BEC}(\varepsilon)$. The channel delivers the input unchanged with probability $1 - \varepsilon$ and outputs the erasure symbol “?” with probability ε .

4.2.1 The Two-Bit Building Block

We start with the smallest nontrivial system, which has block length $n = 2$. Alice has two independent input bits u_1 and u_2 . For the capacity calculation, each is *equally likely*: independently, each bit equals 0 with probability $1/2$ and 1 with probability $1/2$.

The encoder transforms the two inputs into two transmitted bits, and each transmitted bit passes through one independent use of the BEC. The receiver observes y_1 and y_2 and then estimates the original inputs. The complete two-bit communication chain is

$$\begin{aligned}
 (u_1, u_2) &\xrightarrow{\text{polar encoder}} (x_1, x_2), \\
 (x_1, x_2) &\xrightarrow{\text{two independent BEC uses}} (y_1, y_2), \\
 (y_1, y_2) &\xrightarrow{\text{successive decoder}} (\hat{u}_1, \hat{u}_2).
 \end{aligned} \tag{42}$$

There are two bit positions throughout this system, not six separate information bits. The symbols u_i , x_i , and y_i describe the input, transmitted value, and observation at each position as it moves through the chain.

First, we encode. Applying Equation (32) gives

$$x_1 = u_1 \oplus u_2, \quad x_2 = u_2. \tag{43}$$

This transform is reversible, so it has not created two additional bits or discarded either input. Alice sends x_1 and x_2 through the two BEC uses, and each observation y_i is either the corresponding x_i or the erasure symbol “?”.

Let W denote the original physical channel $\text{BEC}(\varepsilon)$. Combining two independent uses of W with the polar transform creates two *synthetic channels*, meaning two successive decoding tasks built from the same pair of observations. The first, W^- , recovers u_1 while u_2 is unknown. The second, W^+ , recovers u_2 with u_1 as side information. The superscripts $-$ and $+$ are branch labels, not arithmetic exponents. A successive decoder actually supplies its earlier estimate $\hat{u}_1$; the erasure calculation below assumes that estimate is correct. Both tasks use y_1 and y_2 , so

neither x_1 nor x_2 alone corresponds to W^- or W^+ .

We will now derive a formula for the erasure probability of each task. The two physical BECs entering this transform both have erasure probability ε .

For the minus task, the decoder needs both observations. If y_1 and y_2 are present, it can compute $u_1 = y_1 \oplus y_2$. If either observation is erased, one of the required values is missing. The erasure probability is therefore

$$\varepsilon^- = 1 - \Pr(y_1 \neq ? \text{ and } y_2 \neq ?) \quad (44a)$$

$$= 1 - (1 - \varepsilon)^2 \quad (44b)$$

$$= 2\varepsilon - \varepsilon^2. \quad (44c)$$

For the plus task's reliability calculation, u_1 is correctly known. The decoder can recover u_2 directly from y_2 , or it can compute $u_2 = u_1 \oplus y_1$. Thus this synthetic channel is erased only when both observations are erased:

$$\varepsilon^+ = \Pr(y_1 = ? \text{ and } y_2 = ?) \quad (45a)$$

$$= \varepsilon^2. \quad (45b)$$

It is useful to name these two reliability updates

$$f(\varepsilon) = 2\varepsilon - \varepsilon^2, \quad (46)$$

$$g(\varepsilon) = \varepsilon^2. \quad (47)$$

The function f gives the erasure probability of the minus descendant and g gives that of the plus descendant. Both descendants are themselves BECs of the form shown in Figure 8. We can now verify that their reliabilities have separated:

$$f(\varepsilon) - \varepsilon = \varepsilon(1 - \varepsilon) > 0, \quad (48a)$$

$$\varepsilon - g(\varepsilon) = \varepsilon(1 - \varepsilon) > 0, \quad (48b)$$

for $0 < \varepsilon < 1$. A larger erasure probability means a less reliable BEC. Equation (48a) therefore shows that the minus task is less reliable than either original channel, while Equation (48b) shows that the plus task is more reliable. This is the first polarization step.

The capacities of the two synthetic channels are

$$C^-(\varepsilon) = 1 - f(\varepsilon) = (1 - \varepsilon)^2, \quad (49)$$

$$C^+(\varepsilon) = 1 - g(\varepsilon) = 1 - \varepsilon^2. \quad (50)$$

Their sum is

$$C^-(\varepsilon) + C^+(\varepsilon) = (1 - \varepsilon)^2 + (1 - \varepsilon^2) \quad (51a)$$

$$= 2(1 - \varepsilon), \quad (51b)$$

exactly the combined capacity of the two BECs supplied to the transform. Thus one local

transform redistributes reliability without creating capacity.

4.2.2 Expanding the Construction to Four Bits

Equations (46) and (47) give us a reusable rule: two identical BECs with erasure probability ε produce descendants with erasure probabilities $f(\varepsilon)$ and $g(\varepsilon)$. This rule makes the analysis recursive. At each new level, ε denotes the erasure probability entering that particular transform, so we can substitute the previous level's result into the same two functions.

To demonstrate the recursive step, expand to a new block with $n = 4 = 2^2$. This four-bit system has two transform levels. Alice begins with four independent, equally likely inputs u_1, u_2, u_3, u_4 . The communication chain is

$$\begin{aligned} (u_1, u_2, u_3, u_4) &\xrightarrow{\text{polar encoder}} (x_1, x_2, x_3, x_4), \\ (x_1, x_2, x_3, x_4) &\xrightarrow{\text{four independent BEC uses}} (y_1, y_2, y_3, y_4), \\ (y_1, y_2, y_3, y_4) &\xrightarrow{\text{successive decoder}} (\hat{u}_1, \hat{u}_2, \hat{u}_3, \hat{u}_4). \end{aligned} \quad (52)$$

As in the two-bit chain, the different symbols describe the same four positions at different stages.

The encoder first applies the two-bit transform to two pairs of inputs:

$$\begin{aligned} (v_1, v_2) &= (u_1 \oplus u_2, u_2), \\ (v_3, v_4) &= (u_3 \oplus u_4, u_4). \end{aligned} \quad (53)$$

It then applies the same transform across the two pairs:

$$\begin{aligned} (x_1, x_3) &= (v_1 \oplus v_3, v_3), \\ (x_2, x_4) &= (v_2 \oplus v_4, v_4). \end{aligned} \quad (54)$$

Combining the two stages gives

$$\begin{aligned} x_1 &= u_1 \oplus u_2 \oplus u_3 \oplus u_4, & x_2 &= u_2 \oplus u_4, \\ x_3 &= u_3 \oplus u_4, & x_4 &= u_4. \end{aligned} \quad (55)$$

Because each local transform is reversible, the complete encoder is also reversible. The four encoded bits then pass through four independent physical BECs.

We now reuse the reliability rule from the two-bit problem. Choose $\varepsilon = 1/2$ so that each physical BEC has capacity $1/2$ and the arithmetic remains simple. At the first level of the reliability tree, f and g give the two intermediate erasure probabilities

$$f\left(\frac{1}{2}\right) = \frac{3}{4}, \quad (56)$$

$$g\left(\frac{1}{2}\right) = \frac{1}{4}. \quad (57)$$

These are the same two reliabilities produced by the two-bit calculation. In the four-bit system, however, they are only intermediate branches. We apply f and g once more to each branch.

Table 1 collects the resulting erasure probabilities and capacities. The two branch symbols record the minus or plus choice at each level. Under the input ordering in Equation (55), $W_4^{(i)}$ is the synthetic channel used to decode u_i .

i	Branch	Reliability calculation	Erasure probability ε_i	Capacity $1 - \varepsilon_i$
1	--	$f(f(1/2)) = f(3/4)$	15/16	1/16
2	-+	$g(f(1/2)) = g(3/4)$	9/16	7/16
3	+-	$f(g(1/2)) = f(1/4)$	7/16	9/16
4	++	$g(g(1/2)) = g(1/4)$	1/16	15/16

Table 1: The four synthetic channels obtained from a BEC(1/2). A smaller erasure probability and a larger capacity both indicate a better synthetic channel.

Their total equals the capacity of the four original BEC uses combined:

$$\frac{1}{16} + \frac{7}{16} + \frac{9}{16} + \frac{15}{16} = 2 = 4C_{\text{BEC}}. \quad (58)$$

After only two stages, the strongest and weakest synthetic channels are already close to capacities 1 and 0. The middle two are less extreme, but further recursion continues the separation.

4.3 From Recursion to a Polar Code

Consider what happens when we continue applying the same idea recursively. Begin with $n = 2^m$ independent uses of the same binary-input channel. At each of the $m = \log_2 n$ stages, pair the input coordinates, apply Equation (32), and repeat the operation across the newly formed groups. The process still produces n synthetic channels; it changes their reliabilities rather than reducing their number. Here, *reliability* describes how likely the input bit of one synthetic channel is to be recovered successfully. It is a property of one synthetic channel, not the block-error probability of the complete code. Block-error probability is the probability that at least one information bit in the decoded block is incorrect. For the binary erasure channels in the worked example above, we measure synthetic-channel reliability by erasure probability: a smaller erasure probability means a more reliable synthetic channel. For other channel models, a different reliability measure may be used. Figure 9 gives a conceptual visualization.

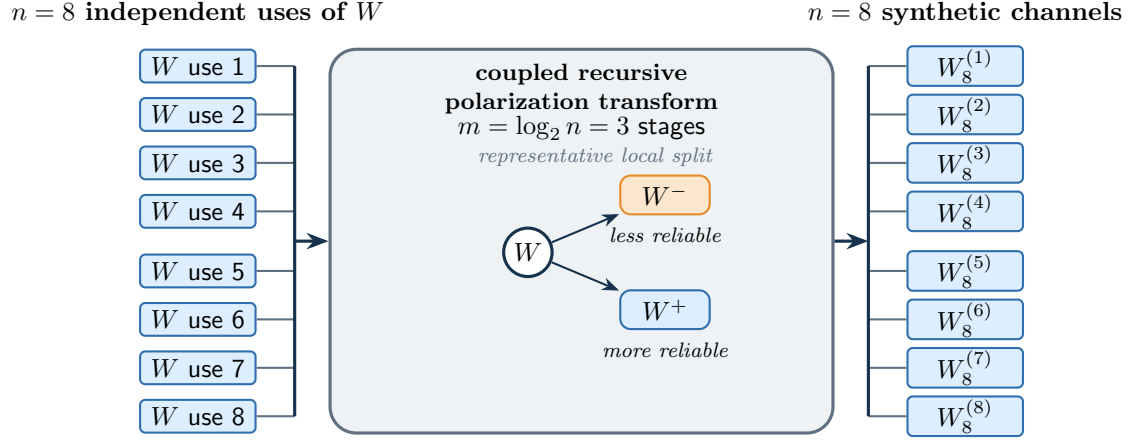


Figure 9: Conceptual diagram of the recursive polarization process. $n = 2^m$ physical channel uses produce n indexed synthetic channels; the indices themselves do not generally rank the channels by reliability.

Thus polar codes use block lengths of the form

$$n = 2^m, \quad (59)$$

where $m = \log_2 n$ is the number of recursion stages.

The polarization theorem describes what happens as this process continues. We measure the capacity of each individual synthetic channel by the mutual information $I(U; Y)$ between its input bit U and its output Y . Here, Y represents all channel observations and previously decoded input bits made available for that particular decoding task, not merely one physical channel output. We write the capacity of the i th synthetic channel as $I(W_n^{(i)})$.

The original physical channel is W , and $I(W)$ denotes its *symmetric capacity*: the mutual information obtained when its binary input is equally likely to be 0 or 1. As n grows through powers of two, the fraction of synthetic channels with capacity close to 1 approaches $I(W)$, the fraction with capacity close to 0 approaches $1 - I(W)$, and the fraction with intermediate capacity approaches zero. This statement is asymptotic; at a finite block length, some synthetic channels may still have intermediate reliabilities.

The final result is a set of n synthetic channels with different reliabilities. We use the most reliable channels to send information, while the least reliable channels are *frozen*: their inputs are set to predetermined values known by both the encoder and decoder. We now state this construction rule precisely. For a chosen block length n , the transmitter and receiver first rank the synthetic channels according to their reliability. In the BEC example, this is easy because a smaller erasure probability means a more reliable channel. For a general channel, the reliabilities must be calculated or approximated for the chosen physical channel.

Let $\mathcal{A}$ be the set of the k most reliable positions. The input vector $\mathbf{u} = (u_1, \dots, u_n)$ is filled according to

$$u_i = \begin{cases} \text{an information bit,} & i \in \mathcal{A}, \\ 0, \text{ a predetermined frozen bit,} & i \notin \mathcal{A}. \end{cases} \quad (60)$$

Frozen bits may be set to any predetermined values, 0 or 1, provided that both the encoder

and decoder know them. They do not carry new information, and the decoder does not need to infer them from the noisy output. In this paper, every frozen bit is set to 0 to simplify the decoder and to keep the resulting polar code linear. Nonzero frozen choices generally produce a translated, or coset, version of the corresponding linear code. The resulting code has rate

$$R = \frac{k}{n}. \quad (61)$$

We can now complete the four-bit example rather than stopping at the reliability table. For the BEC(1/2) in Table 1, let $n = 4$ and $k = 2$. The two most reliable positions are 3 and 4, so

$$\mathcal{A} = \{3, 4\}, \quad u_1 = u_2 = 0. \quad (62)$$

If the two message bits are (a, b) , they are placed in (u_3, u_4) , and Equation (55) gives

$$(a, b) \longrightarrow \mathbf{u} = (0, 0, a, b) \longrightarrow \mathbf{x} = (a \oplus b, b, a \oplus b, b). \quad (63)$$

For example, the message $(a, b) = (1, 0)$ produces

$$\mathbf{u} = (0, 0, 1, 0), \quad \mathbf{x} = (1, 0, 1, 0). \quad (64)$$

This example demonstrates how the reliability ranking determines the input placement and how those inputs are encoded. The choice $\mathcal{A} = \{3, 4\}$ is specific to the BEC(1/2) table above; another physical channel may rank the positions differently.

For a block length $n = 2^m$, the encoder applies the two-bit XOR transform recursively to the completed vector $\mathbf{u}$. Because the transform uses only XOR and copying, and because all frozen bits are zero, the resulting polar code is a binary linear code of the kind introduced in Subsection 2.5. There are $m = \log_2 n$ stages, and each stage performs work proportional to n . The total encoding complexity is therefore

$$O(nm) = O(n \log n). \quad (65)$$

Big-O notation describes how an operation count grows as the input size increases, rather than giving its exact value. More mathematically, $T(n) = O(n \log n)$ means that there are constants c and n_0 such that $T(n) \leq cn \log n$ whenever $n \geq n_0$. Constant factors and smaller-order terms are ignored. The important point is that $n \log n$ grows only slightly faster than n , so the encoding remains efficient even for large block lengths.

The qualitative polarization statement explains the achievable rate: if a target rate R is below $I(W)$, then sufficiently large block lengths provide enough channels near capacity 1 to place information in approximately Rn reliable positions. The stronger claim that the block-error probability also approaches zero requires the rate at which these channels become reliable; that result is included in the polar-coding theorem stated in Subsection 4.5.

4.4 Successive-Cancellation Decoding

The previous subsection showed how Alice selects the information positions and encodes the completed input vector. Although the encoder's XOR transformation is reversible, the

receiver cannot simply run it backward because the channel outputs may be corrupted or erased. Instead, successive-cancellation decoding uses the channel observations and earlier decisions to recover the input coordinates one at a time. The receiver observes the entire output vector $\mathbf{y} = (y_1, \dots, y_n)$ and decodes in the forward order

$$u_1, u_2, \dots, u_n. \quad (66)$$

This order first appeared in the two-bit example: the receiver estimates u_1 through W^- and then uses that decision while estimating u_2 through W^+ . The four-bit construction repeats this dependency recursively. The receiver estimates u_1 through $W_4^{(1)}$, then estimates u_2 through $W_4^{(2)}$ using the decision about u_1 , and continues in this way through u_4 . Thus the synthetic channels are successive decoding tasks built from the same received vector, not isolated physical links.

At a frozen position, the decoder simply inserts the known value 0. At an information position, it compares how likely 0 and 1 are using the entire received vector and all previous decisions, then chooses the more likely value. This method is called *successive-cancellation decoding* because each new decision uses the decisions that came before it. An incorrect early estimate can therefore influence later estimates.

A direct calculation of every conditional probability would repeat a great deal of work. The recursive polar graph allows intermediate likelihood calculations to be reused across decisions. It has $\log_2 n$ stages with work proportional to n per stage, giving successive-cancellation decoding complexity

$$O(n \log n). \quad (67)$$

This is the computational benefit of the recursive structure: the encoder and decoder do not need to store or search an arbitrary list of exponentially many codewords.

4.5 What the Polar-Coding Theorem Does and Does Not Claim

Polar codes are a family of binary linear codes introduced by Arikan in 2008. In his complete 2009 journal treatment, he proved that they achieve the symmetric capacity of binary-input memoryless channels while supporting efficient encoding and decoding [1].

We can now state exactly what the construction accomplishes. Arikan's polarization theorem applies to binary-input discrete memoryless channels and measures rate using the uniform-input mutual information $I(W)$, also called the channel's *symmetric capacity* [1].

Theorem 4.1: Polar coding theorem, simplified

For a binary-input discrete memoryless channel W and any target rate $R < I(W)$, there exists a family of polar codes with block lengths $n = 2^m$ whose rates are at least R and whose block-error probabilities under successive-cancellation decoding approach zero as $n \rightarrow \infty$. Encoding and successive-cancellation decoding can be performed in $O(n \log n)$ operations.

For symmetric channels such as the BSC and BEC, the uniform input distribution is optimal. Their symmetric capacity $I(W)$ therefore equals the ordinary capacity C used in Shannon's theorem. For a nonsymmetric binary-input channel, these two quantities need not be equal,

so the theorem above should not automatically be read as a claim that the basic construction reaches the unrestricted capacity of every channel.

Like Shannon's theorem, the polar-coding theorem describes a family as the block length grows. It does not claim that every finite polar code operates at capacity or has zero probability of error. At finite n , some synthetic channels remain only partly polarized, their reliabilities must be estimated to select $\mathcal{A}$, and basic successive-cancellation decoding need not give the best practical error performance. The asymptotic achievement is still remarkable: one fixed recursive idea supplies an explicit family whose rate approaches the relevant capacity, whose error probability approaches zero, and whose encoding and decoding complexity grows almost linearly with block length.

5 Conclusion

The repetition code showed Alice and Bob that redundancy can protect a message, but it also exposed the central cost: stronger protection can consume more channel uses and reduce the communication rate. Shannon's theorem answers the question created by this trade-off. For the BSC, families of codes can communicate reliably at any rate below $1 - h_2(p)$, but not above it. The typical-noise argument explains this threshold by comparing the number of likely outputs surrounding each codeword with the total output space. Mutual information extends the same idea beyond the BSC by measuring how much a channel output reveals about its input.

Shannon's theorem identifies the limit without selecting a particular code. Polar codes provide one explicit response to this construction problem. Their recursive XOR transform reorganizes identical channel uses into synthetic channels with different reliabilities. The BEC examples showed this separation first for two bits and then recursively for four. Information bits are placed in the most reliable positions, while the remaining positions are frozen to values known by both the encoder and decoder. This addresses the three goals posed in the introduction: rates can approach the symmetric capacity $I(W)$, block-error probability under successive-cancellation decoding can approach zero, and both encoding and decoding require $O(n \log n)$ operations rather than an exhaustive search through the codebook.

These conclusions are asymptotic rather than guarantees about every finite code. At finite block lengths, some synthetic channels remain only partly polarized, and basic successive-cancellation decoding need not provide the best practical error performance. Furthermore, the symmetric capacity $I(W)$ equals the ordinary capacity C for the symmetric BSC and BEC considered in this paper, but the two need not agree for every binary-input channel. Polar codes are therefore not the unique or universally best realization of Shannon's theorem. Their significance here is more precise: Shannon identifies what reliable communication permits, while polarization supplies one explicit and computationally efficient way to approach that limit.

References

- [1] Erdal Arkan. Channel polarization: A method for constructing capacity-achieving codes for symmetric binary-input memoryless channels. *IEEE Transactions on Information Theory*, 55(7):3051–3073, 2009.

-
- [2] Siddhartha Biswas. Introduction to coding theory: Basic codes and shannon's theorem, 2011. Undergraduate research paper from UChicago.
 - [3] Peter Elias. Error-free coding. In *IRE Transactions on Information Theory*, volume 4, pages 29–37, 1954.
 - [4] Omar Elshinawy. Syllabus & agenda: Mathematical undergraduate seminar, Spring 2025.
 - [5] Michel Goemans. Shannon's noisy coding theorem. Technical report, MIT, May 2015.
 - [6] Venkatesan Guruswami and Dan Halperin. More on linear codes. Technical Report Lecture 2, Carnegie Mellon University, Department of Computer Science, October 2006.
 - [7] Venkatesan Guruswami and Widad Machmouchi. Shannon's theorem. Technical Report Lecture 3, Carnegie Mellon University, Department of Computer Science, October 2006.
 - [8] Venkatesan Guruswami and Vibhor Rastogi. Error correcting codes. Technical Report Lecture 1, Carnegie Mellon University, Department of Computer Science, October 2006.
 - [9] Venkatesan Guruswami and Atri Rudra. Proof of shannon's theorem and an explicit code. Technical Report Lecture 4, Carnegie Mellon University, Department of Computer Science, October 2006.
 - [10] Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3–4):379–423, 623–656, 1948.
 - [11] Lucas Slot and Sebastian Zur. Shannon's noisy-channel coding theorem. Technical report, Vrije Universiteit Amsterdam, February 2015.
 - [12] Madhu Sudan and Mirac Suzgun. A gentle introduction to polar codes. Technical Report Lecture 8, Harvard University, January 2019.

Lagrangian Putt-Putt: A Holistic Investigation into Holonomically Constrained Forced-Geodesic Motion on Arbitrary C^2 -Surfaces

Noah Horne

PhDS'26

Abstract. This paper concerns the holistic investigation of golf ball dynamics. Specifically, in this paper we seek to determine the generalized equations of motion for a golf ball constrained to a golf course of the form $z = h(x, y)$ under constant gravity g . In doing so, concepts such as Lagrangian mechanics, the principle of least action, variational calculus, and differential geometry are brought up. The paper is written in such a way that a reader with basic calculus knowledge should be able to meaningfully extract the important information of this paper.

1 Introducing the Golf Ball System (q_1, q_2, q_3)

This article serves as a semi-gentle introduction into the world of Lagrangian Mechanics through an approachable everyday example: mini-golf. The goal of this discussion is to demonstrate how Lagrangian methods can be applied in a holistic and exploratory manner to examine otherwise complicated generalized systems. A **system** in the context of analytical mechanics refers to a collection of time-dependent generalized coordinates $(q_1(t), q_2(t), \dots, q_n(t))$, with time derivatives $\dot{q}_i(t) = \frac{dq_i}{dt}$ $i = 1, 2, \dots, n$ —which serve as a generalized velocity. Using these generalized coordinates, we further define a system's kinetic energy, $T(q_1, q_2, \dots, q_n, \dot{q}_1, \dot{q}_2, \dots, \dot{q}_n)$ and potential energy, $V(q_1, q_2, \dots, q_n)$ as two inextricable features of the system. This means that for a system with n generalized coordinates, it exhibits n degrees of freedom (n spatial components q_i , $i = 1, \dots, n$).

The particular class of system that will be discussed during this article are holonomically constrained systems. This means that rather than simply taking the system as is, we demand a certain relationship between the generalized coordinates to always hold.

Definition 1: Holonomic Constraint
A holonomic constraint on a mechanical system with n degrees of freedom $(q_1, q_2, \dots, q_n)$ and time t is imposed upon only the spatial coordinates (and sometimes time) by the equation $f(q_1, q_2, \dots, q_d, t) = 0$.

Holonomically constrained systems present themselves frequently in the study of mechanics, particularly when objects are subject to spatial constraints, such as sticking to a specific surface, or only moving along one axis or path. For the purposes of this investigation, a very particular class of holonomic constrained systems will be examined—a golf ball with position $(q_1, q_2, q_3) \in \mathbb{R}^3$ in 3 dimensions moving along some smooth surface $h(q_1, q_2)$ under uniform gravity g . Such a system provides invaluable parallels between important subareas of physics, such as mechanics, general relativity, and even planetary motion. In this particular situation, we shall consider the holonomic constraint which binds the golf ball to the surface of the golf course. We do this by restricting the vertical spatial component, and forcing our system to satisfy for a given golf course $h(q_1, q_2)$:

$$h(q_1, q_2) = q_3 \implies \underbrace{h(q_1, q_2) - q_3}_{f(q_1, q_2, q_3)} = 0 \quad (1)$$

Equation 1 is precisely the holonomic constraint we will consider. Note that it is equivalent to saying that the golf ball will permanently remain stuck to the surface of the golf course throughout the course of its trajectory. With the idea of a mechanical system and constraints defined, it is crucial now to define the framework on which we can figure out the dynamics of the system. As is, we have no idea how these generalized coordinates of the golf ball behave, so we should introduce the core concepts of the Lagrangian mechanics toolbox to approach this problem.

2 A Primer on Lagrangian Mechanics

2.1 Motivation

In one's typical formal education, the study of mechanics often begins with Newton's laws, which describe the motion of objects via forces $\sum_i F_i$ and accelerations. However, Lagrangian mechanics provides a powerful alternative framework, especially well-suited for systems with constraints and generalized coordinates. In Newtonian mechanics, introducing a constraint involves determining the precise forces which need to act on an object to ensure the constraint is satisfied. In Lagrangian Mechanics, one does not require to engineer such subtleties, and can rather define any coordinates they wish to describe the system, some which for example may implicitly take the constraint into account.

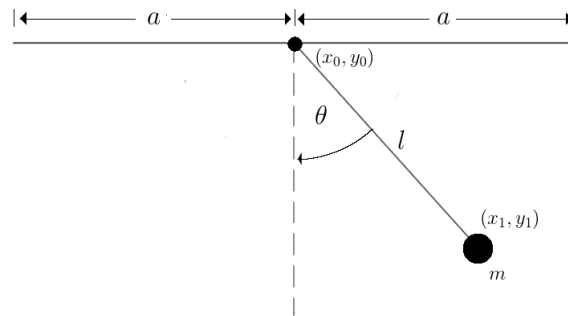


Figure 1: A simple pendulum, illustrating the use of generalized coordinates $(x_1, y_1) \rightarrow \theta$ (RIGuy).

Consider the example of the simple 2D pendulum of length l . In Newtonian mechanics, one may have defined this system in terms of the bobs x and y coordinates. Since the length of the rod is fixed, it would be necessary to find a way to solve for the dynamics while also imposing the constraint that $\sqrt{x^2 + y^2} = l$. Then, while simultaneously taking this constraint into account, one would need to determine the total force on the bob as a sum of the tension $\vec{T}$ and the gravity $\vec{g}$. In the end, this becomes quite complicated, and warrants the use of some nontrivial forces expressed in x and y . With **Lagrangian Mechanics** on the other hand, we can pick our coordinates more freely, indirectly taking the constraints into account. For example, we could take the angle made of the bob from the vertical, θ , as our coordinate. Then, using the Lagrangian toolkit, we simply figure out what the kinetic and potential energy of the system is with respect to this coordinate, and all equations of motion immediately follow.

2.2 The Principle of Stationary Action and Calculus of Variations

The central object in Lagrangian mechanics is the **Lagrangian function**.

Definition 2: The Lagrangian Function

The Lagrangian $L : \mathbb{R}^{2n+1} \rightarrow \mathbb{R}$ is defined as the difference in potential and kinetic energy, T and V ,

$$L(q_1, q_2, \dots, q_n, \dot{q}_1, \dot{q}_2, \dots, \dot{q}_n, t) = T(\dot{q}_1, \dots, \dot{q}_n) - V(q_1, \dots, q_n).$$

The Lagrangian function is extremely important in considering physical systems, as it is a crucial component of one of the most important quantities in physics: the **action functional** $S(L)$.

Definition 3: The Action Functional

The action functional $S(L)$ maps the Lagrangian function to some scalar quantity, representative of some average difference in potential V and kinetic energy T over a time interval $[t_1, t_2]$.

$$S(L) = \int_{t_1}^{t_2} L(q_1, \dots, q_n, \dot{q}_1, \dots, \dot{q}_n, t) dt$$

In physics, one may think of the action $S(L)$ as a measure of the balance of kinetic and potential energy throughout a systems trajectory. Mathematically however, it can be thought of as a **functional** with the Lagrangian function L as its argument.

Definition 4: Functional

A functional is an object which lives in the **dual space** of the normed vector space of functions. In other words, it is a map that takes in a given function, and returns a scalar. Unlike a usual function, which maps scalar to scalar, or vector to scalar, a functional maps an entire function to a scalar. Examples of functionals include: definite integrals, taking the maximum/minimum, the $\mathcal{L}^p$ norm, etc.

As the name suggests, the action functional is a functional. Particularly, it takes in the Lagrangian function of a given physical system evolving over a time interval $[t_1, t_2]$, and

returns a single value representative of the energy fluctuation between kinetic and potential over time. While such a quantity may not immediately pose itself as extremely useful, it singlehandedly underpins much of our understanding of the physical universe, as it obeys one of the most important principles in semi-modern physics: the **principle of stationary action**.

Definition 5: The Principle of Stationary Action

This principle states that the trajectory of a real world physical system $(q_1(t), \dots, q_n(t))$ always yields a Lagrangian function L which results in a **stationary output** of the action functional $\mathcal{S}$. Namely, the **variation** δ of the action functional is zero:

$$\delta\mathcal{S} = 0$$

To fully understand what this principle implies, let us define the concept of the variation of a function, which in as sense can be thought of as the infinitesimal change of a functional with respect to its function argument.

Definition 6: The Variation of a Functional

The variation of a functional is defined as an infinitesimal perturbation of a function. Let $\eta(x)$ be an arbitrary function that admits at least one derivative, and let $\varepsilon \in \mathbb{R}, |\varepsilon| \ll 1$. Then, the variation of a functional $\mathcal{S}(L)$ is given by:

$$\delta\mathcal{S} = \mathcal{S}(L + \varepsilon\eta)$$

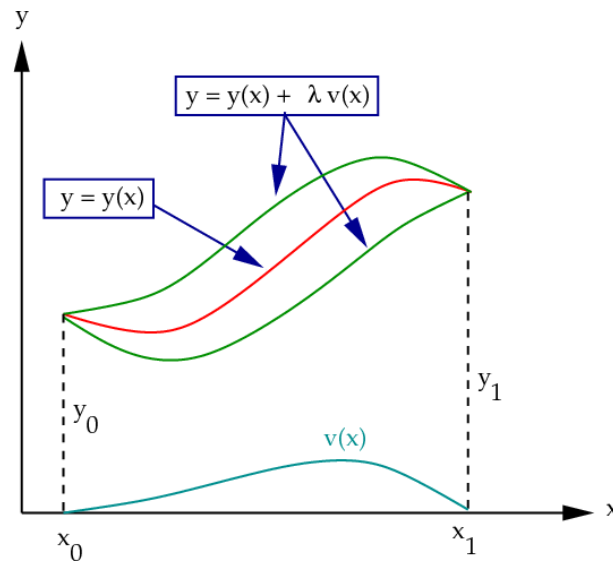


Figure 2: A visualization of a functional variation $v(x)$ on y . Such a diagram demonstrates how we change the path $q(t)$ slightly in the Lagrangian function to impose a variation on the action functional $\mathcal{S}(L)$ (wikiversity).

Notice also that if L is the Lagrangian function of a real system such that the action functional $\mathcal{S}$ is minimal, then $\mathcal{S}(L) \leq \mathcal{S}(L + \varepsilon\eta)$. Let's try to use this idea and more rigorously explore the variation of the action $\mathcal{S}$ for a given Lagrangian function.

Let L be a Lagrangian function that minimizes the action functional $\mathcal{S}$. Furthermore, let $\varepsilon \in \mathbb{R}, \varepsilon \ll 1$ be some small real number, and $\eta(t)$ be some function which admits its first

derivative, and is zero at the boundaries of the time interval, e.g. $\eta(t_1) = \eta(t_2) = 0$. First, we know that the variation of the action is zero. Namely:

$$\delta\mathcal{S} = 0 \quad (2)$$

By combining the definitions 3 and 6, let us define a parameterized variation of the action $\Phi(\varepsilon)$ such that:

$$\Phi(\varepsilon) = \mathcal{S}(L + \varepsilon\eta) \quad (3)$$

By the principle of stationary action, it is clear that:

$$\Phi'(\varepsilon)|_{\varepsilon=0} = \delta\mathcal{S} = 0 \quad (4)$$

Using the definition of the action function, one may rewrite equation 4 as

$$\Phi'(\varepsilon)|_{\varepsilon=0} = \int_{t_1}^{t_2} \frac{dL(q + \varepsilon\eta, \dot{q} + \varepsilon\dot{\eta}, t)}{d\varepsilon} \Big|_{\varepsilon=0} dt. \quad (5)$$

By applying chain rule to the function L , we proceed further with

$$\Phi'(\varepsilon)|_{\varepsilon=0} = \int_{t_1}^{t_2} \left(\frac{\partial L}{\partial q} \eta + \frac{\partial L}{\partial \dot{q}} \dot{\eta} \right) \Big|_{\varepsilon=0} dt, \quad (6)$$

which can be split into two integrals

$$\Phi'(\varepsilon)|_{\varepsilon=0} = \int_{t_1}^{t_2} \frac{\partial L}{\partial q} \eta \Big|_{\varepsilon=0} dt + \int_{t_1}^{t_2} \frac{\partial L}{\partial \dot{q}} \dot{\eta} \Big|_{\varepsilon=0} dt, \quad (7)$$

the second term integrated by parts

$$\Phi'(\varepsilon)|_{\varepsilon=0} = \int_{t_1}^{t_2} \frac{\partial L}{\partial q} \eta \Big|_{\varepsilon=0} dt + \left[\frac{\partial L}{\partial \dot{q}} \eta \right]_{t_1}^{t_2} - \int_{t_1}^{t_2} \frac{d}{dt} \frac{\partial L}{\partial \dot{q}} \eta \Big|_{\varepsilon=0} dt, \quad (8)$$

and resimplified finally as

$$\Phi'(\varepsilon)|_{\varepsilon=0} = \int_{t_1}^{t_2} \eta \left(\frac{\partial L}{\partial q} - \frac{d}{dt} \frac{\partial L}{\partial \dot{q}} \right) dt = 0. \quad (9)$$

By the fundamental lemma of calculus of variations, the only way that this integral can be zero is if

$$\frac{\partial L}{\partial q} - \frac{d}{dt} \frac{\partial L}{\partial \dot{q}} = 0. \quad (10)$$

With that, we have derived the partial differential equation (PDE) which describes the dynamics of our real-world systems' generalized coordinates. Formally, these equations are called the **Euler-Lagrange equations**:

Definition 7: The Euler-Lagrange Equations

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{q}_i} \right) - \frac{\partial L}{\partial q_i} = 0, \quad i = 1, \dots, n.$$

These equations yield the equations of motion for each generalized coordinate q_i . For holonomically constrained systems, such as our golf ball restricted to a surface, the constraints are incorporated either by reducing the number of independent coordinates or by introducing Lagrange multipliers.

In the context of our golf ball example, the holonomic constraint $h(q_1, q_2) - q_3 = 0$ allows us to eliminate q_3 and express all quantities in terms of q_1 and q_2 —since $q_3 = h(q_1, q_2)$. The kinetic and potential energies can then be written as functions of q_1, q_2 , and their time derivatives, enabling us to construct the Lagrangian and apply the Euler-Lagrange equations to derive the equations of motion. Let us do so now.

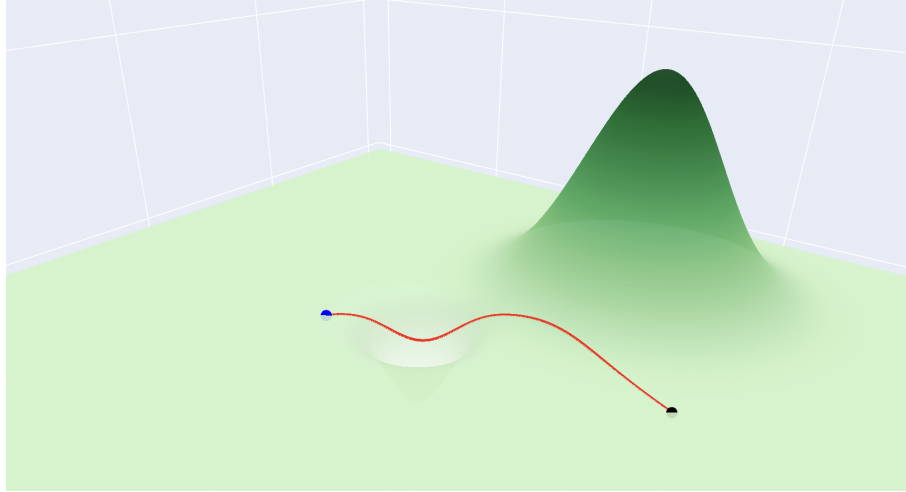
3 Formulation of the Mini-Golf Problem

Figure 3: A simple diagram depicting the way that a golf ball moves on some golf course. The goal of this investigation is to determine the trajectory, e.g. the red line in the image, given an initial starting position (q_1, q_2) and velocity $(\dot{q}_1, \dot{q}_2)$.

Since our golf ball lives in 3 dimensions (since we are 3D creatures), we can express the golf balls' position as a 3 dimensional vector of x, y , and z coordinates, q_1, q_2 , and q_3 respectively. Thus, let us denote the position vector of the golf ball as

$$\vec{q} = \begin{pmatrix} q_1 \\ q_2 \\ q_3 \end{pmatrix} = (q_1, q_2, q_3)^T. \quad (11)$$

This golf ball is bound permanently to a golf course, which we will define as a twice continuously-differentiable surface $q_z = h(q_x, q_y)$. This means that all first and second-order partial derivatives of h , $(\partial_i h, \partial_{ij} h, i, j = 1, 2, 3)$ exist and are continuous. From now on, let $\partial_i h = \frac{\partial h}{\partial q_i}$ for sake of notational simplicity. Recall from 1 that we directly introduced the holonomic constraint of the golf ball system as

$$q_3 = h(q_1, q_2), \quad (12)$$

which simplifies our expression of the golf ball position to only two generalized coordinates: q_1 and q_2 . Now, our originally 3D position $\vec{q} \in \mathbb{R}^3$ can be reduced to a 2D vector of generalized coordinates e.g.

$$\vec{q} = \begin{pmatrix} q_1 \\ q_2 \end{pmatrix}. \quad (13)$$

With our system defined, and our generalized coordinates q_1, q_2 linked to the physical state of the golfball on the course $h(q_1, q_2)$, let us continue to construct the Lagrangian formalism described in Section 2. Since the Lagrangian function is given as the difference between the kinetic and potential energy T, V , let us determine these energies of our golf ball in terms of our new generalized coordinates. From kinematics, it is well known that the kinetic energy of a mass m at velocity $\dot{q}$ is given by

$$T(\dot{q}) = \frac{1}{2} m \dot{q}^2 \quad (14)$$

Thus, the kinetic energy of the golf ball as it rolls across the minigolf course is:

$$T_{ball} = \frac{1}{2} m \|\dot{\vec{q}}\|^2 = \frac{1}{2} m (\dot{q}_1^2 + \dot{q}_2^2 + \underbrace{(\dot{q}_1 \partial_1 h(q_1, q_2) + \dot{q}_2 \partial_2 h(q_1, q_2))}_{\frac{d}{dt}[h(q_1, q_2)]}) \quad (15)$$

And the potential is given by similar logic as:

$$V_{ball} = mgh(q_1, q_2). \quad (16)$$

As the expression for the kinetic energy is quite complicated, it would be convenient to express the large expanded result as the result of a vector matrix multiplication. Infact, letting $G \in Mat_{2 \times 2}(\mathbb{R})$, we define the **metric** for the system as:

$$G(\vec{q}) = \begin{pmatrix} 1 + [\partial_1 h]^2 & \partial_1 h \partial_2 h \\ \partial_1 h \partial_2 h & 1 + [\partial_2 h]^2 \end{pmatrix} \quad (17)$$

and rewrite our kinetic energy in terms of the velocity vector $\dot{\vec{q}}$ and the metric G as

$$T_{ball} = \frac{1}{2} m \dot{\vec{q}}^T G \dot{\vec{q}}. \quad (18)$$

Verification of this reexpression is left as an exercise to the reader, however can be done by simply expanding the matrix-vector product. Now combining our two energies into a single Lagrangian function by definition 2, we achieve our Lagrangian function of the system as

$$L(q_1, q_2, \dot{q}_1, \dot{q}_2) = \frac{1}{2} m \dot{\vec{q}}^T G \dot{\vec{q}} - mgh(q_1, q_2). \quad (19)$$

As we are now dealing with tensors, I will adopt the Einstein-index notation, with upper and lower indices respectively. Then, for $i, j, k = 1, 2$, we can compute the partial derivatives of the Lagrangian function to substitute into our Euler-Lagrange equation.

$$\frac{\partial L}{\partial \dot{q}^i} = \frac{1}{2} m (\partial_i G_{jk}) \dot{q}^j \dot{q}^k - mg \partial_i h \quad (20)$$

$$\frac{\partial L}{\partial \dot{q}^i} = m G_{ij} \dot{q}^j \implies \frac{d}{dt} \frac{\partial L}{\partial \dot{q}^i} = m G_{ij} \ddot{q}^j + m (\partial_k G_{ij}) \dot{q}^k \dot{q}^j \quad (21)$$

Note, $\partial_i G_{jk}$ is calculable explicitly with dependence on the golf course surface h . E.g.,

$$\begin{aligned} \partial_1 G_{11} &= 2\partial_1 h \partial_{11} h, & \partial_2 G_{11} &= 2\partial_1 h \partial_{12} h, \\ \partial_1 G_{12} &= \partial_1 h \partial_2 h + \partial_1 h \partial_{12} h, & \partial_2 G_{12} &= \partial_{12} h \partial_2 h + \partial_1 h \partial_{22} h \\ \partial_1 G_{22} &= 2\partial_2 h \partial_{12} h, & \partial_2 G_{22} &= 2\partial_2 h \partial_{22} h \end{aligned} \quad (22)$$

Substituting equations 20 and 21 into the Euler-Lagrange equation, one achieves, in Einstein-summation notation,

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{q}^i} \right) - \frac{\partial L}{\partial q^i} = G_{ij} \ddot{q}^j + \frac{1}{2} [(\partial_k G_{ij}) + (\partial_j G_{ik}) - (\partial_i G_{jk})] \dot{q}^j \dot{q}^k + g \partial_i h = 0 \quad (23)$$

The term in brackets, $\frac{1}{2} [(\partial_k G_{ij}) + (\partial_j G_{ik}) - (\partial_i G_{jk})]$, can be compactly written using the **Christoffel symbols of the first kind** associated with the metric G .

Definition 8: Christoffel Symbols of the First Kind

Such symbols describe the local curvature of a surface through the partial derivatives of the metric induced by the surface with three derivatives. In this case, our metric is G , induced by the curvature of our golf-course surface $h(x, y)$.

$$\Gamma_{ijk} = \frac{1}{2} (\partial_k G_{ij} + \partial_j G_{ik} - \partial_i G_{jk})$$

where Γ_{ijk} are the Christoffel symbols of the first kind.

Alternatively, using the inverse metric G^{il} , we can define the **Christoffel symbols of the second kind**. Notice that for the inverse metric, we denote it through the use of upper indices rather than the usual lower indices. When we apply the inverse metric to the Christoffel symbol of the first kind, we **raise** the first index of the Christoffel symbol using our metric G :

$$\Gamma_{jk}^i = G^{il} \Gamma_{ljk} \quad (24)$$

This idea of raising and lowering indices through the use of a metric is unanimously used in topics such as special relativity, where many-indexed tensors are commonplace. In our equation, we can apply the inverse metric to get rid of the metric coefficient on the generalized acceleration term $\ddot{q}^j$. Applying the inverse metric to 23, we achieve

$$G^{ij}G_{ij}\ddot{q}^j + G^{ij} \left(\underbrace{\frac{1}{2} [(\partial_k G_{ij}) + (\partial_j G_{ik}) - (\partial_i G_{jk})]}_{\Gamma_{ijk}} \ddot{q}^j \ddot{q}^k \right) + G^{ij}g\partial_i h = 0,$$

which transforms the middle Christoffel symbol to the second kind. Thus, the equations of motion for the golf ball constrained to the surface $h(q_1, q_2)$ can be elegantly written as:

$$\ddot{q}^i + \Gamma_{jk}^i \dot{q}^j \dot{q}^k + gG^{ij}\partial_j h = 0 \quad (25)$$

where $i, j, k = 1, 2$ and G^{ij} is the inverse of the metric G_{ij} . This equation describes the forced geodesic motion of the golf ball on the surface, with the Christoffel symbols encoding the curvature of the surface and the last term representing the effect of gravity projected onto the surface coordinates. To the mathematician who is trained in differential geometry, one may recognize equation 25 as a variant of the standard **geodesic equation**—a second order DE which represents the path that a free particle would take on manifold given no external accelerations. While the standard geodesic equation for a manifold (think of this as a smooth surface such as the golf course) M would be written as

$$\frac{d^2 x^a}{ds^2} + \Gamma_{bc}^a \frac{dx^b}{ds} \frac{dx^c}{ds} = 0, \quad (26)$$

note that our example has one extra term: $gG^{ij}\partial_j h$. This is because in our example, while the golf ball does indeed travel on some twice differentiable manifold, it does so while subject to a gravitational acceleration, influencing the particles trajectory. In a very direct sense, one could picture this term ($gG^{ij}\partial_j h$) as a direct expression for the components of force due to gravity on the golf ball due to the constraint of sticking to the surface of the course. With that, we can write out our expression as the standard geodesic equation with the right hand side equal to the constrained gravitational force, yielding

$$\boxed{\ddot{q}^i + \Gamma_{jk}^i \dot{q}^j \dot{q}^k = -gG^{ij}\partial_j h}, \quad (27)$$

which is the equation that completely describes the behavior of our generalized coordinates with respect to the x and y directions.

Now, this equation is written in a very compact form. In reality, it is two equations: one for the q_1 and one for q_2 . Writing these two equations out, we achieve a system of ODEs.

$$\ddot{q}^1 = -\Gamma_{jk}^1 \dot{q}^j \dot{q}^k - gG^{1j}\partial_j h \quad (28)$$

$$\ddot{q}^2 = -\Gamma_{jk}^2 \dot{q}^j \dot{q}^k - gG^{2j}\partial_j h \quad (29)$$

To reduce the order of the system of ODEs, but increase the number of variables, we introduce

a new variable for the generalized velocity, $v^i = \dot{q}^i$. Now, we achieve a system of four equations.

$$v^1 = \dot{q}^1 \quad (30)$$

$$v^2 = \dot{q}^2 \quad (31)$$

$$\dot{v}^1 = -\Gamma_{jk}^1 v^j v^k - g G^{1j} \partial_j h \quad (32)$$

$$\dot{v}^2 = -\Gamma_{jk}^2 v^j v^k - g G^{2j} \partial_j h \quad (33)$$

This system is therefore the most general simplified description of the trajectory of a golfball on an arbitrary height field $h(q^1, q^2)$ under gravity g . It is important to keep in mind as well that these Christoffel symbols with indicies are simply a representation of a sum of many Christoffel symbols through each index, each of which are expressions in terms of the second partial derivatives of our golf course height field $h(q^1, q^2)$. Written out in full expansion, this system of 4 ODEs is, which reveals slightly the true nature of the complexity of this problem.

$$v^1 = \dot{q}^1 \quad (34)$$

$$v^2 = \dot{q}^2 \quad (35)$$

$$\begin{aligned} \dot{v}^1 = & -[G^{11}\Gamma_{111}v^1v^1 + G^{11}\Gamma_{112}v^1v^2 + G^{11}\Gamma_{121}v^2v^1 + G^{11}\Gamma_{122}v^2v^2 + G^{12}\Gamma_{211}v^1v^1 \\ & + G^{12}\Gamma_{212}v^1v^2 + G^{12}\Gamma_{221}v^2v^1 + G^{12}\Gamma_{222}v^2v^2] - g(G^{11}\partial_1 h + G^{12}\partial_2 h) \end{aligned} \quad (36)$$

$$\begin{aligned} \dot{v}^2 = & -[G^{21}\Gamma_{111}v^1v^1 + G^{21}\Gamma_{112}v^1v^2 + G^{21}\Gamma_{121}v^2v^1 + G^{21}\Gamma_{122}v^2v^2 + G^{22}\Gamma_{211}v^1v^1 \\ & + G^{22}\Gamma_{212}v^1v^2 + G^{22}\Gamma_{221}v^2v^1 + G^{22}\Gamma_{222}v^2v^2] - g(G^{21}\partial_1 h + G^{22}\partial_2 h) \end{aligned} \quad (37)$$

Recall as well that the metric where the metric G_{ij} and its inverse G^{ij} are given by:

$$G_{11} = 1 + (\partial_1 h)^2, \quad G_{12} = \partial_1 h \partial_2 h, \quad G_{22} = 1 + (\partial_2 h)^2$$

$$G = \begin{pmatrix} G_{11} & G_{12} \\ G_{12} & G_{22} \end{pmatrix}, \quad G^{-1} = \frac{1}{\det G} \begin{pmatrix} G_{22} & -G_{12} \\ -G_{12} & G_{11} \end{pmatrix}$$

with $\det G = G_{11}G_{22} - (G_{12})^2$. Again for clarity, all partial derivatives are with respect to q^1 and q^2 , e.g. $\partial_1 h = \frac{\partial h}{\partial q^1}$, $\partial_{12} h = \frac{\partial^2 h}{\partial q^1 \partial q^2}$, etc.

Should one solve these equations for $(q^1(t), q^2(t))$ for some given initial conditions $(q_0^1, q_0^2, \dot{q}_0^1, \dot{q}_0^2)$, they will find the constrained geodesics of the golfball, or in other words, the complete golfball trajectory. With the groundwork now laid, it is now time to play with the equations and see which cases allow for an analytic solution to present itself.

4 Examples of Analytic Solutions of the Forced Golfball Geodesic System

4.1 The Boring Golf-Course: $h(q^1, q^2) = 0$

Let us first consider the simplest possible golf-ball course to ensure that the derived model works. That is, let us consider a perfectly flat golfball course, characterized by the height equation $h(x, y) = 0$. If our height is zero, then it is immediately clear that our metric G

becomes the identity matrix I , also known as the kronecker-delta tensor, where the indices of the metric are characterized by $G_{ij} = \delta_{ij}$. In this case, our system of equations derived from 27 becomes

$$v^1 = \dot{q}^1 \quad (38)$$

$$v^2 = \dot{q}^2 \quad (39)$$

$$\dot{v}^1 = 0 \quad (40)$$

$$\dot{v}^2 = 0 \quad (41)$$

For initial conditions of the golf ball $(q_0^1, q_0^2, v_0^1, v_0^2)$, this describes uniform motion in both q^1 and q^2 directions, as expected for unconstrained motion on a flat surface with no external forces acting in the plane. Solving the system of differential equations, we achieve

$$q^1(t) = v_0^1 t + q_0^1, \quad q^2(t) = v_0^2 t + q_0^2, \quad (42)$$

which describes straight line motion on the plane starting at our initial position, as demonstrated in the figure below.

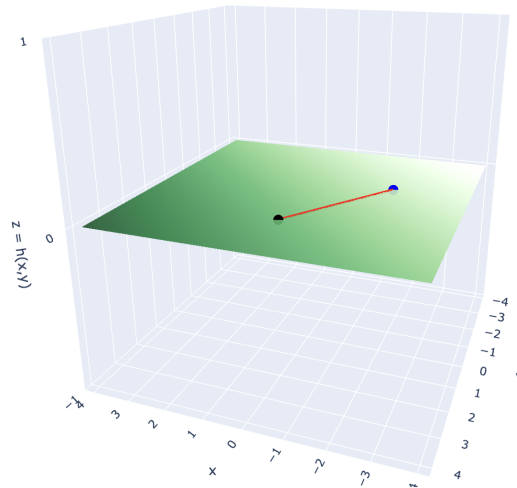


Figure 4: Diagram demonstrating the trajectory of a golfball on a flat course. Notice that the ball exhibits straight line motion.

So far, the model is consistent with the observed results. Let us now examine a slightly more complex example.

4.2 A Slightly Less-Boring Golf Course: $h(q^1, q^2) = \alpha q^1 + \beta q^2$

Let us now consider the example of a "tilted" golf course. That is, let us describe the golf course as an inclined plane $h(q^1, q^2) = \alpha q^1 + \beta q^2$. Since the expression of the height field is first order in q^1 and q^2 , all second derivatives vanish, yielding a constant metric G , namely

$$G = \begin{pmatrix} 1 + \alpha^2 & \alpha\beta \\ \alpha\beta & 1 + \beta^2 \end{pmatrix}. \quad (43)$$

Since the metric itself is constant, so is its inverse G^{-1} . Since Christoffel symbols function on the derivatives of the entries of the metric G , all Christoffel symbols are likewise zero, reducing our equations of motion to a very simple second order ODE, e.g.

$$\ddot{q}^i = -gG^{ij}\partial_j h. \quad (44)$$

Since $h(q^1, q^2) = \alpha q^1 + \beta q^2$, the partial derivatives $\partial_1 h = \alpha$, $\partial_2 h = \beta$. Let $A^i = gG^{i1}\alpha + gG^{i2}\beta$ (note also that G^{ij} is the inverse metric). This reduces our equations of motion for the golfball to a simple constant vector acceleration model, that is

$$\ddot{q}^i = -A^i \implies \begin{pmatrix} \ddot{q}^1 \\ \ddot{q}^2 \end{pmatrix} = - \begin{pmatrix} A^1 \\ A^2 \end{pmatrix} \quad (45)$$

which is a constant acceleration in both directions. The general solution is therefore

$$q^i(t) = q_0^i + v_0^i t - \frac{1}{2} A^i t^2, \quad (46)$$

or, more explicitly in vector notation,

$$\begin{pmatrix} q^1(t) \\ q^2(t) \end{pmatrix} = \begin{pmatrix} q_0^1 \\ q_0^2 \end{pmatrix} + \begin{pmatrix} v_0^1 \\ v_0^2 \end{pmatrix} t - \frac{1}{2} \begin{pmatrix} A^1 \\ A^2 \end{pmatrix} t^2 \quad (47)$$

where

$$\begin{pmatrix} A^1 \\ A^2 \end{pmatrix} = gG^{-1} \begin{pmatrix} \alpha \\ \beta \end{pmatrix}$$

This describes a parabola in the (q^1, q^2) plane, rotated according to the slope (α, β) of the surface. The trajectory is the projection of uniform acceleration down the gradient of the plane, as expected. Below is a diagram of this exact behavior when the ODE is solved numerically for the given height field. In the example the height field has parameters $\alpha = 0.2, \beta = 0.2$.

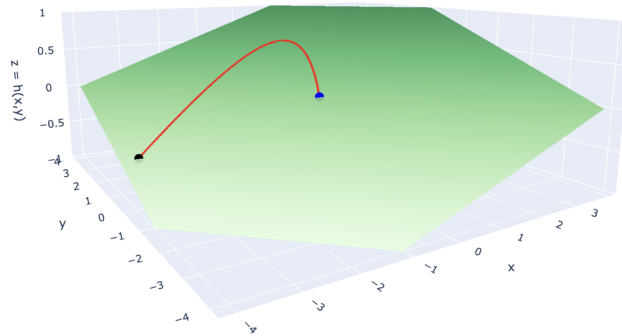


Figure 5: Diagram demonstrating the trajectory of a golfball on a tilted plane $h(q^1, q^2) = 0.2q^1 + 0.2q^2$. Notice that the ball exhibits a rotated parabolic path.

For more complex course geometries, of which there are many, the Christoffel symbols no longer reduce to zero (indicative of some further local curvature), greatly complicating the computation process of the analytic trajectory expression. While some cases may be possible,

it is often more practical in the real world to make very refined estimates using **numerical methods**, which are commonplace for physicists and computational mathematicians seeking to understand the behavior or more complex dynamical systems or systems of DEs. To reap the full benefits of this abstract minigolf model, let us now consider some more abstract course geometries which may tell us some more insightful things about how particles move on specific geometries when subject to specific forces, for now in a solely qualitative manner.

5 Investigating more Complex Geometries Numerically

It is often practical in mathematics to find alternative ways of modelling otherwise complicated systems in terms of other more abstract models. Let us take for example something already brought up in this paper as an example of the application of Lagrangian Mechanics: the Simple Pendulum.

Consider the case where the golf course surface is a perfectly hemispherical well of radius a centered at the origin, described biparametrically (in the best way I could figure out) by the height field

$$h(q^1, q^2) = -a \sqrt{\frac{1 - \frac{(q^1)^2 + (q^2)^2}{a^2} + \sqrt{\left(1 - \frac{(q^1)^2 + (q^2)^2}{a^2}\right)^2 + \varepsilon^2}}{2}}, \quad (48)$$

where the hemispherical well has radius a and $\varepsilon \ll 1$ or ≈ 0 describing the hemisphere edge smoothness as it transitions to a flat plane. As previously discussed, the holonomic constraint $q^3 = h(q^1, q^2)$ restricts the ball to move along the hemisphere's surface. In this geometry, the equations of motion derived previously reduce precisely to those governing a 3D simple pendulum: the ball's position is always at a fixed distance a from the origin, and its dynamics are determined by the interplay of gravity and the surface's curvature. This equivalence demonstrates that the golf ball model is not limited to mini-golf scenarios, but rather encodes the dynamics of a wide class of physical systems. The same formalism describes the motion of a pendulum, a particle on a sphere, or even planetary orbits under certain constraints. Thus, by analyzing the forced geodesic equations for arbitrary surfaces, we gain insight into the behavior of many seemingly different systems, unified by the underlying mathematics of constrained motion. Let us take a qualitative look at the dynamics of the golf ball on such a hemispherical surface when let to move around with some arbitrary initial velocity and position.

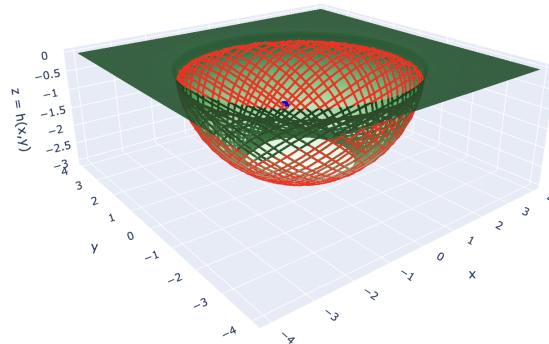


Figure 6: Diagram demonstrating the trajectory of a golf ball confined to a hemispherical well golf course. It's dynamics exactly mimic that of a simple pendulum!

Amazing! The dynamics of 6 exactly mimic that of the bob of a simple 3D pendulum. Notice the beautiful patterns that it traces as it evolves through its trajectory. Below is a top down image for a more clear vizualization of the clear pattern.

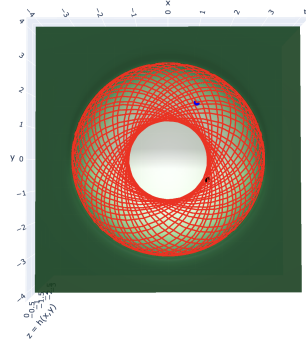


Figure 7: Top down image of the hemispherical course's golf ball trajectory

6 Conclusion

In conclusion, the mini-golf system explored in this paper serves as a compelling microcosm of the deep interplay between mathematics and physics. By formulating the motion of a golf ball constrained to an arbitrary surface, we have traversed a landscape rich with mathematical structure and physical insight. The journey began with the calculus of variations, where the principle of stationary action provided a unifying framework for deriving the equations of motion. The action functional,

$$\mathcal{S}(L) = \int_{t_1}^{t_2} L(q_1, q_2, \dot{q}_1, \dot{q}_2, t) dt, \quad (49)$$

encapsulates the physical requirement that nature selects paths which extremize the balance between kinetic and potential energy. Through the machinery of functional variation, we arrived at the Euler-Lagrange equations,

$$\frac{d}{dt} \left(\frac{\partial L}{\partial \dot{q}_i} \right) - \frac{\partial L}{\partial q_i} = 0, \quad (50)$$

which elegantly encode the system's dynamics in terms of generalized coordinates.

The introduction of holonomic constraints, binding the ball to a surface $h(q_1, q_2)$, naturally led us into the realm of differential geometry. The metric tensor G_{ij} , derived from the surface's partial derivatives, and the associated Christoffel symbols Γ_{jk}^i , revealed how curvature and geometry shape the motion. The resulting forced geodesic equation,

$$\ddot{q}^i + \Gamma_{jk}^i \dot{q}^j \dot{q}^k = -g G^{ij} \partial_j h, \quad (51)$$

is a profound statement which ties very nicely to the already derived **geodesic equation** from differential geometry. From it, we are able to generalize Newtonian dynamics to curved spaces, showing that the trajectory of the ball is governed not only by external forces (gravity) but also by the intrinsic geometry of the constraint surface.

This framework we developed seamlessly bridges concepts from classical mechanics, variational

calculus, and differential geometry: which is what makes this a great topic for discussion. Analytic solutions for simple surfaces (flat planes, inclined planes) demonstrated how the abstract formalism reduces to familiar results from classical mechanics, while more complex geometries (such as the hemispherical well) reveal rich dynamical behavior analogous to pendulum motion or planetary orbits (if we were to use a potential well style golf course). The equations themselves encode a multitude of physical phenomena: the metric G captures local stretching and bending, the Christoffel symbols Γ_{ijk} describe how straight lines become curved geodesics on the golf course, and the gravitational term projects external forces onto the golf course manifold.

Ultimately, this example illustrates how mathematical abstraction empowers us as mathematicians and physicists to model, analyze, and understand physical systems of great complexity. The Lagrangian mechanics approach not only simplifies the treatment of constraints, but also exposes the geometric and variational principles which underlie physical laws. Finally, the forced geodesic equation stands as a testament to the unity of mathematics and physics: it is at once a statement about optimal paths, curvature, and dynamical evolution. By studying the golf ball's journey across arbitrary surfaces, we glimpse the universal structures that govern motion in our universe, from the rolling of a ball to the orbits of planets and the propagation of light in curved spacetime. This synthesis of ideas exemplifies the beauty and power of mathematical physics, where equations become windows into the fabric of reality.

A Python Simulation Code

Below is the Python code used to numerically simulate the computed force geodics of the golf ball along the golf course $h(q^1, q^2)$ for all examples and figures present in the paper involving golf ball trajectories.

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
from matplotlib import cm
from scipy.integrate import solve_ivp
import sympy as sp

x_sym, y_sym = sp.symbols('x y')

# Pendulum case
r=3
eps=0.01
rho = sp.sqrt(x_sym**2 + y_sym**2)
u = rho / r
phi = (1 - u**2 + sp.sqrt((1 - u**2)**2 + eps**2))/2
h_expr = - r * sp.sqrt(phi)

# Two planet gravity system
# h_expr = -1/((x_sym)**2+(y_sym-0.9)**2)-1/((x_sym)**2+y_sym**2)

#h_expr = 0.2*x_sym + 0.2*y_sym

# Compute derivatives symb.
hx_expr = sp.diff(h_expr, x_sym)
hy_expr = sp.diff(h_expr, y_sym)
hxx_expr = sp.diff(hx_expr, x_sym)
hyx_expr = sp.diff(hy_expr, x_sym)
hyy_expr = sp.diff(hy_expr, y_sym)
hxy_expr = sp.diff(hx_expr, y_sym)

# Turn them into functions (through numpy which is fast)
h = sp.lambdify((x_sym, y_sym), h_expr, 'numpy')
```

```

h_x = sp.lambdify((x_sym, y_sym), hx_expr, 'numpy')
h_y = sp.lambdify((x_sym, y_sym), hy_expr, 'numpy')
h_xx = sp.lambdify((x_sym, y_sym), hxx_expr, 'numpy')
h_yy = sp.lambdify((x_sym, y_sym), hyy_expr, 'numpy')
h_xy = sp.lambdify((x_sym, y_sym), hxy_expr, 'numpy')

# -----
# Metric tensor and Christoffel symbols computation
# -----
def metric(x, y):
    hx, hy = h_x(x,y), h_y(x,y)
    G = np.array([[1+hx**2, hx*hy],
                  [hx*hy, 1+hy**2]])
    return G

def christoffel(x, y):
    # Christoffel symbols of first kind: Gamma_ijk = 1/2(partial_j g_ik + partial_k g_ij - partial_i g_jk)
    # Then raise index with inverse metric.
    hx, hy = h_x(x,y), h_y(x,y)
    hxx, hyy, hxy = h_xx(x,y), h_yy(x,y), h_xy(x,y)

    G = metric(x,y)
    Ginv = np.linalg.inv(G)

    # Partial derivatives of metric
    dGx = np.array([[2*hx*hxx, hxx*hy + hx*hxy],
                    [hxx*hy + hx*hxy, 2*hy*hxy]])
    dGy = np.array([[2*hx*hxy, hxy*hy + hx*hyy],
                    [hxy*hy + hx*hyy, 2*hy*hyy]])

    dG = [dGx, dGy]

    # Compute Christoffel symbols Gamma^i_jk
    Gamma = np.zeros((2,2,2))
    for i in range(2):
        for j in range(2):
            for k in range(2):
                term = 0.0
                for l in range(2):
                    term += Ginv[i,l]*(dG[j][l,k] + dG[k][l,j] - dG[l][j,k])
                Gamma[i,j,k] = 0.5*term
    return Gamma

# -----
# Equations of motion
# -----
g = 9.81
m = 1.0

def dynamics(t, state):
    x, y, vx, vy = state
    qdot = np.array([vx, vy])

    G = metric(x,y)
    Ginv = np.linalg.inv(G)
    Gamma = christoffel(x,y)

    # Compute RHS force term: -G^{ij} partial_j V/m, with V = m g h(x,y)
    gradV = np.array([m*g*h_x(x,y), m*g*h_y(x,y)])
    force = -Ginv @ (gradV/m)

    # Acceleration: -Gamma^i_jk v^j v^k + force^i
    accel = np.zeros(2)
    for i in range(2):
        for j in range(2):
            for k in range(2):
                accel[i] -= Gamma[i,j,k]*qdot[j]*qdot[k]
            accel[i] += force[i]

    return [vx, vy, accel[0], accel[1]]

# -----
# Initial conditions
# -----
x0, y0 = 1, 2
vx0, vy0 = -3, 5

```

```

state0 = [x0, y0, vx0, vy0]

# Time span
t_span = (0, 100) # n seconds
t_eval = np.linspace(t_span[0], t_span[1], 1000)

sol = solve_ivp(dynamics, t_span, state0, t_eval=t_eval, rtol=1e-8, atol=1e-8)

# Extract trajectory
X, Y = sol.y[0], sol.y[1]
Z = h(X,Y)
import plotly.graph_objects as go
import numpy as np

# Create surface
gx = np.linspace(-4, 4, 500)
gy = np.linspace(-4, 4, 500)
GX, GY = np.meshgrid(gx, gy)
GZ = h(GX, GY)

# Surface
surface = go.Surface(x=GX, y=GY, z=GZ, colorscale="Greens", opacity=0.9)

# Trajectory
traj = go.Scatter3d(x=X, y=Y, z=Z, mode="lines", line=dict(color="red", width=5), name="Trajectory")

# Start/End points
start = go.Scatter3d(x=[X[0]], y=[Y[0]], z=[Z[0]], mode="markers", marker=dict(size=6, color="blue"), name="Start")
end = go.Scatter3d(x=[X[-1]], y=[Y[-1]], z=[Z[-1]], mode="markers", marker=dict(size=6, color="black"), name="End")

fig = go.Figure(data=[surface, traj, start, end])
fig.update_layout(scene=dict(
    xaxis_title="x",
    yaxis_title="y",
    zaxis_title="z = h(x,y)",
    #zaxis=dict(range=[-1, 1])
))
fig.show()

```

References

- [1] V. I. Arnold. *Mathematical Methods of Classical Mechanics*. Springer-Verlag, 2 edition, 1989.
- [2] S. Bonnabel. A simple intrinsic reduced observer for geodesic flow. arXiv preprint arXiv:0811.3709, 2008.
- [3] J. F. Cariñena, J. de Lucas, and C. Sardón. Geodesic and newtonian vector fields and symmetries of constrained systems. *Symmetry*, 15(1):181, 2023.
- [4] T. Frankel. *The Geometry of Physics: An Introduction*. Cambridge University Press, 3 edition, 2011.
- [5] H. Goldstein, C. P. Poole, and J. L. Safko. *Classical Mechanics*. Addison-Wesley, 3 edition, 2002.
- [6] L. D. Landau and E. M. Lifshitz. *Mechanics*, volume 1. Pergamon Press, 3 edition, 1976.
- [7] J. M. Lee. *Introduction to Riemannian Manifolds*. Springer, 2 edition, 2018.
- [8] C. McCord. Are hamiltonian flows geodesic flows? *American Mathematical Monthly*, 110(3):219–230, 2003.

- [9] Y. P. Shah. A study of geodesic equation from variational principle. *Cognition: Journal of Philosophy*, 4(1):31–42, 2023.
- [10] D. Tong. Classical dynamics. Lecture notes, University of Cambridge, 2014.

A Classification of Principal Divisors on $\mathbb{CP}^1$

Pablo Santos Guerrero

MMDA'27

Abstract. Every nonzero polynomial over $\mathbb{C}$ has exactly as many roots as its degree. This paper develops the analogue of that statement for the much larger class of meromorphic functions on the complex projective line $\mathbb{CP}^1 = \mathbb{C} \cup \{\infty\}$, where a function may not only vanish but also blow up. To each such function we attach its divisor: the formal record of where it vanishes and where it has poles, together with the integer multiplicity at each such point. Divisors form an abelian group $\text{Div}(\mathbb{CP}^1)$; those arising from functions form a subgroup $\text{Prin}(\mathbb{CP}^1)$; and the total multiplicity gives a homomorphism $\deg : \text{Div}(\mathbb{CP}^1) \rightarrow \mathbb{Z}$. Our main result is that on $\mathbb{CP}^1$ these two notions agree exactly: $\text{Prin}(\mathbb{CP}^1) = \ker(\deg)$. From this we compute the divisor class group $\text{Cl}(\mathbb{CP}^1) \cong \mathbb{Z}$ and, by an explicit construction with transition functions, the Picard group $\text{Pic}(\mathbb{CP}^1) \cong \mathbb{Z}$.

1 Introduction

1.1 From counting roots to counting zeros and poles

The Fundamental Theorem of Algebra says that a nonzero polynomial $P \in \mathbb{C}[z]$ of degree d has exactly d roots, counted with multiplicity. This is a counting statement: the local data (a root, together with an integer recording how many times it is a root) adds up to a global invariant (the degree).

Polynomials, however, are a very restricted class of functions. As soon as we allow quotients — rational functions such as $(z - 1)^2/z$ — two things change. First, the function may blow up: at $z = 0$ our example has a *pole*, which we should think of as a zero of negative multiplicity. Second, and less obviously, the count above stops balancing unless we look at ∞ as well. Our example has a double zero at $z = 1$ and a simple pole at $z = 0$, and the two multiplicities 2 and -1 do not cancel. The missing contribution sits at infinity: as $|z| \rightarrow \infty$ the function grows like z , and once we make sense of “the order of f at ∞ ” it will turn out to be -1 . The three numbers 2, -1 , -1 now sum to zero.

This is the phenomenon the paper is about. To state it we need a space on which ∞ is an honest point — the complex projective line $\mathbb{CP}^1$, equivalently the Riemann sphere — and a bookkeeping device that records all the local multiplicities at once. That device is the divisor.

1.2 The three objects, informally

Before the formal definitions, here is what the three main objects mean.

- A *divisor* is a finite formal sum $D = \sum_p n_p p$ of points of $\mathbb{CP}^1$ with integer coefficients. It is nothing more than a way of writing down a finite list of points together with an integer attached to each. Divisors add coefficientwise, so they form an abelian group $\text{Div}(\mathbb{CP}^1)$. The divisor $\text{div}(f)$ of a function records its zeros with positive coefficients and its poles with negative ones.
- A divisor is *principal* if it is $\text{div}(f)$ for some nonzero rational function f — that is, if the prescribed configuration of zeros and poles is actually achieved by a function. These form a subgroup $\text{Prin}(\mathbb{CP}^1)$.
- The *degree* $\deg(D) = \sum_p n_p$ adds up all the coefficients. It is the total signed multiplicity, and it is a homomorphism $\text{Div}(\mathbb{CP}^1) \rightarrow \mathbb{Z}$.

The question the paper answers is: *which* divisors are principal? Since not every list of points and integers can plausibly come from a function, one expects some obstruction. The content of the main theorem is that on $\mathbb{CP}^1$ there is exactly one obstruction, and it is the degree.

1.3 Statement of results

Theorem A

On $\mathbb{CP}^1$ the principal divisors are exactly the divisors of degree zero:

$$\text{Prin}(\mathbb{CP}^1) = \ker(\deg : \text{Div}(\mathbb{CP}^1) \rightarrow \mathbb{Z}).$$

This is proved as Theorem 6.3 in Section 6. Two consequences follow.

Theorem B

The degree induces an isomorphism $\text{Cl}(\mathbb{CP}^1) := \text{Div}(\mathbb{CP}^1) / \text{Prin}(\mathbb{CP}^1) \xrightarrow{\sim} \mathbb{Z}$.

Theorem C

The assignment $D \mapsto \mathcal{O}(D)$ induces an isomorphism $\text{Cl}(\mathbb{CP}^1) \cong \text{Pic}(\mathbb{CP}^1)$, and consequently $\text{Pic}(\mathbb{CP}^1) \cong \mathbb{Z}$: every holomorphic line bundle on $\mathbb{CP}^1$ is isomorphic to $\mathcal{O}(n)$ for a unique $n \in \mathbb{Z}$.

Theorem B is Theorem 7.4 and Theorem C is Theorem 8.9. We emphasise that Theorem C is proved here directly, by exhibiting line bundles through their transition functions on the two standard charts, rather than by quoting the general divisor–line-bundle correspondence; see Remark 14 for the relation to the algebraic formulation.

1.4 Prerequisites and outline

We assume the basic theory of holomorphic functions of one variable (power series, Laurent expansion, the maximum modulus principle, Cauchy estimates, and the existence of primitives with vanishing periods), as in Ahlfors [1], together with elementary group theory through

the First Isomorphism Theorem. No prior acquaintance with Riemann surfaces or algebraic geometry is assumed.

Section 1 collects the local analytic facts: every zero or pole of a meromorphic function is measured by a unique integer. Section 3 constructs $\mathbb{CP}^1$ and proves that its meromorphic functions are exactly the rational ones. Section 4 assembles the local integers into divisors. Section 5 records the two exact sequences that organise the material. Section 6 proves Theorem A, Section 7 proves Theorem B, and Section 8 proves Theorem C.

2 Preliminaries from algebra and complex analysis

The goal of this section is a single statement, Corollary 2.7: near any point, a nonzero meromorphic function looks like a power of a local coordinate times a nonvanishing function, and the exponent is a well-defined integer. That integer is what a divisor records.

2.1 The polynomial ring and the rational function field

The polynomial ring over $\mathbb{C}$ is

$$\mathbb{C}[z] = \left\{ \sum_{j=0}^n a_j z^j : a_j \in \mathbb{C}, n \geq 0 \right\},$$

an integral domain, and its field of fractions is the field of rational functions

$$\mathbb{C}(z) = \left\{ \frac{P(z)}{Q(z)} : P, Q \in \mathbb{C}[z], Q \neq 0 \right\}.$$

We write $\mathbb{C}^\times = \mathbb{C} \setminus \{0\}$ and $\mathbb{C}(z)^\times$ for the group of nonzero rational functions under multiplication.

Proposition 2.1: Reduced form

Every $f \in \mathbb{C}(z)^\times$ may be written as

$$f(z) = c \frac{\prod_{i=1}^r (z - a_i)^{m_i}}{\prod_{j=1}^s (z - b_j)^{n_j}}, \quad c \in \mathbb{C}^\times,$$

where $r, s \geq 0$ (empty products being 1), the integers m_i, n_j are positive, and the $r + s$ points $a_1, \dots, a_r, b_1, \dots, b_s \in \mathbb{C}$ are *pairwise distinct*. In such an expression the a_i are precisely the zeros of f in $\mathbb{C}$, with multiplicities m_i , and the b_j are precisely its poles in $\mathbb{C}$, with orders n_j .

Proof. Write $f = P/Q$ with $P, Q \in \mathbb{C}[z]$, $Q \neq 0$. Since $\mathbb{C}$ is algebraically closed, the Fundamental Theorem of Algebra factors both into linear factors,

$$P(z) = c \prod_i (z - \alpha_i)^{k_i}, \quad Q(z) = \prod_j (z - \beta_j)^{\ell_j},$$

with the α_i pairwise distinct and the β_j pairwise distinct. If some α_i equals some β_j , cancel the common factor $(z - \alpha_i)^{\min(k_i, \ell_j)}$; this is legitimate because $\mathbb{C}[z]$ is an integral domain, and it strictly decreases $\deg P + \deg Q$, so after finitely many steps no α_i coincides with any β_j . Discarding the factors with exponent 0 and renaming, we obtain the displayed form with all

$r + s$ points pairwise distinct.

For the final assertion, fix a point $p \in \mathbb{C}$. If $p \notin \{a_i\} \cup \{b_j\}$, then every factor is holomorphic and nonzero at p , so $f(p) \in \mathbb{C}^\times$. If $p = a_i$, then $f(z) = (z - a_i)^{m_i} u(z)$ where u collects the remaining factors and the constant; u is holomorphic and nonzero at a_i precisely because no b_j equals a_i . The case $p = b_j$ is identical with $(z - b_j)^{-n_j}$ in place of $(z - a_i)^{m_i}$. $\square$

Remark 1. The cancellation step is not a formality: without it, $f = (z - 1)^2 / (z - 1)$ satisfies the displayed formula with $a_1 = b_1 = 1$, and the reading of the exponents as zero and pole orders fails. The reduced form is used in an essential way in Proposition 6.2.

Definition 1

Let $P \in \mathbb{C}[z]$ and $a \in \mathbb{C}$. We say a is a zero of multiplicity m of P if $P(z) = (z - a)^m Q(z)$ with $Q \in \mathbb{C}[z]$ and $Q(a) \neq 0$.

Example 1. Let $P(z) = (z - 1)^2(z + 2)$. At $a = 1$ we may write $P(z) = (z - 1)^2 Q(z)$ with $Q(z) = z + 2$ and $Q(1) = 3 \neq 0$, so 1 is a zero of multiplicity 2. At $a = -2$ we write $P(z) = (z + 2) R(z)$ with $R(z) = (z - 1)^2$ and $R(-2) = 9 \neq 0$, so -2 is a zero of multiplicity 1. At every other point P does not vanish. The multiplicities sum to $3 = \deg P$, as Proposition 2.2 records in general.

Remark 2. Definition 1 is stated for polynomials because that is all Proposition 2.2 needs. It is the special case, for f a polynomial and $m \geq 1$, of the general notion of local order established in Corollary 2.7 below, which applies to any meromorphic function and allows m to be negative.

Proposition 2.2

If $P(z) = c \prod_{j=1}^r (z - a_j)^{m_j}$ with $c \in \mathbb{C}^\times$, then $\deg(P) = \sum_{j=1}^r m_j$.

Proof. Since $\mathbb{C}[z]$ is an integral domain, the degree of a product of nonzero polynomials is the sum of the degrees. Each factor $(z - a_j)^{m_j}$ has degree m_j and the constant c has degree 0, whence the claim. $\square$

2.2 Laurent expansions, poles, and meromorphic functions

Theorem 2.3: Laurent expansion; [1, Ch. 5]

Let $A = \{z \in \mathbb{C} : 0 < |z - a| < R\}$ and let f be holomorphic on A . Then there are unique coefficients $c_n \in \mathbb{C}$ ($n \in \mathbb{Z}$) with

$$f(z) = \sum_{n=-\infty}^{\infty} c_n (z - a)^n \quad (z \in A),$$

the convergence being absolute and locally uniform on A . The coefficients are given by $c_n = \frac{1}{2\pi i} \oint_{|z-a|=\rho} \frac{f(z)}{(z-a)^{n+1}} dz$ for any $0 < \rho < R$.

The negative-power part of the expansion classifies the singularity at a : it vanishes identically for a removable singularity, has finitely many nonzero terms for a pole, and infinitely many for an essential singularity.

Definition 2

Let f be holomorphic on a punctured neighbourhood of $a \in \mathbb{C}$. We say a is a *pole of order m* of f , where $m \geq 1$, if the Laurent expansion of f at a has the form

$$f(z) = \sum_{n=-m}^{\infty} c_n(z-a)^n, \quad c_{-m} \neq 0.$$

Only now can we say what a meromorphic function is.

Definition 3

Let $U \subseteq \mathbb{C}$ be open. A function f is *meromorphic* on U if there is a subset $S \subseteq U$, discrete and closed in U , such that f is holomorphic on $U \setminus S$ and every point of S is a pole of f in the sense of Definition 2.

Remark 3. The requirement that each singularity be a *pole* is what the word meromorphic contributes: essential singularities are excluded by definition. Thus $e^{1/z}$ is not meromorphic on any neighbourhood of 0, whereas $1/z^2$ is. Equivalently, f is meromorphic on U if and only if it extends to a holomorphic map $U \rightarrow \mathbb{CP}^1$ that is not identically ∞ ; we will use this formulation in Section 3.

Proposition 2.4

If a is a pole of order m of f , then there is a function u , holomorphic on a full neighbourhood of a , with

$$f(z) = (z-a)^{-m}u(z), \quad u(a) \neq 0.$$

Proof. By Definition 2, $f(z) = \sum_{n \geq -m} c_n(z-a)^n$ on a punctured disc $0 < |z-a| < R$, with $c_{-m} \neq 0$. Factoring out $(z-a)^{-m}$ gives $f(z) = (z-a)^{-m} \sum_{n \geq -m} c_n(z-a)^{n+m}$. Reindexing by $k = n+m \geq 0$, define

$$u(z) := \sum_{k=0}^{\infty} c_{k-m}(z-a)^k.$$

This is a power series in nonnegative powers of $z-a$. It converges wherever the original Laurent series does, hence on $0 < |z-a| < R$; but a power series converging on a punctured disc converges on the whole disc, since its radius of convergence is determined by its coefficients alone. Therefore u is holomorphic on $|z-a| < R$, in particular at a — which is the point of the reindexing, since f itself was only defined on the punctured disc. Finally $u(a) = c_{-m} \neq 0$. $\square$

2.3 Local factorisation and the local order**Proposition 2.5**

Let f be holomorphic near $a \in \mathbb{C}$ and not identically zero near a . Then either $f(a) \neq 0$, or there exist a unique integer $m \geq 1$ and a holomorphic function u near a with

$$f(z) = (z-a)^m u(z), \quad u(a) \neq 0.$$

Proof. Expand $f(z) = \sum_{n \geq 0} c_n(z-a)^n$ near a . If $c_0 = f(a) \neq 0$ there is nothing to prove. Otherwise, since $f \not\equiv 0$, not all c_n vanish; let $m \geq 1$ be least with $c_m \neq 0$. Then $f(z) =$

$(z - a)^m \sum_{k \geq 0} c_{k+m} (z - a)^k$, and as in Proposition 2.4 the series $u(z) := \sum_{k \geq 0} c_{k+m} (z - a)^k$ is holomorphic near a with $u(a) = c_m \neq 0$.

Uniqueness is proved in Lemma 2.6 below, in the generality we shall need. $\square$

Lemma 2.6: Uniqueness of the local exponent

Let $a \in \mathbb{C}$ and suppose

$$(z - a)^m u(z) = (z - a)^n v(z)$$

on a punctured neighbourhood of a , where $m, n \in \mathbb{Z}$ and u, v are holomorphic and nonvanishing at a . Then $m = n$ and $u = v$.

Proof. Suppose $m \neq n$; without loss of generality $m < n$. Dividing by $(z - a)^m$ — legitimate for $z \neq a$ — gives $u(z) = (z - a)^{n-m} v(z)$ on a punctured neighbourhood. Both sides are holomorphic at a and agree off a , so by continuity they agree at a as well; since $n - m \geq 1$ the right-hand side vanishes at a , giving $u(a) = 0$, contrary to hypothesis. Hence $m = n$, and then $u = v$ off a and so, again by continuity, everywhere near a . $\square$

Corollary 2.7: Local order

Let f be meromorphic and not identically zero near $a \in \mathbb{C}$. Then there are a unique $m \in \mathbb{Z}$ and a function u , holomorphic and nonvanishing at a , with $f(z) = (z - a)^m u(z)$ near a . We call m the *order* of f at a .

Proof. Existence: if f has a pole at a apply Proposition 2.4 (with $m < 0$); otherwise f is holomorphic at a and we apply Proposition 2.5, taking $m = 0$ and $u = f$ when $f(a) \neq 0$. Uniqueness is Lemma 2.6. $\square$

Thus zeros and poles alike are measured by a single integer, positive for zeros, negative for poles, zero when the function is finite and nonvanishing. This is the fact that makes the divisor construction possible.

3 The Riemann sphere and the projective line

We now build the space on which ∞ becomes an ordinary point. We need only its construction, its compactness, its two charts, and the identification of its meromorphic functions.

3.1 The complex projective line

Definition 4

The *complex projective line* is $\mathbb{CP}^1 = (\mathbb{C}^2 \setminus \{(0, 0)\}) / \sim$, where $(X, Y) \sim (\lambda X, \lambda Y)$ for all $\lambda \in \mathbb{C}^\times$. The class of (X, Y) is written $[X : Y]$.

We give $\mathbb{CP}^1$ the quotient topology induced by $\pi : \mathbb{C}^2 \setminus \{(0, 0)\} \rightarrow \mathbb{CP}^1$, so that $U \subseteq \mathbb{CP}^1$ is open exactly when $\pi^{-1}(U)$ is open.

Proposition 3.1

$\mathbb{CP}^1$ is compact and connected.

Proof. Let $S^3 = \{(X, Y) \in \mathbb{C}^2 : |X|^2 + |Y|^2 = 1\}$, a compact connected subset of $\mathbb{C}^2 \setminus \{(0, 0)\}$. Every class $[X : Y]$ has a representative on S^3 , namely $(X, Y) / \sqrt{|X|^2 + |Y|^2}$, so $\pi|_{S^3} : S^3 \rightarrow \mathbb{CP}^1$ is surjective. It is continuous, and continuous images of compact (respectively connected) spaces are compact (respectively connected). $\square$

Remark 4. That $\mathbb{CP}^1$ is Hausdorff and second countable is a routine verification which we omit; both are needed for $\mathbb{CP}^1$ to be a complex manifold in the usual sense, and neither is used in any argument below beyond the standing assumption that $\mathbb{CP}^1$ is a Riemann surface.

3.2 The standard affine charts

Put $U_0 = \{[X : Y] : Y \neq 0\}$ and $U_1 = \{[X : Y] : X \neq 0\}$, and define

$$\phi_0 : U_0 \rightarrow \mathbb{C}, [X : Y] \mapsto \frac{X}{Y}, \quad \phi_1 : U_1 \rightarrow \mathbb{C}, [X : Y] \mapsto \frac{Y}{X}.$$

Proposition 3.2

ϕ_0 and ϕ_1 are bijections; each of U_0, U_1 is thus identified with a copy of $\mathbb{C}$.

Proof. We argue for ϕ_0 . Let $\psi_0 : \mathbb{C} \rightarrow U_0, z \mapsto [z : 1]$. Then $\phi_0(\psi_0(z)) = z/1 = z$. Conversely let $[X : Y] \in U_0$, so $Y \neq 0$; taking $\lambda = Y^{-1}$ in the equivalence relation gives $[X : Y] = [X/Y : 1]$, whence $\psi_0(\phi_0([X : Y])) = [X/Y : 1] = [X : Y]$. So ϕ_0 is a bijection with inverse ψ_0 . Similarly ϕ_1 is a bijection with inverse $\psi_1(z) = [1 : z]$. $\square$

On the overlap $U_0 \cap U_1 = \{[X : Y] : X, Y \neq 0\}$ the transition map is

$$\phi_1 \circ \phi_0^{-1}(z) = \frac{1}{z}, \quad z \in \mathbb{C}^\times,$$

which is holomorphic. Hence the two charts make $\mathbb{CP}^1$ a Riemann surface.

3.3 The point at infinity

Every point of U_0 is uniquely $[z : 1]$ with $z \in \mathbb{C}$, so U_0 is a copy of $\mathbb{C}$. Exactly one point of $\mathbb{CP}^1$ is missing from U_0 , namely $[1 : 0]$. We call it the *point at infinity* and write ∞ , so that

$$\mathbb{CP}^1 = \mathbb{C} \cup \{\infty\}.$$

Remark 5 (What ∞ is doing). It is worth pausing on why this one extra point changes anything. In $\mathbb{C}$, the statement “ $f(z)$ tends to infinity as $|z| \rightarrow \infty$ ” is a statement about a limiting process, not about the value of f anywhere. Adjoining ∞ turns it into an ordinary local statement: the chart U_1 supplies the coordinate $w = 1/z$ centred at ∞ , and the behaviour of f “for large z ” becomes the behaviour of $w \mapsto f(1/w)$ near $w = 0$ — an ordinary zero or pole, to be analysed by Corollary 2.7 like any other. Concretely, the polynomial z has no zeros or poles in $\mathbb{C}$ apart from its zero at the origin, and looks unbalanced; on $\mathbb{CP}^1$ it has a simple zero at 0 and a simple pole at ∞ , and balances. Geometrically, $\mathbb{CP}^1$ is the sphere obtained by wrapping the plane up and capping it off with a single point, which is why it is also called the Riemann sphere; compactness (Proposition 3.1) is the precise form of “capped off”, and it is what drives every global result below.

3.4 Meromorphic functions on $\mathbb{CP}^1$

Definition 5

A function f on $\mathbb{CP}^1$ is *meromorphic* if it is not identically ∞ and if, in each of the two charts, it is represented by a meromorphic function of one complex variable in the sense of Definition 3. Explicitly: $z \mapsto f(z)$ is meromorphic on $\mathbb{C}$, and $w \mapsto f(1/w)$ is meromorphic near $w = 0$.

Remark 6. The exclusion of the constant ∞ is needed: as a holomorphic map $\mathbb{CP}^1 \rightarrow \mathbb{CP}^1$ it is perfectly good, but it is not a rational function and would falsify Theorem 3.3.

Theorem 3.3

Every meromorphic function on $\mathbb{CP}^1$ is rational: on the chart $\mathbb{C} = U_0$ it is given by an element of $\mathbb{C}(z)$.

Proof. Let f be meromorphic on $\mathbb{CP}^1$. The set of poles of f is discrete and closed in $\mathbb{CP}^1$; as $\mathbb{CP}^1$ is compact (Proposition 3.1), it is finite. Let $p_1, \dots, p_k \in \mathbb{C}$ be the poles lying in $\mathbb{C}$, of orders $k_1, \dots, k_k$, and put $K = \sum_i k_i$ and $g(z) := f(z) \prod_{i=1}^k (z - p_i)^{k_i}$.

By Proposition 2.4, multiplying by $(z - p_i)^{k_i}$ removes the pole at p_i , so g is holomorphic on all of $\mathbb{C}$, i.e. entire. We claim it has polynomial growth. Indeed, f is meromorphic at ∞ , so in the coordinate $w = 1/z$ it has at worst a pole there, say of order $N \geq 0$; hence $|f(z)| \leq C_1 |z|^N$ for $|z|$ large. Also $|\prod_i (z - p_i)^{k_i}| \leq C_2 |z|^K$ for $|z|$ large. Therefore $|g(z)| \leq C |z|^{N+K}$ for $|z| \geq R$, for suitable constants.

An entire function with $|g(z)| \leq C |z|^d$ for large $|z|$ is a polynomial of degree at most d : by the Cauchy estimates, its Taylor coefficients satisfy $|c_n| \leq C \rho^{d-n}$ for all $\rho \geq R$, which forces $c_n = 0$ for $n > d$. Hence $g \in \mathbb{C}[z]$ and $f = g / \prod_i (z - p_i)^{k_i} \in \mathbb{C}(z)$. $\square$

Remark 7. Theorem 3.3 is the reason a polynomial deserves to be viewed on $\mathbb{CP}^1$: as a function on $\mathbb{C}$ it is holomorphic, but as a meromorphic function on $\mathbb{CP}^1$ it acquires a pole at ∞ , of order equal to its degree (Example 6). From now on we use “meromorphic function on $\mathbb{CP}^1$ ” and “element of $\mathbb{C}(z)$ ” interchangeably.

4 Orders and divisors on $\mathbb{CP}^1$

4.1 Local parameters and the order at a point

To treat ∞ on the same footing as every other point we fix, once and for all, a coordinate centred at each point.

Definition 6

For $p \in \mathbb{CP}^1$ the *local parameter* at p is

$$t_p(z) := z - p \quad (p \in \mathbb{C}), \quad t_\infty(z) := \frac{1}{z}.$$

In both cases t_p is a holomorphic coordinate on a neighbourhood of p vanishing exactly to

order one at p : for finite p this is clear, and $t_\infty = w$ is precisely the coordinate of the chart U_1 .

Proposition 4.1

Let $f \in \mathbb{C}(z)^\times$ and $p \in \mathbb{CP}^1$. There are a unique integer m and a function u , holomorphic and nonvanishing at p , with $f = t_p^m u$ near p . We write $\text{ord}_p(f) := m$.

Proof. For $p \in \mathbb{C}$ this is Corollary 2.7. For $p = \infty$, apply Corollary 2.7 to the function $w \mapsto f(1/w)$ at $w = 0$, which is meromorphic there by Definition 5 and not identically zero since $f \neq 0$; the resulting factorisation $f(1/w) = w^m u(w)$ with $u(0) \neq 0$ is the assertion, as $t_\infty = w$. Uniqueness in both cases is Lemma 2.6. $\square$

So $\text{ord}_p(f) > 0$ means f has a zero of that order at p , $\text{ord}_p(f) < 0$ means a pole of order $-\text{ord}_p(f)$, and $\text{ord}_p(f) = 0$ means f is finite and nonzero at p . Note that $\text{ord}_\infty(f) = \text{ord}_0(f(1/w))$, which is the concrete recipe for computing at infinity.

Example 2. For $f(z) = z$ we have $f(1/w) = 1/w = w^{-1}$, so $\text{ord}_\infty(z) = -1$: the identity function has a simple pole at infinity. More generally $f(z) = z^n$ gives $f(1/w) = w^{-n}$, so $\text{ord}_\infty(z^n) = -n$.

Proposition 4.2

For all $f, g \in \mathbb{C}(z)^\times$ and all $p \in \mathbb{CP}^1$, $\text{ord}_p(fg) = \text{ord}_p(f) + \text{ord}_p(g)$.

Proof. Near p write $f = t_p^m u$ and $g = t_p^n v$ with $m = \text{ord}_p(f)$, $n = \text{ord}_p(g)$ and u, v holomorphic and nonvanishing at p (Proposition 4.1). Then $fg = t_p^{m+n}(uv)$, and uv is again holomorphic and nonvanishing at p . By the uniqueness in Proposition 4.1, $\text{ord}_p(fg) = m + n$. $\square$

4.2 The divisor group

Each point of $\mathbb{CP}^1$ contributes one integer to the description of a function: its order there. All but finitely many of these integers are zero, since a nonzero rational function has only finitely many zeros and poles. A divisor is simply the record of all of them at once — a bookkeeping device which turns the local integers of Proposition 4.1 into a single global object that can be added, subtracted, and quotiented.

Definition 7

The *divisor group* of $\mathbb{CP}^1$ is the free abelian group on the points of $\mathbb{CP}^1$,

$$\text{Div}(\mathbb{CP}^1) := \bigoplus_{p \in \mathbb{CP}^1} \mathbb{Z} \cdot p.$$

Its elements are the finite formal sums $D = \sum_{p \in \mathbb{CP}^1} n_p(p)$ with $n_p \in \mathbb{Z}$ almost all zero, added coefficientwise. The *support* of D is $\text{supp}(D) = \{p : n_p \neq 0\}$, a finite set.

We write the point p regarded as a divisor as (p) , so that a typical divisor reads $2(a) - 3(b) + (\infty)$. Being free abelian on the point set is exactly what makes the quotient computations of Sections 7 formal.

Definition 8

A divisor $D = \sum_p n_p(p)$ is *effective*, written $D \geq 0$, if $n_p \geq 0$ for every p . For divisors D, E we write $D \geq E$ to mean $D - E \geq 0$. For $U \subseteq \mathbb{CP}^1$ we write $D|_U := \sum_{p \in U} n_p(p)$ for the divisor obtained by discarding the points outside U .

Definition 9

The *degree* of $D = \sum_p n_p(p)$ is $\deg(D) := \sum_p n_p \in \mathbb{Z}$.

Proposition 4.3

$\deg : \text{Div}(\mathbb{CP}^1) \rightarrow \mathbb{Z}$ is a group homomorphism.

Proof. Addition of divisors is coefficientwise, so $\deg(D + E) = \sum_p (n_p + m_p) = \sum_p n_p + \sum_p m_p = \deg(D) + \deg(E)$, all sums being finite. $\square$

Example 3. If $D = 2(a) - 3(b) + (\infty)$ with $a, b \in \mathbb{C}$ distinct, then $\deg(D) = 2 - 3 + 1 = 0$. The degree is a *signed* count: it does not merely tally the points of the support.

4.3 The divisor of a rational function**Definition 10**

For $f \in \mathbb{C}(z)^\times$ the *divisor of f* is

$$\text{div}(f) := \sum_{p \in \mathbb{CP}^1} \text{ord}_p(f)(p).$$

This is a finite sum, hence really an element of $\text{Div}(\mathbb{CP}^1)$: by Proposition 2.1 a nonzero rational function has finitely many zeros and poles in $\mathbb{C}$, and ∞ contributes at most one further nonzero term.

Proposition 4.4

The map $\text{div} : \mathbb{C}(z)^\times \rightarrow \text{Div}(\mathbb{CP}^1)$ is a group homomorphism from the multiplicative group of nonzero rational functions to the additive group of divisors.

Proof. By Proposition 4.2, for every p we have $\text{ord}_p(fg) = \text{ord}_p(f) + \text{ord}_p(g)$. Summing over p ,

$$\text{div}(fg) = \sum_p \text{ord}_p(fg)(p) = \sum_p (\text{ord}_p(f) + \text{ord}_p(g))(p) = \text{div}(f) + \text{div}(g). \quad \square$$

We record the basic computations, in increasing generality.

Proposition 4.5

For every $a \in \mathbb{C}$, $\text{div}(z - a) = (a) - (\infty)$.

Proof. The function $z - a$ has a simple zero at a and no other zero or pole in $\mathbb{C}$, so $\text{ord}_a = 1$ and

$\text{ord}_p = 0$ for $p \in \mathbb{C} \setminus \{a\}$. At infinity, substituting $z = 1/w$,

$$z - a = \frac{1}{w} - a = w^{-1}(1 - aw),$$

and $1 - aw$ is holomorphic and nonzero at $w = 0$; by Proposition 4.1, $\text{ord}_\infty(z - a) = -1$. Hence $\text{div}(z - a) = (a) - (\infty)$. $\square$

Proposition 4.6

$\text{div}(c) = 0$ for every $c \in \mathbb{C}^\times$.

Proof. A nonzero constant is holomorphic and nonvanishing at every point of $\mathbb{CP}^1$, so $\text{ord}_p(c) = 0$ for all p . (The converse — that only constants have zero divisor — is Proposition 5.2.) $\square$

Example 4. Let us compute $\text{div}(f)$ for $f(z) = \frac{(z-1)^2}{z}$ point by point.

- At $p = 1$. Write $f(z) = (z-1)^2 u(z)$ with $u(z) = 1/z$, which is holomorphic and nonzero at 1. So $\text{ord}_1(f) = 2$.
- At $p = 0$. Write $f(z) = z^{-1} v(z)$ with $v(z) = (z-1)^2$, which is holomorphic and nonzero at 0 (indeed $v(0) = 1$). So $\text{ord}_0(f) = -1$.
- At other finite p . Both numerator and denominator are holomorphic and nonzero, so $\text{ord}_p(f) = 0$.
- At $p = \infty$. Substituting $z = 1/w$,

$$f(1/w) = \frac{(w^{-1} - 1)^2}{w^{-1}} = \frac{w^{-2}(1 - w)^2}{w^{-1}} = w^{-1}(1 - w)^2,$$

and $(1 - w)^2$ is holomorphic and nonzero at $w = 0$. So $\text{ord}_\infty(f) = -1$.

Therefore

$$\text{div}(f) = 2(1) - (0) - (\infty), \quad \deg \text{div}(f) = 2 - 1 - 1 = 0.$$

The zero total is not an accident; it is Theorem A, and the reader may find it instructive to see where the contribution at ∞ came from: the numerator outgrew the denominator by one degree.

Example 5. Similarly, for $f(z) = \frac{z^2 - 1}{z^3}$ one finds simple zeros at 1 and -1 , a pole of order 3 at 0, and, since the denominator outgrows the numerator by one degree, $\text{ord}_\infty(f) = 3 - 2 = 1$. Hence

$$\text{div}(f) = (1) + (-1) - 3(0) + (\infty),$$

of degree $1 + 1 - 3 + 1 = 0$.

Example 6. Let $P(z) = c \prod_{j=1}^r (z - a_j)^{m_j}$ be a nonzero polynomial. By Propositions 4.4, 4.5 and 4.6,

$$\text{div}(P) = \sum_{j=1}^r m_j [(a_j) - (\infty)] = \sum_{j=1}^r m_j (a_j) - \left(\sum_{j=1}^r m_j \right) (\infty) = \sum_{j=1}^r m_j (a_j) - (\deg P)(\infty),$$

the last step by Proposition 2.2. So a polynomial of degree d has a pole of order exactly d at infinity, and its divisor has degree 0.

4.4 Principal divisors

Definition 11

A divisor is *principal* if it equals $\text{div}(f)$ for some $f \in \mathbb{C}(z)^\times$. The set of principal divisors is

$$\text{Prin}(\mathbb{CP}^1) := \text{im}(\text{div} : \mathbb{C}(z)^\times \rightarrow \text{Div}(\mathbb{CP}^1)),$$

a subgroup of $\text{Div}(\mathbb{CP}^1)$, since it is the image of the group homomorphism div (Proposition 4.4).

Thus the principal divisors are exactly the zero–pole configurations that some meromorphic function actually realises, and the question of the paper is to determine which those are.

5 The two exact sequences

Two questions organise what follows.

- Which divisors come from meromorphic functions — and how much of a function is remembered by its divisor? This is answered in this section, by the first exact sequence, and completed by Theorem A.
- How far is an arbitrary divisor from being principal? The second exact sequence sets up the quotient that measures this; the measurement itself is carried out in Section 7.

5.1 The kernel of the divisor map

Lemma 5.1

Every function holomorphic on all of $\mathbb{CP}^1$ is constant.

Proof. Let h be holomorphic on $\mathbb{CP}^1$. Then $|h|$ is continuous on the compact space $\mathbb{CP}^1$ (Proposition 3.1), so it attains a maximum at some $p_0 \in \mathbb{CP}^1$. Choose a chart around p_0 ; in it h is a holomorphic function of one variable whose modulus has an interior maximum, so by the maximum modulus principle [1, Ch. 4] h is constant, say equal to c , on a neighbourhood of p_0 . The set $\{p \in \mathbb{CP}^1 : h(p) = c\}$ is therefore nonempty and open; it is closed by continuity; and $\mathbb{CP}^1$ is connected (Proposition 3.1). Hence it is all of $\mathbb{CP}^1$. $\square$

Proposition 5.2

$\ker(\text{div}) = \mathbb{C}^\times$.

Proof. If $c \in \mathbb{C}^\times$ then $\text{div}(c) = 0$ by Proposition 4.6, so $\mathbb{C}^\times \subseteq \ker(\text{div})$. Conversely let $f \in \mathbb{C}(z)^\times$ with $\text{div}(f) = 0$. Then $\text{ord}_p(f) = 0$ for every $p \in \mathbb{CP}^1$, so f has no poles and is holomorphic on all of $\mathbb{CP}^1$. By Lemma 5.1 f is constant, and it is nonzero, so $f \in \mathbb{C}^\times$. $\square$

Remark 8. Equivalently: two rational functions have the same divisor if and only if they differ by a nonzero constant multiple. Indeed $\text{div}(f) = \text{div}(g)$ gives $\text{div}(f/g) = 0$, so $f/g \in \mathbb{C}^\times$. A divisor therefore remembers a function exactly up to scale — no more and no less.

5.2 The first exact sequence

Proposition 5.3

The sequence

$$1 \longrightarrow \mathbb{C}^\times \longrightarrow \mathbb{C}(z)^\times \xrightarrow{\text{div}} \text{Prin}(\mathbb{CP}^1) \longrightarrow 1$$

is exact.

Proof. The first map is the inclusion of constants, which is injective. Exactness at $\mathbb{C}(z)^\times$ is Proposition 5.2. Exactness at $\text{Prin}(\mathbb{CP}^1)$ is the surjectivity of div onto its image, which holds by Definition 11. $\square$

Remark 9. By the First Isomorphism Theorem this says $\text{Prin}(\mathbb{CP}^1) \cong \mathbb{C}(z)^\times / \mathbb{C}^\times$: the group of principal divisors is the group of nonzero rational functions modulo scaling.

5.3 Linear equivalence and the divisor class group

Definition 12

Divisors $D, E \in \text{Div}(\mathbb{CP}^1)$ are *linearly equivalent*, written $D \sim E$, if $D - E \in \text{Prin}(\mathbb{CP}^1)$.

Proposition 5.4

Linear equivalence is an equivalence relation on $\text{Div}(\mathbb{CP}^1)$.

Proof. This is exactly the statement that $\text{Prin}(\mathbb{CP}^1)$ is a subgroup of the abelian group $\text{Div}(\mathbb{CP}^1)$ (Definition 11). Reflexivity: $D - D = 0 = \text{div}(1)$. Symmetry: if $D - E = \text{div}(f)$ then $E - D = -\text{div}(f) = \text{div}(f^{-1})$. Transitivity: if $D - E = \text{div}(f)$ and $E - F = \text{div}(g)$ then $D - F = \text{div}(f) + \text{div}(g) = \text{div}(fg)$, using Proposition 4.4 throughout. $\square$

Definition 13

The *divisor class group* of $\mathbb{CP}^1$ is the quotient

$$\text{Cl}(\mathbb{CP}^1) := \text{Div}(\mathbb{CP}^1) / \text{Prin}(\mathbb{CP}^1),$$

whose elements are the linear equivalence classes $[D]$.

5.4 The second exact sequence

Proposition 5.5

The sequence $0 \rightarrow \text{Prin}(\mathbb{CP}^1) \rightarrow \text{Div}(\mathbb{CP}^1) \rightarrow \text{Cl}(\mathbb{CP}^1) \rightarrow 0$ is exact.

Proof. The first map is the inclusion of a subgroup, hence injective. The second is the quotient map $q : D \mapsto [D]$, which is surjective by construction and has kernel $\text{Prin}(\mathbb{CP}^1)$ by the definition of a quotient group. $\square$

6 Principal divisors and degree

We now prove Theorem A in two halves.

6.1 Every principal divisor has degree zero

Proposition 6.1

For every $f \in \mathbb{C}(z)^\times$ we have $\deg(\operatorname{div}(f)) = 0$. Equivalently, $\operatorname{Prin}(\mathbb{CP}^1) \subseteq \ker(\deg)$.

Proof. By Proposition 2.1 write f in reduced form,

$$f(z) = c \frac{\prod_{i=1}^r (z - a_i)^{m_i}}{\prod_{j=1}^s (z - b_j)^{n_j}}, \quad c \in \mathbb{C}^\times,$$

with all $r + s$ points pairwise distinct. By the last assertion of Proposition 2.1, the zeros of f in $\mathbb{C}$ are exactly the a_i with $\operatorname{ord}_{a_i}(f) = m_i$, the poles in $\mathbb{C}$ are exactly the b_j with $\operatorname{ord}_{b_j}(f) = -n_j$, and $\operatorname{ord}_p(f) = 0$ at all other finite p . Set $M := \sum_i m_i$ and $N := \sum_j n_j$.

It remains to compute $\operatorname{ord}_\infty(f)$. Substituting $z = 1/w$,

$$f(1/w) = c \frac{\prod_i (w^{-1} - a_i)^{m_i}}{\prod_j (w^{-1} - b_j)^{n_j}} = c \frac{\prod_i w^{-m_i} (1 - a_i w)^{m_i}}{\prod_j w^{-n_j} (1 - b_j w)^{n_j}} = w^{N-M} g(w),$$

where

$$g(w) := c \frac{\prod_i (1 - a_i w)^{m_i}}{\prod_j (1 - b_j w)^{n_j}}.$$

Each factor $1 - a_i w$ and $1 - b_j w$ takes the value 1 at $w = 0$, so g is holomorphic near $w = 0$ with $g(0) = c \neq 0$. Thus $f(1/w) = w^{N-M} g(w)$ is a local factorisation at $w = 0$ in the sense of Proposition 4.1, and therefore

$$\operatorname{ord}_\infty(f) = N - M = \sum_{j=1}^s n_j - \sum_{i=1}^r m_i.$$

Adding up all the local contributions,

$$\deg(\operatorname{div}(f)) = \sum_{i=1}^r m_i - \sum_{j=1}^s n_j + \operatorname{ord}_\infty(f) = M - N + (N - M) = 0. \quad \square$$

Remark 10. This is the precise sense in which a meromorphic function on $\mathbb{CP}^1$ has “as many zeros as poles”. Specialising to a polynomial P recovers the Fundamental Theorem of Algebra in the form of Example 6: the $\deg P$ roots in $\mathbb{C}$ are balanced by a pole of order $\deg P$ at infinity.

6.2 Every divisor of degree zero is principal

Proposition 6.2

Let $D \in \operatorname{Div}(\mathbb{CP}^1)$ with $\deg(D) = 0$. Then D is principal.

Proof. Write $D = \sum_{k=1}^r n_k(a_k) + n_\infty(\infty)$ with $a_1, \dots, a_r \in \mathbb{C}$ distinct and $n_k, n_\infty \in \mathbb{Z}$; every divisor has this shape, the point ∞ being separated off from the finite ones. Since $\deg(D) =$

$\sum_k n_k + n_\infty = 0$ we have $n_\infty = -\sum_k n_k$, and so

$$D = \sum_{k=1}^r n_k(a_k) - \left(\sum_{k=1}^r n_k\right)(\infty) = \sum_{k=1}^r n_k[(a_k) - (\infty)].$$

By Proposition 4.5 each bracket is $\operatorname{div}(z - a_k)$, and div is a homomorphism (Proposition 4.4), so

$$D = \sum_{k=1}^r n_k \operatorname{div}(z - a_k) = \operatorname{div}\left(\prod_{k=1}^r (z - a_k)^{n_k}\right),$$

where negative exponents are read as factors in the denominator. Hence D is principal. $\square$

Remark 11. The proof is constructive: given a degree-zero divisor it writes down a function realising it. Note also that the argument handles the coefficient at ∞ automatically, by using $\deg(D) = 0$ to eliminate n_∞ before building the function; this is necessary, since the very first nontrivial principal divisor $\operatorname{div}(z) = (0) - (\infty)$ has ∞ in its support.

6.3 The main theorem

Theorem 6.3

On $\mathbb{CP}^1$ one has $\operatorname{Prin}(\mathbb{CP}^1) = \ker(\deg)$. That is, a divisor is principal if and only if its degree is zero.

Proof. Proposition 6.1 gives $\operatorname{Prin}(\mathbb{CP}^1) \subseteq \ker(\deg)$, and Proposition 6.2 gives $\ker(\deg) \subseteq \operatorname{Prin}(\mathbb{CP}^1)$. $\square$

Corollary 6.4

$\ker(\deg)$ is generated as a group by the differences $(p) - (q)$ with $p, q \in \mathbb{CP}^1$, and each such difference is principal.

Proof. The proof of Proposition 6.2 exhibits an arbitrary degree-zero divisor as an integer combination of the differences $(a_k) - (\infty)$, so these already generate. Each $(p) - (q)$ has degree 0 and is therefore principal by Theorem 6.3; explicitly $(a) - (b) = \operatorname{div}((z - a)/(z - b))$ for finite a, b , and $(a) - (\infty) = \operatorname{div}(z - a)$. $\square$

Remark 12 (Why $\mathbb{CP}^1$ is special). Theorem 6.3 says that on the Riemann sphere the degree is the *only* obstruction to a divisor being principal. This is a genuinely special feature of $\mathbb{CP}^1$ and not a general phenomenon. On a compact Riemann surface X of genus g , the degree-zero part of the class group is

$$\operatorname{Cl}^0(X) \cong \operatorname{Jac}(X),$$

a complex torus of dimension g called the Jacobian of X . Thus a degree-zero divisor is principal precisely when its class in this torus vanishes, which for $g \geq 1$ is a nontrivial condition on a positive-dimensional family of classes. The sphere is the case $g = 0$, where the torus is a single point and the condition is vacuous — which is exactly Theorem 6.3. Everything below should be read as the genus-zero shadow of a much larger theory; see Section 10.

7 The divisor class group

7.1 The degree descends to classes

Proposition 7.1

The degree induces a well-defined group homomorphism $\overline{\deg} : \text{Cl}(\mathbb{CP}^1) \rightarrow \mathbb{Z}$, $[D] \mapsto \deg(D)$.

Proof. If $[D] = [E]$ then $D - E \in \text{Prin}(\mathbb{CP}^1) = \ker(\deg)$ by Theorem 6.3, so $\deg(D) - \deg(E) = \deg(D - E) = 0$. Hence $\deg(D)$ depends only on $[D]$. It is a homomorphism because $\deg$ is one and the quotient map is one. $\square$

Proposition 7.2

$\overline{\deg}$ is surjective.

Proof. Given $n \in \mathbb{Z}$, pick any $p \in \mathbb{CP}^1$; then $\overline{\deg}([n(p)]) = \deg(n(p)) = n$. $\square$

Proposition 7.3

$\overline{\deg}$ is injective.

Proof. A homomorphism of groups is injective if and only if its kernel is trivial, so it suffices to show $\overline{\deg}([D]) = 0$ forces $[D] = 0$. If $\overline{\deg}([D]) = 0$ then $\deg(D) = 0$, so $D \in \ker(\deg) = \text{Prin}(\mathbb{CP}^1)$ by Theorem 6.3, and hence $[D] = 0$ in the quotient. $\square$

Theorem 7.4

$\overline{\deg} : \text{Cl}(\mathbb{CP}^1) \rightarrow \mathbb{Z}$ is an isomorphism; in particular $\text{Cl}(\mathbb{CP}^1) \cong \mathbb{Z}$.

Proof. By Propositions 7.1, 7.2 and 7.3 it is a well-defined bijective group homomorphism. $\square$

7.2 The same computation, packaged

It is worth seeing that the three verifications above are exactly the content of one standard theorem, since this both shortens the argument and explains what made it work.

Proposition 7.5

$\text{Cl}(\mathbb{CP}^1) \cong \mathbb{Z}$.

Proof. By Theorem 6.3, $\text{Prin}(\mathbb{CP}^1) = \ker(\deg)$, so

$$\text{Cl}(\mathbb{CP}^1) = \frac{\text{Div}(\mathbb{CP}^1)}{\text{Prin}(\mathbb{CP}^1)} = \frac{\text{Div}(\mathbb{CP}^1)}{\ker(\deg)} \cong \text{im}(\deg)$$

by the First Isomorphism Theorem. Finally $\deg(n(p)) = n$ for any fixed p , so $\text{im}(\deg) = \mathbb{Z}$. $\square$

The point is that well-definedness, injectivity and surjectivity in Section 5.1 are precisely the three things the First Isomorphism Theorem checks in one step, once one knows $\text{Prin}(\mathbb{CP}^1) = \ker(\deg)$. All the mathematical content sits in Theorem 6.3.

7.3 Normal form and interpretation

Proposition 7.6

Every $D \in \text{Div}(\mathbb{CP}^1)$ satisfies $D \sim (\deg D)(\infty)$, and the integer $\deg D$ is the unique one with this property.

Proof. The divisor $D - (\deg D)(\infty)$ has degree $\deg(D) - \deg(D) = 0$, hence lies in $\ker(\deg) = \text{Prin}(\mathbb{CP}^1)$ by Theorem 6.3; that is, $D \sim (\deg D)(\infty)$. If also $D \sim n(\infty)$ then $n = \deg(n(\infty)) = \deg(D)$ by Proposition 7.1. $\square$

Remark 13. Note that Proposition 7.6 is a consequence of Theorem 6.3 and cannot be used in its proof.

So on $\mathbb{CP}^1$ a divisor class carries exactly one piece of information: an integer. Unwinding the definitions, this says something concrete about functions. A rational function is determined by its zeros and poles up to a constant factor (Proposition 5.2); a prescribed configuration of zeros and poles is realised by a function exactly when the multiplicities balance (Theorem 6.3); and what survives of a configuration after allowing oneself to modify it by any function at all is precisely its total multiplicity.

8 The Picard group

Divisors do not only govern functions; they also produce line bundles, and the group of line bundles is the geometric counterpart of the class group. In this section we compute it. We work throughout in the holomorphic (analytic) category: $\mathbb{CP}^1$ is a Riemann surface and all bundles, sections and transition functions are holomorphic. See Remark 14 for the algebraic counterpart.

8.1 Line bundles by transition functions

Definition 14

A holomorphic line bundle on a Riemann surface X is a holomorphic vector bundle $L \rightarrow X$ of rank 1. The set $\text{Pic}(X)$ of isomorphism classes of holomorphic line bundles is a group under tensor product: the identity is the class of the trivial bundle $\mathcal{O} = X \times \mathbb{C}$, and the inverse of $[L]$ is the class of the dual bundle $L^\vee$, since $L \otimes L^\vee \cong \mathcal{O}$.

Concretely, a line bundle is described by trivialising it over an open cover and recording how the trivialisations differ. We use only the standard cover $\mathbb{CP}^1 = U_0 \cup U_1$ of Section 3, on whose overlap $U_0 \cap U_1 = \mathbb{C}^\times$ we have the coordinate z .

Definition 15

Let $\mathcal{O}^*(V)$ denote the group, under multiplication, of nowhere-vanishing holomorphic functions on an open $V \subseteq \mathbb{CP}^1$. Given $g \in \mathcal{O}^*(\mathbb{C}^\times)$, let L_g be the line bundle obtained by glueing $U_0 \times \mathbb{C}$ to $U_1 \times \mathbb{C}$ along the overlap via $(z, \xi_1) \mapsto (z, g(z)\xi_1)$. We call g the *transition function* of L_g relative to this cover.

Lemma 8.1

$L_g \cong L_{g'}$ if and only if there are $h_0 \in \mathcal{O}^*(U_0)$ and $h_1 \in \mathcal{O}^*(U_1)$ with $g' = h_0 g h_1^{-1}$ on $\mathbb{C}^\times$.
Moreover $L_g \otimes L_{g'} \cong L_{gg'}$.

Proof. An isomorphism $L_g \rightarrow L_{g'}$ is precisely a pair of nowhere-vanishing holomorphic functions h_i on U_i ($i = 0, 1$) — the two trivialised descriptions of the isomorphism — compatible on the overlap, and compatibility reads $g' = h_0 g h_1^{-1}$. The tensor statement holds because tensoring multiplies transition functions. $\square$

We shall use one imported fact, the only result in this paper not proved here.

Fact 8.2 ([2, §30]). Every holomorphic line bundle on $\mathbb{CP}^1$ admits trivialisations over U_0 and over U_1 , and is therefore isomorphic to L_g for some $g \in \mathcal{O}^*(\mathbb{C}^\times)$.

This holds because each U_i is biholomorphic to $\mathbb{C}$, and every holomorphic line bundle on $\mathbb{C}$ is trivial. We isolate it as an explicit hypothesis rather than absorbing it silently.

8.2 Winding numbers and the classification of transition functions**Definition 16**

For $g \in \mathcal{O}^*(\mathbb{C}^\times)$ the *winding number* is

$$\text{wind}(g) := \frac{1}{2\pi i} \oint_{|z|=1} \frac{g'(z)}{g(z)} dz.$$

It is an integer — the winding number of the loop $g(e^{i\theta})$ about the origin — and $\text{wind}(gh) = \text{wind}(g) + \text{wind}(h)$, since $(gh)'/(gh) = g'/g + h'/h$. Note $\text{wind}(z^n) = n$.

Lemma 8.3

If $h_0 \in \mathcal{O}^*(U_0)$ and $h_1 \in \mathcal{O}^*(U_1)$ then $\text{wind}(h_0) = \text{wind}(h_1) = 0$.

Proof. h_0 is holomorphic and nowhere zero on $U_0 = \mathbb{C}$, so h'_0/h_0 is holomorphic on the disc $|z| \leq 1$ and its integral over the boundary vanishes by Cauchy's theorem. For h_1 , pass to the coordinate $w = 1/z$: the function $w \mapsto h_1(1/w)$ is holomorphic and nowhere zero on $U_1 \cong \mathbb{C}$, so by the same argument $\oint_{|w|=1} \frac{d}{dw} \log h_1 dw = 0$; substituting $w = 1/z$ changes the integral by a sign, so $\text{wind}(h_1) = 0$ as well. $\square$

Lemma 8.4: Factorisation of transition functions

Every $g \in \mathcal{O}^*(\mathbb{C}^\times)$ can be written as

$$g(z) = z^n h_0(z) h_1(z)^{-1}, \quad n = \text{wind}(g),$$

with $h_0 \in \mathcal{O}^*(U_0)$ and $h_1 \in \mathcal{O}^*(U_1)$.

Proof. Put $n := \text{wind}(g)$ and $G(z) := z^{-n}g(z)$, so that $\text{wind}(G) = \text{wind}(g) - n = 0$. The function G'/G is holomorphic on $\mathbb{C}^\times$, and $\mathbb{C}^\times$ has the homotopy type of a circle, so G'/G admits a primitive on $\mathbb{C}^\times$ precisely when its integral over $|z| = 1$ vanishes [1, Ch. 4] — which is exactly the statement $\text{wind}(G) = 0$. Let φ be such a primitive. Then $(Ge^{-\varphi})' = e^{-\varphi}(G' - G\varphi') = 0$, so

$G = \alpha e^\varphi$ for a constant $\alpha \in \mathbb{C}^\times$; absorbing α into φ (replace φ by $\varphi + \log \alpha$ for any choice of logarithm) we may assume $G = e^\varphi$.

Now expand φ in a Laurent series on $\mathbb{C}^\times$ (Theorem 2.3), $\varphi(z) = \sum_{k \in \mathbb{Z}} a_k z^k$, and split it into its nonnegative and negative parts,

$$\varphi_0(z) := \sum_{k \geq 0} a_k z^k, \quad \varphi_1(z) := \sum_{k < 0} a_k z^k = \sum_{j > 0} a_{-j} w^j.$$

The series φ_0 converges on all of $\mathbb{C}$, hence is holomorphic on U_0 ; the series φ_1 , viewed in the coordinate $w = 1/z$, is a power series in w with no constant term, converging on all of $\mathbb{C}$, hence holomorphic on U_1 . On the overlap $\varphi = \varphi_0 + \varphi_1$. Setting $h_0 := e^{\varphi_0} \in \mathcal{O}^*(U_0)$ and $h_1 := e^{-\varphi_1} \in \mathcal{O}^*(U_1)$ gives $G = e^{\varphi_0} e^{\varphi_1} = h_0 h_1^{-1}$ and therefore $g = z^n h_0 h_1^{-1}$. $\square$

Theorem 8.5

The map $\mathbb{Z} \rightarrow \text{Pic}(\mathbb{CP}^1)$, $n \mapsto [L_{z^n}]$, is a group isomorphism.

Proof. It is a homomorphism by the tensor statement of Lemma 8.1, since $z^m z^n = z^{m+n}$.

Surjectivity. By Fact 8.2 every line bundle is L_g for some $g \in \mathcal{O}^*(\mathbb{C}^\times)$. By Lemma 8.4, $g = z^n h_0 h_1^{-1}$ with $n = \text{wind}(g)$, so $L_g \cong L_{z^n}$ by Lemma 8.1.

Injectivity. Suppose $L_{z^m} \cong L_{z^n}$. By Lemma 8.1 there are $h_i \in \mathcal{O}^*(U_i)$ with $z^m = h_0 z^n h_1^{-1}$. Taking winding numbers and using Lemma 8.3, $m = \text{wind}(h_0) + n - \text{wind}(h_1) = n$. $\square$

8.3 The line bundle of a divisor

Definition 17

Let $D \in \text{Div}(\mathbb{CP}^1)$. For open $U \subseteq \mathbb{CP}^1$ set

$$\mathcal{O}(D)(U) := \{f \in \mathbb{C}(z)^\times : (\text{div}(f) + D)|_U \geq 0\} \cup \{0\},$$

using the notation of Definition 8. Explicitly, f belongs to $\mathcal{O}(D)(U)$ if and only if $\text{ord}_p(f) + n_p \geq 0$ for every $p \in U$, that is, if f is allowed a pole of order at most n_p at p when $n_p > 0$ and is required to vanish to order at least $-n_p$ when $n_p < 0$. The zero function is adjoined separately because $\text{div}(0)$ is undefined.

Proposition 8.6

$\mathcal{O}(D)$ is a line bundle: every $p \in \mathbb{CP}^1$ has a neighbourhood V on which $\mathcal{O}(D)$ is free of rank one over the holomorphic functions, with generator $t_p^{-n_p}$.

Proof. Choose $V \ni p$ meeting $\text{supp}(D)$ only in p , possible since $\text{supp}(D)$ is finite. For f meromorphic on V the condition $(\text{div}(f) + D)|_V \geq 0$ reads $\text{ord}_p(f) \geq -n_p$ together with $\text{ord}_q(f) \geq 0$ for $q \in V \setminus \{p\}$; by Propositions 4.1 and 4.2 this holds if and only if $t_p^{n_p} f$ is holomorphic on V . Hence $\mathcal{O}(D)(V) = t_p^{-n_p} \cdot \mathcal{O}(V)$, as claimed. $\square$

Proposition 8.7

If $D \sim E$ then $\mathcal{O}(D) \cong \mathcal{O}(E)$. In particular, if D is principal then $\mathcal{O}(D) \cong \mathcal{O}$.

Proof. Let $D - E = \text{div}(f)$. For any open U and any $g \in \mathcal{O}(D)(U)$,

$$\text{div}(gf) + E = \text{div}(g) + \text{div}(f) + E = \text{div}(g) + (D - E) + E = \text{div}(g) + D,$$

which is ≥ 0 on U by hypothesis; hence multiplication by f maps $\mathcal{O}(D)(U)$ into $\mathcal{O}(E)(U)$. Multiplication by f^{-1} is an inverse map, so this is an isomorphism, compatible with restriction and with multiplication by holomorphic functions. Taking $E = 0$ gives the last statement, since $\mathcal{O}(0) = \mathcal{O}$. $\square$

Proposition 8.8

For $n \in \mathbb{Z}$ we have $\mathcal{O}(n(\infty)) \cong L_{z^n}$.

Proof. Apply Proposition 8.6 with the cover $\{U_0, U_1\}$, which is legitimate because $\text{supp}(n(\infty)) = \{\infty\}$ meets U_0 not at all and U_1 only in ∞ . Over U_0 the condition is that f be holomorphic, so $\mathcal{O}(n(\infty))(U_0) = \mathcal{O}(U_0)$ with generator $e_0 = 1$. Over U_1 the generator is $t_\infty^{-n} = z^n$, so $e_1 = z^n$.

A section f over the overlap has coordinates ξ_0, ξ_1 determined by $f = \xi_0 e_0 = \xi_1 e_1$, i.e. $\xi_0 = f$ and $\xi_1 = fz^{-n}$. Hence $\xi_0 = z^n \xi_1$, so the transition function relative to $\{U_0, U_1\}$ is z^n , and $\mathcal{O}(n(\infty)) \cong L_{z^n}$ by Definition 15. $\square$

8.4 The Picard group of $\mathbb{CP}^1$

Theorem 8.9

The assignment $D \mapsto \mathcal{O}(D)$ induces a group isomorphism

$$\text{Cl}(\mathbb{CP}^1) \xrightarrow{\sim} \text{Pic}(\mathbb{CP}^1),$$

and consequently $\text{Pic}(\mathbb{CP}^1) \cong \mathbb{Z}$. Writing $\mathcal{O}(n) := \mathcal{O}(n(\infty))$, every holomorphic line bundle on $\mathbb{CP}^1$ is isomorphic to $\mathcal{O}(n)$ for a unique $n \in \mathbb{Z}$, namely $n = \text{wind}$ of its transition function.

Proof. The assignment is well defined on classes by Proposition 8.7 and lands in $\text{Pic}(\mathbb{CP}^1)$ by Proposition 8.6. Consider the diagram of maps

$$\text{Cl}(\mathbb{CP}^1) \xrightarrow{[D] \mapsto [\mathcal{O}(D)]} \text{Pic}(\mathbb{CP}^1) \xleftarrow{n \mapsto [L_{z^n}]} \mathbb{Z}.$$

By Proposition 7.6 every class satisfies $[D] = [(\deg D)(\infty)]$, so by Proposition 8.8 the left-hand map sends $[D]$ to $[L_{z^{\deg D}}]$; that is, it equals the composite of $\overline{\deg} : \text{Cl}(\mathbb{CP}^1) \rightarrow \mathbb{Z}$ with $n \mapsto [L_{z^n}]$. The first factor is an isomorphism by Theorem 7.4 and the second by Theorem 8.5, so the left-hand map is an isomorphism as well. In particular it is a homomorphism, and $\text{Pic}(\mathbb{CP}^1) \cong \text{Cl}(\mathbb{CP}^1) \cong \mathbb{Z}$. The final statement is Theorem 8.5 combined with Proposition 8.8. $\square$

Thus line bundles on $\mathbb{CP}^1$, like divisor classes, are classified by a single integer, and that integer is the degree.

Remark 14 (The algebraic counterpart). We have worked analytically throughout: $\text{Pic}(\mathbb{CP}^1)$ was the group of *holomorphic* line bundles on a complex manifold. In algebraic geometry one defines Pic as the group of invertible sheaves on a scheme, and proves $\text{Cl}(X) \cong \text{Pic}(X)$ for a

noetherian, integral, separated, locally factorial scheme [5, Cor. II.6.16]; for $X = \mathbb{P}_{\mathbb{C}}^1$ this gives $\text{Pic}(\mathbb{P}_{\mathbb{C}}^1) \cong \mathbb{Z}$ as well [5, Prop. II.6.4], [8, Tag 0BXJ]. That the algebraic and analytic answers agree is not a formality: it is an instance of Serre’s GAGA theorems [7], which compare coherent sheaves on a projective variety with their analytic counterparts. We have avoided relying on this comparison by computing the analytic group directly.

9 Summary

We began with the counting statement contained in the Fundamental Theorem of Algebra and asked for its analogue for meromorphic functions. Making the question precise required two things: a space on which infinity is an ordinary point, which is $\mathbb{CP}^1$; and a device recording all local multiplicities at once, which is the divisor.

Local analysis (Section 1) showed that each point contributes a single integer. Section 3 constructed $\mathbb{CP}^1$ and identified its meromorphic functions with $\mathbb{C}(z)$. Section 4 assembled the local integers into the homomorphism $\text{div} : \mathbb{C}(z)^{\times} \rightarrow \text{Div}(\mathbb{CP}^1)$, whose kernel is $\mathbb{C}^{\times}$ and whose image is $\text{Prin}(\mathbb{CP}^1)$; these facts are the two exact sequences of Section 5. The central result, Theorem 6.3, is that $\text{Prin}(\mathbb{CP}^1) = \ker(\deg)$: the balance of zeros and poles is the only obstruction. From it we obtained $\text{Cl}(\mathbb{CP}^1) \cong \mathbb{Z}$ (Theorem 7.4) and, by an explicit computation with transition functions on the two standard charts, $\text{Pic}(\mathbb{CP}^1) \cong \mathbb{Z}$ (Theorem 8.9).

As explained in Remark 12, all of this is the genus-zero case of a theory in which the answers are far richer.

10 Future directions

Higher genus and Jacobians. On a compact Riemann surface X of genus $g \geq 1$ the equality $\text{Prin}(X) = \ker(\deg)$ fails, and the degree-zero class group is the Jacobian $\text{Jac}(X) \cong \mathbb{C}^g / \Lambda$, a complex torus. Abel’s theorem characterises which degree-zero divisors are principal in terms of the vanishing of a period integral, and Jacobi’s inversion theorem identifies the image; together they are the correct generalisation of Theorem 6.3. See Miranda [6, Ch. VIII].

Riemann–Roch. Theorem 6.3 may be read as a statement about the dimension $\ell(D)$ of the space of global sections of $\mathcal{O}(D)$: on $\mathbb{CP}^1$ one has $\ell(D) = \deg D + 1$ for $\deg D \geq 0$ and $\ell(D) = 0$ otherwise. The Riemann–Roch theorem, $\ell(D) - \ell(K - D) = \deg D + 1 - g$, contains this as the case $g = 0$ and is the natural next result to study.

Vector bundles and cohomology. Grothendieck’s splitting theorem states that every holomorphic vector bundle on $\mathbb{CP}^1$ is a direct sum $\mathcal{O}(n_1) \oplus \cdots \oplus \mathcal{O}(n_r)$; Theorem 8.9 is the case $r = 1$. In the sheaf-theoretic language, $\text{Pic}(X) \cong H^1(X, \mathcal{O}^*)$, and Lemma 8.4 is the computation of that cohomology group for $\mathbb{CP}^1$ by hand using the two-set cover.

References

- [1] L. V. Ahlfors. *Complex Analysis*. McGraw–Hill, 3 edition, 1979.
- [2] O. Forster. *Lectures on Riemann Surfaces*, volume 81 of *Graduate Texts in Mathematics*. Springer, 1981.

- [3] E. Freitag and R. Busam. *Complex Analysis*. Springer, 2 edition, 2009.
- [4] P. Griffiths and J. Harris. *Principles of Algebraic Geometry*. Wiley, 1978.
- [5] R. Hartshorne. *Algebraic Geometry*, volume 52 of *Graduate Texts in Mathematics*. Springer, 1977.
- [6] R. Miranda. *Algebraic Curves and Riemann Surfaces*, volume 5 of *Graduate Studies in Mathematics*. American Mathematical Society, 1995.
- [7] J.-P. Serre. Géométrie algébrique et géométrie analytique. *Annales de l'Institut Fourier*, 6:1–42, 1956.
- [8] The Stacks Project Authors. The stacks project. Tag 0BXJ.

Reaching the Impossible Agreement

Taha Hbirri

CS'27

Abstract. Reaching agreement in a distributed system is a fundamental problem in computer science. We study the consensus problem, in which each process proposes a value and all correct processes must decide on a common value satisfying agreement, validity, and termination. The central result of this paper is the Fischer–Lynch–Paterson (FLP) impossibility theorem, which states that no deterministic algorithm can solve consensus in an asynchronous message-passing system tolerating even a single crash failure. We present the full proof. We then study Paxos which, under partial synchrony, can achieve consensus through a two-phase proposal protocol followed by a learning phase; its safety guarantee reduces to a single combinatorial invariant. Notably, this safety guarantee holds even without partial synchrony, which is needed only to guarantee that the protocol eventually terminates. We prove this safety property.

1 Preliminaries

This paper assumes familiarity with basic set theory, mathematical induction, and proof by contradiction. We also assume elementary graph-theory vocabulary (vertices, edges, paths, connectivity), at the level covered in a first discrete mathematics course. No prior knowledge of distributed systems or computer networking is required.

1.1 Distributed Networks as Graphs

Definition 1: Distributed Network
A <i>distributed system</i> is modelled as a directed graph $G = (V, E)$ where V is a finite set of <i>nodes</i> (processes, computers, phones) and $E \subseteq V \times V$ is a set of directed communication links, with no self-loops. The graph is called <i>undirected</i> when E is symmetric, i.e. $(u, v) \in E$ if and only if $(v, u) \in E$.

Three topologies recur throughout this paper: a **ring** (a connected graph in which each node has exactly two neighbours), a **tree** (connected and acyclic, $n - 1$ edges), and an **arbitrary connected graph** (the structure of the Internet itself).

Definition 2: Degree, Connectivity, Diameter

For an undirected graph, the *degree* of node v is $\deg(v) = |\{u : (v, u) \in E\}|$. A graph is *connected* if every pair of nodes is joined by a path. The *diameter* of G is the maximum shortest-path distance over all pairs.

1.2 Transition Systems**Definition 3: Transition System**

A *transition system* is a triple $(C, \rightarrow, I)$ where C is a set of *configurations* (global states), $\rightarrow \subseteq C \times C$ is a *transition relation*, and $I \subseteq C$ is a set of *initial configurations*. An *execution* is a sequence $\gamma_0, \gamma_1, \dots$ where $\gamma_0 \in I$ and $\gamma_i \rightarrow \gamma_{i+1}$ for all i .

Definition 4: Safety and Liveness

A *safety* property asserts that “something bad never happens.” A *liveness* property asserts that “something good eventually happens.” Correctness of a distributed algorithm requires proving both.

1.3 Synchrony Models

The behaviour of a distributed algorithm depends critically on what assumptions we make about message delays. We distinguish three models:

Definition 5: Synchronous Model

A distributed system is *synchronous* if there exist known upper bounds Δ on message delivery time and Φ on relative process speeds (or, equivalently for this discussion, on the time required for a process to take a step). Every message sent is guaranteed to arrive within Δ time units, and a process that is silent for sufficiently long can therefore be distinguished from one that is merely slow.

Definition 6: Asynchronous Model

A distributed system is *asynchronous* if there is no bound on message delivery time or relative process speed. Communication channels are reliable (messages are not lost), but delivery order is arbitrary and messages may be delayed by arbitrarily large finite amounts in an admissible execution. Crucially, a node cannot distinguish a slow process from a crashed one.

Definition 7: Partially Synchronous Model

A system is *partially synchronous* if there are bounds on message delay and relative process speed, but these bounds are either unknown in advance or are guaranteed only after some unknown global stabilisation time. This model lies between synchrony and asynchrony and is the timing assumption used by many practical consensus protocols [10].

The asynchronous model makes the fewest timing assumptions: it gives the adversarial

scheduler the most freedom. An algorithm that is correct under asynchrony is therefore also correct under stronger timing assumptions. This is why proving impossibility in the asynchronous model gives a particularly robust barrier; the FLP theorem is proved there precisely to maximise its generality.

1.4 Logical Time

In a distributed system every node has its own clock, and physical clocks drift. Lamport [16] proposed assigning a *logical timestamp* $C(e) \in \mathbb{N}$ to every event e via a simple counter rule: increment on every local event or send; on receive, take the maximum of local and incoming counter, then increment. The key property is:

Proposition 1: Clock Condition [16]
--

If event a causally precedes event b , then $C(a) < C(b)$.

Vector clocks strengthen this property to a biconditional with respect to the happened-before relation: for events a and b , a happened before b if and only if the vector timestamp of a is componentwise no greater than that of b and strictly smaller in at least one component. This enables systems to detect concurrent events and writes. Proofs can be found in [11, 16].

2 Motivation: Why Agreement Is Hard

Everyone has been in a group chat where messages appear out of order, where a reply appears before the message it answers, or where three people start planning three different things simultaneously because nobody knows who is in charge. That everyday chaos is not merely annoying. In distributed systems, this kind of uncertainty can make agreement impossible under the wrong timing assumptions.

Every digital system we rely on confronts this problem at scale. WhatsApp routes messages among billions of phones with no single server seeing every conversation. Netflix serves video from thousands of globally distributed caches that must agree on content availability. Google's search index is partitioned across hundreds of thousands of machines that coordinate results in milliseconds. In none of these systems is there a single omniscient boss.

Example 1 (The Restaurant Problem). Five friends choose a restaurant via group chat. Each proposes one option. They must all commit to *exactly* one choice. Now imagine one friend's phone dies mid-conversation. The remaining four must still agree, without knowing whether the fifth will come back online. Do they wait? Do they decide without her? Can they even tell the difference between "she has not replied yet" and "her phone is dead"?

This example captures the three core difficulties that make distributed agreement hard:

- (i) **No shared clock.** Nodes cannot tell the global order of events.
- (ii) **No shared memory.** Coordination requires messages, which take time to arrive and, in the model we use throughout this paper, may be arbitrarily but finitely delayed.
- (iii) **Failures are indistinguishable from slowness.** A node that has crashed looks identical to one that is merely slow. An asynchronous system, one with no bound on message delays, cannot tell the difference.

Point (iii) is the crux. It is the observation that drives the central impossibility result of this paper.

3 Literature and Problem Statement

The study of distributed coordination has a rich history. We summarise the key results we rely on, and then state the central problem of this paper.

Logical clocks and causal order. Lamport [16] introduced the happened-before relation and logical clocks; Fidge [11] and Mattern [18] independently extended this to vector clocks that fully characterise causality. We borrow these results in Section 1.4.

Global snapshots. Chandy and Lamport [6] showed how to record a consistent global state of a running distributed system using marker messages on FIFO channels, a result we use implicitly when reasoning about system states.

Leader election. A *leader election algorithm* is a distributed algorithm whose goal is to have all processes in a network agree on a single distinguished process—the *leader*—that will act as coordinator. Each process starts as a candidate and the algorithm terminates with exactly one process declaring itself leader and all others acknowledging it. Leader election is a weaker form of coordination than full consensus, yet solving it efficiently in large networks is already non-trivial. A related but distinct coordination primitive is *mutual exclusion*, in which processes contend for exclusive access to a shared resource rather than for a single distinguished identity; Ricart and Agrawala [21] gave a classic permission-based algorithm for this problem. This illustrates that distributed coordination includes tasks with different correctness conditions from leader election and consensus.

Chang and Roberts [7] gave an $\mathcal{O}(n^2)$ election algorithm on rings; Peterson [20] and Dolev, Klawe, and Rodeh [9] improved this to $\mathcal{O}(n \log n)$ worst-case. Gallager, Humblet, and Spira [13] gave a distributed algorithm for building a minimum-weight spanning tree in $\mathcal{O}(|E| + n \log n)$ messages; since the root of a spanning tree is a natural choice of leader, their algorithm also solves leader election as a by-product. These results establish that useful coordination primitives are achievable, which makes the following impossibility all the more striking.

The consensus problem. The central problem of this paper is:

Definition 8: Consensus

In the *consensus* problem, each process p_i starts with an input value $v_i \in \{0, 1\}$. We restrict to binary inputs; any finite set of values can be encoded as bit-strings, so a binary consensus primitive can in principle serve as a building block for deciding among any number of alternatives. We note, however, that this reduction from multi-valued to binary consensus is standard but not entirely free of subtlety: one must argue that repeated binary decisions can be composed without violating agreement or validity for the original multi-valued problem, which we do not develop here. The following properties must hold:

- (i) **Agreement:** All correct processes decide the same value d .
- (ii) **Validity:** The decided value d was proposed by some process (i.e. $d = v_i$ for some

i).

(iii) **Termination:** Every correct process eventually decides.

Validity as stated here is deliberately weak: it would, in isolation, permit degenerate rules such as always deciding the fixed input of a single distinguished process. We retain this weak form because the impossibility result proved in Section 5 does not require a stronger validity condition; proving impossibility under the weakest reasonable validity condition makes the theorem *stronger*, not weaker, since it rules out an even larger space of candidate algorithms.

Consensus is not merely a theoretical puzzle. Chandra and Toueg [5] showed that *atomic broadcast*—the property that all nodes deliver all messages in the same total order, meaning every node processes message number k before message number $k + 1$, and every node agrees on which message is number k —is reducible to consensus and, in the relevant asynchronous crash-failure model, consensus and atomic broadcast are computationally equivalent. Atomic broadcast is what makes distributed databases consistent: it is the reason your bank balance does not disagree between two ATMs.

Our goal. In this paper we prove the Fischer–Lynch–Paterson impossibility theorem (Theorem 2), which shows that consensus is impossible in the fully asynchronous model even with a single crash failure. We then study Paxos [17] as the canonical practical algorithm that circumvents this barrier under a weaker (but realistic) assumption.

4 The Main Result: FLP Impossibility

We work in the *asynchronous message-passing* model (see Section 1.3): processes communicate by sending reliable messages, there is no bound on message delivery time or relative process speed, and processes may fail by crashing (stopping permanently and silently). We assume $n \geq 2$ processes and that at most one process may crash. Unlike the ring, tree, and arbitrary-graph topologies discussed in Section 1.1, the result below assumes a *complete* communication topology: every pair of processes can exchange messages directly. This is the standard assumption in the original FLP setting. An algorithm for a sparse connected topology can be viewed as an algorithm for the complete topology that simply never uses the additional edges; therefore, if consensus were possible on a sparse topology, it would also be possible on the complete topology. Contrapositively, impossibility on the complete topology implies impossibility on the sparse topology.

Theorem 2: Fischer–Lynch–Paterson, 1985 [12]

There is no deterministic algorithm that, in every admissible execution of an asynchronous message-passing system of $n \geq 2$ processes in which at most one process crashes, satisfies agreement, validity, and termination.

Remark 1 (Significance). The FLP theorem is widely regarded as one of the most important results in the theory of distributed computing. It shows that, under the stated assumptions, agreement, validity, termination, and tolerance of one crash failure cannot all be guaranteed

simultaneously by a deterministic algorithm. Every practical consensus system (Paxos, Raft, ZooKeeper Atomic Broadcast) is, in a precise sense, an engineering response to this theorem.

5 Proof of the FLP Theorem

5.1 Setup and Key Definitions

Definition 9: Configuration

A *configuration* C consists of the local state of every process together with the set of messages in transit. An initial configuration has every process in its initial state, determined by its input value, and has no messages in transit. A transition consists of one atomic step by a process: the process either receives one pending message or receives no message (a null receive), then performs its deterministic local computation and sends any resulting messages. A decision, once made, is irrevocable. This instantiates the generic notion of a configuration from Definition 3: the global state C is, concretely, the pair (local process states, messages in transit), and the transition relation is given by delivering one message at a time.

Definition 10: Admissible Execution

An execution is *admissible* if at most one process crashes and every message sent to a non-faulty process is eventually delivered. The scheduler may choose the delivery order and may delay messages by arbitrarily large finite amounts.

Definition 11: Valency

A configuration C is *v-valent* (for $v \in \{0,1\}$) if every admissible execution from C eventually decides v . It is *bivalent* if there exist admissible executions from C that decide 0 and 1. Under the assumed termination property, every configuration is therefore either 0-valent, 1-valent, or bivalent.

Intuitively, a bivalent configuration is one from which the scheduler can still steer the execution toward either decision value; the algorithm has not yet committed to a unique outcome.

5.2 The Proof

Proof of Theorem 2. Assume for contradiction that a deterministic consensus algorithm $\mathcal{A}$ exists that tolerates one crash failure in an asynchronous system.

Step 1: There exists a bivalent initial configuration.

Suppose for contradiction that every initial configuration is univalent. Consider the set of all possible initial configurations parameterised by process input values $v_1, \dots, v_n \in \{0,1\}$. By validity, the all-0 configuration is 0-valent and the all-1 configuration is 1-valent. Changing inputs one process at a time, there must be two adjacent configurations C_0 (0-valent) and C_1 (1-valent) that differ only in the input of a single process p .

Now consider an admissible execution in which p crashes before taking any step. By the assumed crash tolerance and termination property, the remaining processes eventually decide. Their local states and received messages are identical whether the initial configuration was C_0 or C_1 : their own inputs are identical in both configurations, and p is silent from the start. Because $\mathcal{A}$ is deterministic, the same sequence of steps by the surviving processes must therefore lead to the same decision in both executions. But C_0 is 0-valent and C_1 is 1-valent, so such a common decision is impossible. Hence our assumption was false, and *some initial configuration is bivalent*.

Step 2: From any bivalent configuration, a bivalent configuration is reachable after any applicable event.

Let C be a bivalent configuration and let $e = (p, m)$ be an event applicable at C . Let $\mathcal{E}$ denote the set of configurations reachable from C without applying e ; e remains applicable throughout $\mathcal{E}$ because messages may be delayed arbitrarily and applying other events does not remove m . Let $\mathcal{D} = e(\mathcal{E})$ be the set of configurations obtained by applying e to configurations in $\mathcal{E}$.

We repeatedly use the basic fact that *valency is preserved forward*: if a configuration is v -valent, then every configuration reachable from it is also v -valent. This follows because every admissible execution from the successor can be extended to an admissible execution from the original configuration.

Suppose, for contradiction, that every configuration in $\mathcal{D}$ is univalent. Because C is bivalent, there exists a 0-valent configuration and a 1-valent configuration reachable from C . For each $i \in \{0, 1\}$, choose an i -valent configuration E_i reachable from C . If $E_i \in \mathcal{E}$, then $F_i = e(E_i) \in \mathcal{D}$ is univalent and is reachable from E_i , so forward preservation implies that F_i is i -valent. Otherwise, e was applied somewhere on the path from C to E_i . Let $F_i \in \mathcal{D}$ be the configuration immediately after the first application of e on that path. Then E_i is reachable from F_i , and since F_i is univalent it must again be i -valent. Thus $\mathcal{D}$ contains both a 0-valent and a 1-valent configuration.

Call two configurations *neighbours* if one results from the other in a single event. By considering a finite path in $\mathcal{E}$ from a configuration leading to a 0-valent member of $\mathcal{D}$ to one leading to a 1-valent member, there exist neighbouring configurations $C_0, C_1 \in \mathcal{E}$ such that

$$D_i = e(C_i)$$

is i -valent for $i \in \{0, 1\}$. Without loss of generality, let $C_1 = e'(C_0)$, where $e' = (p', m')$.

Case A: $p' \neq p$. The events e and e' affect different processes and therefore commute:

$$e(e'(C_0)) = e'(e(C_0)).$$

Hence

$$D_1 = e'(D_0).$$

But D_0 is 0-valent, so every configuration reachable from D_0 , including D_1 , must also be 0-valent. This contradicts the fact that D_1 is 1-valent.

Case B: $p' = p$. Consider a finite deciding execution from C_0 in which process p takes no steps;

such an execution exists by the assumed tolerance of one crash failure and termination. Let σ be its event sequence and let $A = \sigma(C_0)$. Because σ contains no event of p , the sequence σ can also be applied after either $D_0 = e(C_0)$ or $D_1 = e(C_1)$. Since D_0 is 0-valent and D_1 is 1-valent,

$$E_0 = \sigma(D_0)$$

is 0-valent and

$$E_1 = \sigma(D_1)$$

is 1-valent. Moreover, by commutativity with the events of σ ,

$$E_0 = e(A)$$

and, because $C_1 = e'(C_0)$,

$$E_1 = e(e'(A)).$$

Thus from A one can reach both a 0-valent and a 1-valent configuration. Hence A is bivalent. But A is reached by a deciding execution from C_0 , so A already has a decision value and is therefore univalent. This is a contradiction.

In both cases we obtain a contradiction. Therefore $\mathcal{D}$ contains a bivalent configuration. Equivalently, after any applicable event e , there is a finite execution fragment ending with e whose final configuration is bivalent. This is the central bivalency-preservation lemma of Fischer, Lynch, and Paterson [12].

Step 3: Constructing an infinite non-deciding execution.

Start from the bivalent initial configuration guaranteed by Step 1. Maintain a fixed fair ordering of the processes, and at each round select the next process p in that ordering. Let m be the oldest message currently pending for p , if one exists; otherwise let $m = \perp$ denote a null receive event in which p takes a step without receiving a message. Let $e = (p, m)$.

By Step 2, there is a finite execution fragment that avoids applying e until its final event and ends with a bivalent configuration after applying e . Use this fragment as the current round. Since e is the final event of the round, the selected process makes progress at every round, while the final configuration remains bivalent.

Repeat this construction indefinitely, cycling fairly through all processes. A process is selected infinitely often, and whenever a message is pending for that process, the oldest such message is eventually selected and delivered. Hence every message sent to a non-faulty process is eventually delivered. The construction contains no crashed process at all, so the resulting infinite execution is admissible.

Because every round ends in a bivalent configuration, no process can have reached an irrevocable decision: once a decision occurs, the configuration is univalent. Thus the constructed admissible execution never reaches a decision, contradicting the assumed termination property of $\mathcal{A}$. Therefore no such deterministic consensus algorithm exists. $\square$

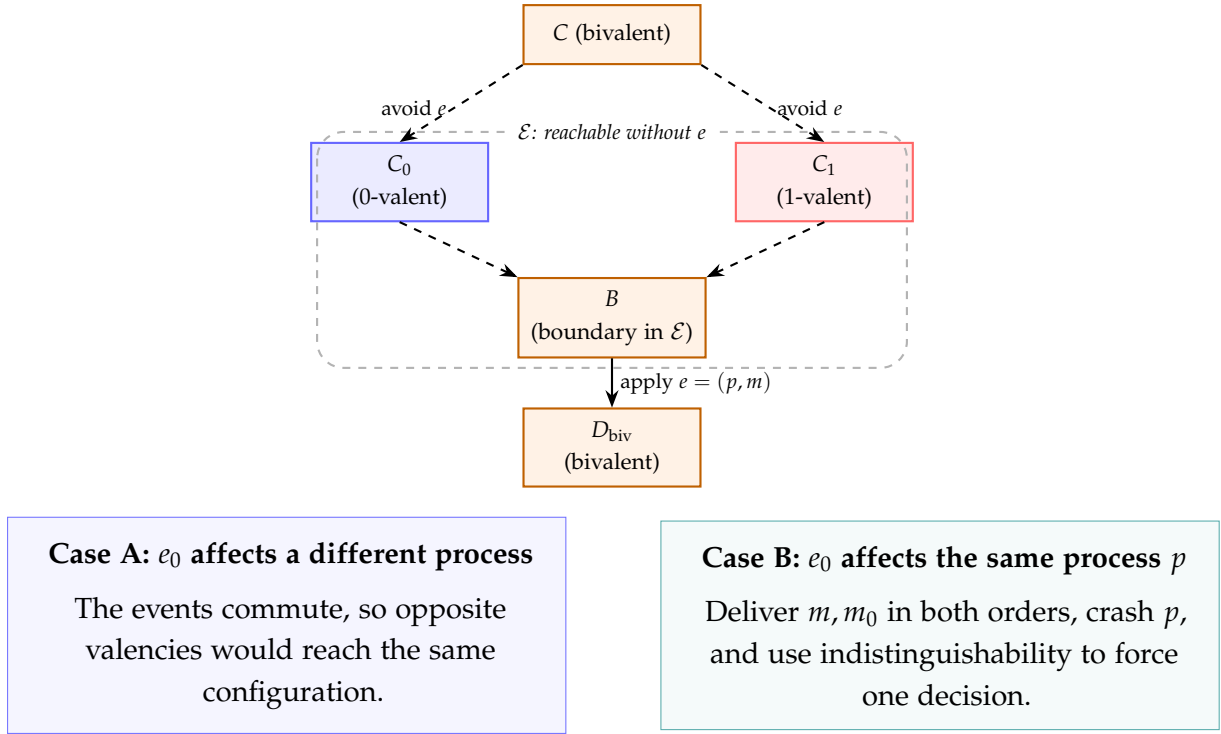


Figure 1: Illustration of Steps 1 and 2 of the proof.

5.3 What FLP Does Not Say

FLP does not say consensus is impossible in all settings. It applies specifically to *deterministic* algorithms in *fully asynchronous* systems. Practical systems circumvent the theorem in 3 ways:

- (i) **Randomisation.** Algorithms such as Ben-Or's use coin flips to break symmetry, achieving consensus with probability 1 in an asynchronous model. Because the algorithm is no longer deterministic, the FLP argument does not apply.
- (ii) **Partial synchrony.** If messages are eventually delivered within some (unknown) time bound, correct algorithms exist. This is the assumption under which Paxos operates.
- (iii) **Failure detectors.** Abstract oracles that provide (possibly imperfect) information about crashed processes. Chandra and Toueg [5] studied which failure-detector classes suffice for consensus, while Chandra, Hadzilacos, and Toueg [4] characterised the weakest failure detector for solving consensus in asynchronous systems with crash failures.

6 Paxos: Engineering Around Impossibility

6.1 The Setting

Paxos, introduced by Lamport [17], is designed to make progress under *partial synchrony*: the system is not always responsive, but eventually messages get through. The partial-synchrony model is due to Dwork, Lynch, and Stockmeyer [10]. It is important to separate two guarantees here. **Safety** (Theorem 4, proved in Section 6.3) holds unconditionally: it relies only on the quorum-intersection combinatorics of Lemma 3 and is not violated even in a fully asynchronous run with arbitrarily delayed messages. **Liveness** (terminating with a decision) is what requires partial synchrony: under indefinite asynchrony, the adversarial scheduler of Step 3 can always

find a way to delay messages forever and prevent termination, exactly as FLP predicts; partial synchrony is what eventually removes the adversary's ability to do so.

Roles. Processes play three (potentially overlapping) roles:

- **Proposers** suggest values.
- **Acceptors** vote on proposals and form quorums.
- **Learners** discover the chosen value.

Quorums.

Definition 12: Quorum

Let $\mathcal{A}$ be the set of all acceptors with $|\mathcal{A}| = n$. A *quorum* is any subset $Q \subseteq \mathcal{A}$ satisfying $|Q| > n/2$. Equivalently, a quorum is any strict majority of the acceptors. We denote by Ω the family of all quorums: $\Omega = \{Q \subseteq \mathcal{A} : |Q| > n/2\}$.

The key property is:

Lemma 3: Quorum Intersection

Any two quorums $Q_1, Q_2 \in \Omega$ share at least one acceptor, i.e. $Q_1 \cap Q_2 \neq \emptyset$.

Proof. By Definition 12, $|Q_1| > n/2$ and $|Q_2| > n/2$. If disjoint, then $|Q_1 \cup Q_2| = |Q_1| + |Q_2| > n$, which is impossible since $Q_1, Q_2 \subseteq \mathcal{A}$ and $|\mathcal{A}| = n$. Therefore $Q_1 \cap Q_2 \neq \emptyset$. $\square$

This simple combinatorial (pigeonhole) fact is the core of the safety guarantee of Paxos: any two quorums used by the protocol must share a witness acceptor, which is what prevents two different values from ever being chosen.

6.2 The Protocol

Algorithm 1 Paxos (Single-Decree)

- 1: **Phase 1a – Prepare:** Proposer selects a globally unique proposal number $n \in \mathbb{N}$ and sends $\text{prepare}(n)$ to a quorum of acceptors.
- 2: **Phase 1b – Promise:** On receiving $\text{prepare}(n)$: if n is greater than every proposal number for which the acceptor has already made a promise, the acceptor promises not to accept proposals numbered less than n and replies $\text{promise}(n, v_{\text{prev}}, n_{\text{prev}})$. Here n_{prev} is the proposal number of the highest-numbered proposal this acceptor has *itself* previously accepted, and v_{prev} is the value associated with that accepted proposal. Both fields belong to the acceptor's own local history, not to the current proposer. The acceptor replies $(n_{\text{prev}}, v_{\text{prev}}) = \perp$ if it has not yet accepted any proposal.
- 3: **Phase 2a – Accept:** On receiving promises from a quorum: the proposer sends $\text{accept}(n, v)$ to a quorum of acceptors, where v is the value v_{prev} paired with the highest reported n_{prev} , or the proposer's own value if every promise returned $\perp$.
- 4: **Phase 2b – Accepted:** On receiving $\text{accept}(n, v)$: acceptor accepts unless it has already promised a proposal number greater than n . If a quorum accepts (n, v) , the value v is *chosen*.
- 5: **Phase 3 – Learn:** Each acceptor that accepts (n, v) notifies the learners directly, or (more commonly) notifies a single distinguished acceptor which forwards the result to all learners once it detects a quorum has accepted. A learner outputs v once it has confirmation that a quorum accepted (n, v) for some n .

6.3 Safety of Paxos

Theorem 4: Safety

If value v is chosen with proposal number n , then every subsequently chosen value also equals v .

Proof. We proceed by strong induction on the globally unique proposal numbers. Suppose v was chosen with proposal number n , meaning a quorum Q_1 accepted (n, v) . We show that every proposal (n', v') with $n' > n$ must satisfy $v' = v$.

Base case ($n' = n + 1$, when such a proposal number is used). The proposer of n' collected promises from some quorum Q_2 . By Lemma 3, $Q_1 \cap Q_2 \neq \emptyset$; let $a \in Q_1 \cap Q_2$ be a shared acceptor. Since $a \in Q_1$, acceptor a previously accepted (n, v) , so it reports $(n_{\text{prev}}, v_{\text{prev}}) = (n, v)$ to the proposer of n' in Phase 1b. Moreover, an acceptor can accept a proposal numbered k only after it has responded to a prepare for k , thereby making a promise for at least k . Hence an acceptor that has promised n' cannot previously have accepted a proposal numbered at least n' . Therefore every accepted proposal reported by members of Q_2 has number less than n' . Because there is no proposal number strictly between n and $n' = n + 1$, the highest proposal number reported across the promises in Q_2 is at most n . Thus the highest reported accepted proposal is (n, v) , and the proposer sets $v' = v$.

Inductive step ($n' > n + 1$). Assume as the strong inductive hypothesis that every proposal (k, w) with $n < k < n'$ carries value $w = v$. The proposer of n' collected promises from some quorum Q_2 . Again choose $a \in Q_1 \cap Q_2$. Since a accepted (n, v) , at least one promise in Q_2 reports an accepted proposal with number at least n . Let k be the highest proposal number reported across all promises in Q_2 . By the promise argument above, $k < n'$.

If $k = n$, then the corresponding value is v . If $k > n$, then $k < n'$ and by the strong inductive hypothesis every proposal with number between n and n' carries value v . Hence in either case the value paired with the highest reported k is v , and the proposer sets $v' = v$.

In both cases $v' = v$, completing the induction. $\square$

Remark 2. Paxos does *not* guarantee liveness in the worst case: two competing proposers can continuously pre-empt each other. Practical deployments use a distinguished leader proposer to avoid this. This does not reintroduce a permanent single point of failure in the way a non-replicated coordinator would: if the leader itself crashes, the system detects this (typically via timeouts) and elects a new one, so the leader's failure causes a temporary liveness hiccup—a pause in progress—rather than a lasting outage, and safety is preserved throughout by Theorem 4 regardless of how many leaders come and go.

6.4 Real-World Deployments

Paxos and its derivatives are the backbone of modern distributed infrastructure:

- **Google Chubby** [2] uses Paxos-derived protocols for distributed locking, and **Google Spanner** [8] uses them for globally consistent transactions.
- **Apache ZooKeeper** [14] implements ZAB (ZooKeeper Atomic Broadcast), a Paxos variant, and underpins Kafka, Hadoop, and HBase.

- **etcd**, a widely deployed distributed key-value store, uses the Raft protocol [19]—a more understandable re-presentation of Paxos—to power the configuration management layer of large-scale cloud infrastructure.
- **Apache Cassandra** [15] uses a lightweight Paxos variant for linearisable writes.

Every time you send a message, make a payment, or deploy software in a cloud environment, you are relying on a system whose correctness ultimately rests on the safety proof of Theorem 4.

7 Future Work and Open Directions

This paper studied single-decree Paxos, which agrees on one value. Practical systems need to agree on a sequence of values (a log of commands). **Multi-Paxos** extends the protocol by using a stable leader who can skip Phase 1 for subsequent rounds, reducing message complexity. **Raft** [19] was designed as a more understandable alternative, making leader election and log replication explicit; it has become a widely used implementation choice in new systems.

Several fundamental questions remain active research areas:

- (i) **Byzantine fault tolerance.** The FLP model assumes crash failures (a node goes silent). Byzantine failures allow nodes to send arbitrary or malicious messages. Castro and Liskov’s PBFT [3] and, more recently, blockchain consensus protocols such as Tendermint and HotStuff [22] address this harder setting.
- (ii) **Optimal resilience.** Let f denote the number of processes an algorithm must tolerate as faulty (crashed, in the crash-failure model, or Byzantine, in the Byzantine-failure model). Once liveness is restored by appropriate timing assumptions, crash-tolerant consensus in the standard partially synchronous model requires $n \geq 2f + 1$, while Byzantine consensus in the standard unauthenticated synchronous or partially synchronous model requires $n \geq 3f + 1$. These classical resilience bounds are tight for those models; active work includes reducing message and time complexity and extending tight bounds to weaker or hybrid failure models (e.g. mixtures of crash and Byzantine faults, or networks with only partial connectivity).
- (iii) **Randomised consensus in practice.** Ben-Or’s algorithm [1] achieves consensus probabilistically in a fully asynchronous model, bypassing FLP. Modern protocols such as HotStuff [22] instead use a partially synchronous Byzantine-fault model and provide linear communication complexity; the broader frontier is to combine strong liveness and efficiency with weaker timing and failure assumptions.

8 Conclusion

The problem of reaching agreement in a distributed system sits at the intersection of theory and engineering. We have shown that the three properties of consensus (agreement, validity, and termination) cannot all be simultaneously guaranteed by any deterministic algorithm when the network is asynchronous and even a single process may crash. This is the content of the Fischer–Lynch–Paterson theorem, and its proof rests on a single elegant observation: in an asynchronous system, an adversarial scheduler can always keep the global configuration in a state of genuine uncertainty, because silence is indistinguishable from failure. Yet the

theorem does not close the door. Paxos demonstrates that weakening one assumption, replacing full asynchrony with partial synchrony, is enough to make consensus achievable. Its safety guarantee reduces to a combinatorial pigeonhole argument on quorum intersection, and that argument is the hidden backbone of every consistent distributed database deployed at scale today. The broader lesson is one of precision in assumptions. Impossibility results are not obstacles; they are maps. They tell us exactly which assumption to relax, and in doing so, they chart the design space for every practical system that follows.

References

- [1] M. Ben-Or. Another advantage of free choice: Completely asynchronous agreement protocols. In *Proc. 2nd ACM Symposium on Principles of Distributed Computing*, pages 27–30, 1983.
- [2] M. Burrows. The chubby lock service for loosely-coupled distributed systems. In *Proc. 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 335–350, 2006.
- [3] M. Castro and B. Liskov. Practical byzantine fault tolerance. In *Proc. 3rd USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 173–186, 1999.
- [4] T. D. Chandra, V. Hadzilacos, and S. Toueg. The weakest failure detector for solving consensus. *Journal of the ACM*, 43(4):685–722, 1996.
- [5] T. D. Chandra and S. Toueg. Unreliable failure detectors for reliable distributed systems. *Journal of the ACM*, 43(2):225–267, 1996.
- [6] K. M. Chandy and L. Lamport. Distributed snapshots: Determining global states of distributed systems. *ACM Transactions on Computer Systems*, 3(1):63–75, 1985.
- [7] E. J. H. Chang and R. Roberts. An improved algorithm for decentralised extrema-finding in circular configurations of processes. *Communications of the ACM*, 22(5):281–283, 1979.
- [8] J. C. Corbett et al. Spanner: Google’s globally-distributed database. In *Proc. 10th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 251–264, 2012.
- [9] D. Dolev, M. Klawe, and M. Rodeh. An $O(n \log n)$ unidirectional distributed algorithm for extrema finding in a circle. *Journal of Algorithms*, 3(3):245–260, 1982.
- [10] C. Dwork, N. A. Lynch, and L. Stockmeyer. Consensus in the presence of partial synchrony. *Journal of the ACM*, 35(2):288–323, 1988.
- [11] C. J. Fidge. Timestamps in message-passing systems that preserve the partial ordering. In *Proc. 11th Australian Computer Science Conference*, pages 56–66, 1988.
- [12] M. J. Fischer, N. A. Lynch, and M. S. Paterson. Impossibility of distributed consensus with one faulty process. *Journal of the ACM*, 32(2):374–382, 1985.

-
- [13] R. G. Gallager, P. A. Humblet, and P. M. Spira. A distributed algorithm for minimum-weight spanning trees. *ACM Transactions on Programming Languages and Systems*, 5(1):66–77, 1983.
 - [14] P. Hunt, M. Konar, F. P. Junqueira, and B. Reed. Zookeeper: Wait-free coordination for internet-scale systems. In *Proc. USENIX Annual Technical Conference*, 2010.
 - [15] A. Lakshman and P. Malik. Cassandra: A decentralized structured storage system. *ACM SIGOPS Operating Systems Review*, 44(2):35–40, 2010.
 - [16] L. Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM*, 21(7):558–565, 1978.
 - [17] L. Lamport. The part-time parliament. *ACM Transactions on Computer Systems*, 16(2):133–169, 1998.
 - [18] F. Mattern. Virtual time and global states of distributed systems. In *Proc. International Workshop on Parallel and Distributed Algorithms*, pages 215–226, 1989.
 - [19] D. Ongaro and J. Ousterhout. In search of an understandable consensus algorithm. In *Proc. USENIX Annual Technical Conference*, pages 305–320, 2014.
 - [20] G. L. Peterson. An $o(n \log n)$ unidirectional algorithm for the circular extrema problem. *ACM Transactions on Programming Languages and Systems*, 4(4):758–762, 1982.
 - [21] G. Ricart and A. K. Agrawala. An optimal algorithm for mutual exclusion in computer networks. *Communications of the ACM*, 24(1):9–17, 1981.
 - [22] M. Yin, D. Malkhi, M. K. Reiter, G. G. Gueta, and I. Abraham. Hotstuff: Bft consensus with linearity and responsiveness. In *Proc. ACM Symposium on Principles of Distributed Computing (PODC)*, pages 347–356, 2019.